

Hailo Imaging User Guide

Release 1.11.0
30 March 2026

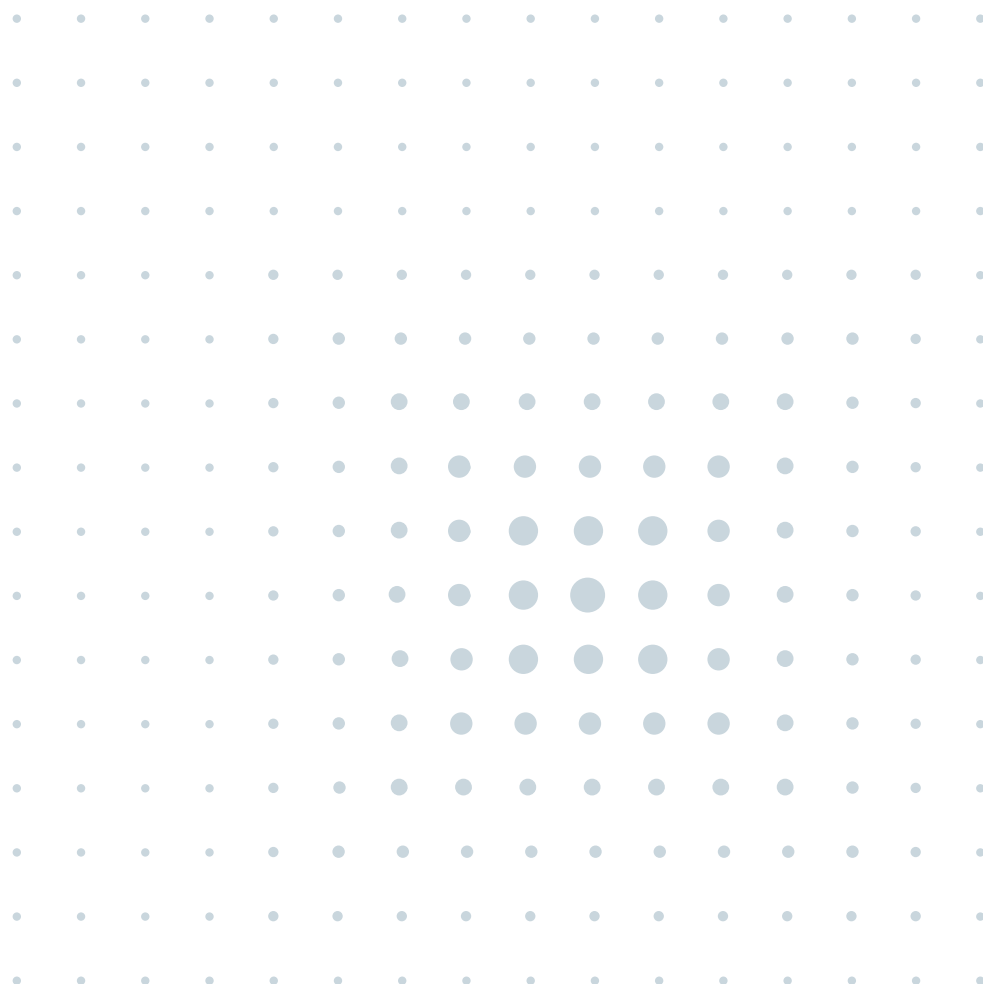


Table of Contents

I	User Guide	2
1	Overview	3
1.1	Introduction	3
1.2	Features	3
1.3	Software Architecture	3
1.4	V4L2 Devices	4
1.5	ISP Media Server	9
1.6	Video Encoder	10
2	Changelog	11
3	Imaging Algorithms Overview	16
3.1	Introduction	16
3.2	ISP Algorithms	16
3.3	Hailo Media Library Vision Algorithms	18
4	Tutorials	19
4.1	v4l_ctrl_example	19
4.2	v4l_event_handling_example	21
4.3	v4l_stream_example	23
4.4	HDR Manager Example Code	26
5	Sensor Porting Guide	30
5.1	Introduction	30
5.2	Prerequisites	30
5.3	System Overview	30
5.4	Sensor Driver Frameworks	32
5.5	Setting Up Yocto for Kernel Driver Development	32
5.6	Kernel Driver Development	35
5.7	ISS Driver Development	40
II	API Reference	49
6	V4L Control	50
6.1	AE	50
6.2	AWB	52
6.3	AF	54
6.4	2DNR	54
6.5	GC	55
6.6	WDR	56
6.7	EE	56
6.8	CPROC	58
6.9	CA	58
6.10	CAC	59
6.11	3DNR	60
6.12	BLS	60
6.13	DMSC	61
6.14	DPCC	66
6.15	DPF	66
6.16	LSC	67
6.17	DCI	69

7	Automatic Control Module Configuration	70
7.1	Introduction	70
7.2	3aconfig.json Structure	70
8	Automatic Control Parameters	75
8.1	AdaptiveAe	75
8.2	Priority Entry	76
8.3	HDRAdaptiveAE	76
8.4	Awbv2	77
8.5	Afv1	78
8.6	AGamma64	79
8.7	ACproc	79
8.8	Aeev1	79
8.9	AWdrv4	80
8.10	A3dnrv1	81
8.11	A2dnrv5	81
8.12	ADpf	82
8.13	ABIs	82
8.14	ADmscv2	83
8.15	ACprocPost	83
8.16	ALsc	85

Disclaimer and Proprietary Information Notice

Copyright

© 2026 Hailo Technologies Ltd ("Hailo"). All Rights Reserved.

No part of this document may be reproduced or transmitted in any form without the expressed, written permission of Hailo. Nothing contained in this document should be construed as granting any license or right to use proprietary information for that matter, without the written permission of Hailo.

This version of the document supersedes all previous versions.

General Notice

Hailo, to the fullest extent permitted by law, provides this document "as-is" and disclaims all warranties, either express or implied, statutory or otherwise, including but not limited to the implied warranties of merchantability, non-infringement of third parties' rights, and fitness for particular purpose.

Although Hailo used reasonable efforts to ensure the accuracy of the content of this document, it is possible that this document may contain technical inaccuracies or other errors. Hailo assumes no liability for any error in this document, and for damages, whether direct, indirect, incidental, consequential or otherwise, that may result from such error, including, but not limited to loss of data or profits.

The content in this document is subject to change without prior notice and Hailo reserves the right to make changes to content of this document without providing a notification to its users.

Part I

User Guide

1. Overview

1.1. Introduction

The Hailo Imaging software suite provides a complete software stack for managing image capture, processing, and encoding on the Hailo-15™ vision subsystem. It includes camera sensors, ISP, video pipelines, and hardware-accelerated encoding through standard Linux™ interfaces and Hailo-provided middleware. The library is designed to support low-level customization, enabling flexibility across a wide range of camera and imaging use cases. This document serves as a practical guide to help developers quickly and efficiently use the Imaging software suite.

1.2. Features

Hailo-15™ bolsters real-time high resolution video processing capabilities, including the following features:

- Up to 4K UHD resolution
- Up to 60 FPS
- Video4Linux2 (V4L2) integration
- Hailo Media Library API

1.3. Software Architecture

The Hailo Imaging software suite includes the following key components:

- Input/output Video Drivers – Sensor, ISP, and MIPI CSI drivers for camera input and image acquisition.
- V4L2 Device Drivers – Standard Linux Video4Linux2 (V4L2) interfaces for controlling and streaming video devices.
- ISP Media Server (Daemon) – A user-space daemon responsible for:
 - Image quality and ISP runtime management.
 - Camera sensor control.
 - 3A (Auto Exposure, Auto White Balance, Auto Focus)
 - Image quality statistics collection and tuning
- Encoder subsystem – Hardware-accelerated H.264 / H.265 video encoding, including:
 - Kernel driver
 - User-space low-level encoder library (high-level encoder library is part of the Hailo Media Library)
 - Integration with Hailo Media Library and GStreamer

Most applications are expected to use the Hailo Media Library APIs, which provides a high-level abstraction for video capture, ISP control, and encoding. Advanced users may directly access:

- V4L2 video devices for streaming
- V4L2 sub-devices for fine-grained sensor and ISP control
- Standard tools such as v4l2-tools, FFmpeg, or GStreamer

This dual-layer approach allows portability while retaining compatibility with standard Linux imaging frameworks.

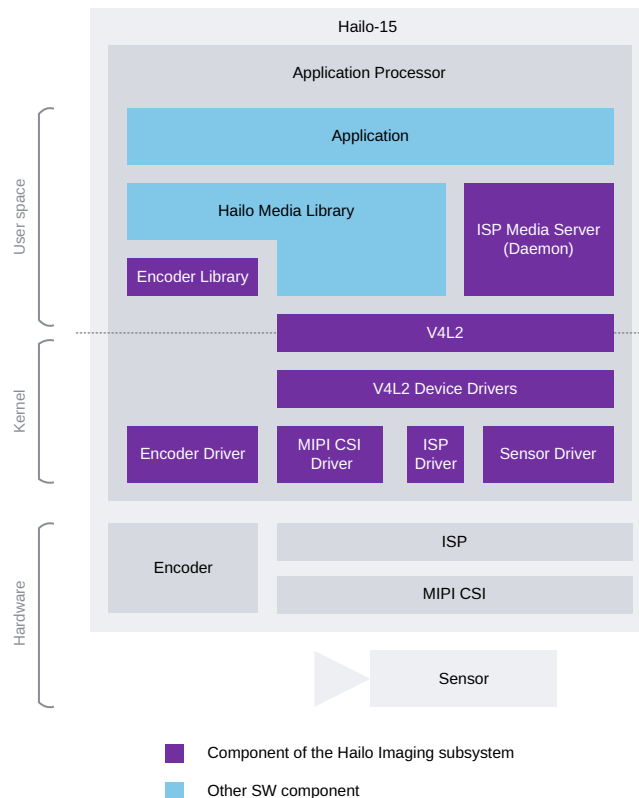


Figure 1. Hailo Imaging software components

1.4. V4L2 Devices

1.4.1. Overview

The Hailo-15™ vision subsystem is compatible with the standard Linux® API for video devices, Video4Linux2 (V4L2). V4L2 abstracts the vision hardware and uses a set of system calls to the operating system to communicate with device drivers and issue commands to the hardware. It is comprised of kernel space and user space modules.

The [V4L2 software architecture](#) diagram describes how the Hailo Linux distribution provides integration between V4L2 and the Vision subsystem. Linux V4L represents hardware and subcomponents through the file system using the /dev/video* logical files. Access and control of the vision subsystem is performed through a standard file interface (open, close, ioctl etc).

There are two types of devices:

- Video Device – logical representation mostly of a video producer or consumer device. These are the main and for the most part, the only interaction points with the user application. Different files can represent different hardware or a hardware data path.
- V4L Sub-device – logical representation of internal hardware and mechanisms. The user may use these to fine tune or configure specific or unique features. For the most part of the process, the user does not need to directly use these device files.

The Hailo-15 V4L logical topology is further divided into the following sub-devices and video devices:

- V4L2-sub devices
 - imx678 (or any other camera sensor)
 - cdns_csi2rx
 - hailo-pixel-mux

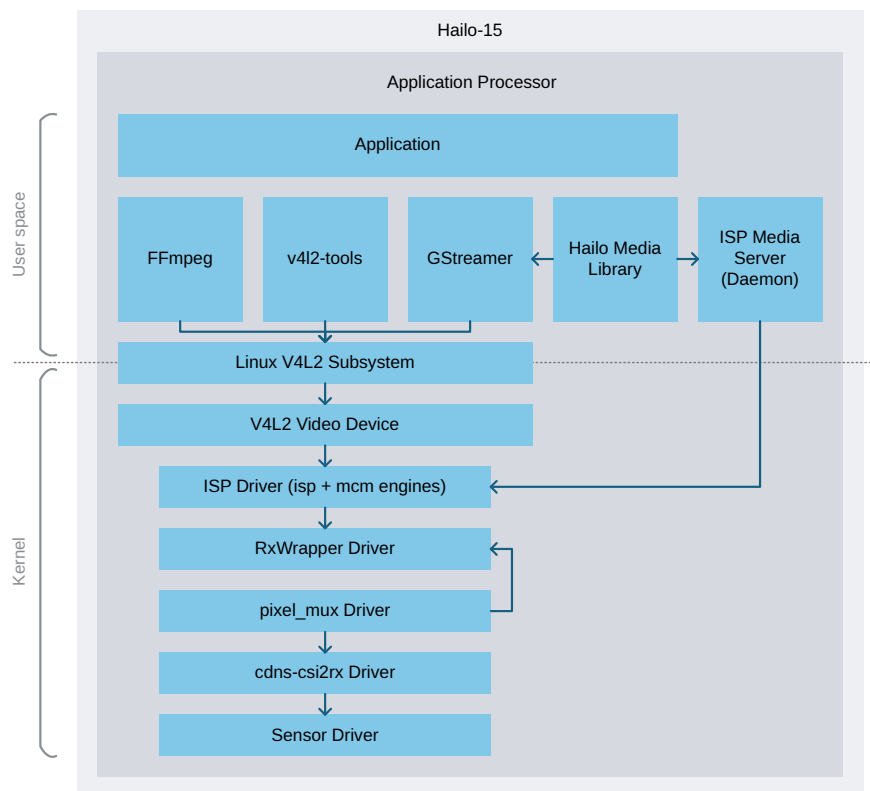


Figure 2. V4L2 software architecture. The arrows represent calls, as a part of the control and data flows.

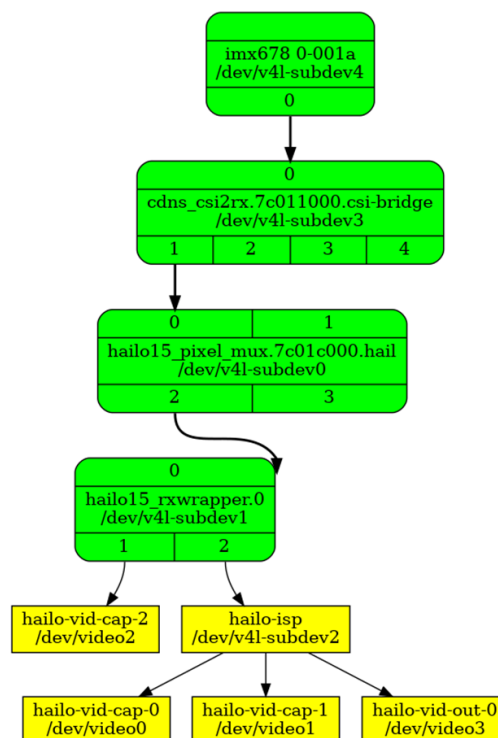


Figure 3. V4L2 logical topology

- hailo-rxwrapper
- isp
- Video Devices
 - hailo-vid-cap-0
 - hailo-vid-cap-1
 - hailo-vid-cap-2
 - hailo-vid-out-0

Video Devices

hailo-vid-cap-2

The MIPI CSI2 subsystem is connected directly to the camera sensor at one end and in most cases, sends data directly to the ISP, however, it is also possible to send data directly to RAM using this video device.

hailo-vid-cap-0

This is the primary ISP output. This path supports full resolution and all ISP image quality features, transformations, and format conversions.

hailo-vid-cap-1

This is the secondary ISP output. This path supports full resolution and some ISP image quality features, transformations, and format conversions.

hailo-vid-out-0

This device is used for sending frames from DDR memory to ISP. It sends raw frames for HDR stitching. The HDR is wrapped by Media Library for convenience.

The following table lists the V4L2 IOCTL calls that are supported:

Table 1. Video devices V4L2 IOCTL support

V4L2 IOCTL	Description	Constraints
VIDIOC_QUERYCAP	Query device capabilities	
VIDIOC_ENUM_FMT	Enumerate image formats	
VIDIOC_S_FMT	Set the data format	Not supported on self-path video devices
VIDIOC_G_FMT	Get the data format	
VIDIOC_TRY_FMT	Try a data format	
VIDIOC_CREATE_BUFS	Create buffers for Memory Mapped or User Pointer or DMA Buffer I/O	
VIDIOC_G_FBUF	Get frame buffer overlay parameters	
VIDIOC_REQBUFS	Initiate Memory Mapping, User Pointer I/O or DMA buffer I/O	Only V4L2_MEMORY_MMAP buffer type is supported
VIDIOC_QUERYBUF	Query the status of a buffer	
VIDIOC_QBUF	Enqueue a buffer to the driver	
VIDIOC_DQBUF	Dequeue a buffer from the driver	
VIDIOC_EXPBUF	Export a buffer as a DMABUF file descriptor	
VIDIOC_STREAMON	Start streaming I/O	
VIDIOC_STREAMOFF	Stop streaming I/O	

The Video Devices also support custom IOCTL codes.

Table 2. Video device custom IOCTL codes

V4L2 IOCTL	Description	Constraints
VIDEO_GET_VSM_IOC	Get global motion vector delta	
VIDEO_FPS_MONITOR_SUBDEV_IOC	Get measured FPS	

Video device `/dev/video0` and `/dev/video1` also support v4l2 controls. An expanded Image quality API is provided in the form of custom v4l2 controls. These APIs allow controlling the ISP behavior and impact the resulting image, similarly to the cam device API.

Video Sub-Devices

V4L2 IOCTL calls that are supported:

Table 3. Video sub-devices V4L2 IOCTL support

V4L2 IOCTL	Description
VIDIOC_QUERYCAP	Query device capabilities
VIDIOC_G_CTRL	Gets a value of a control
VIDIOC_S_CTRL	Sets a value of a control

imx678 (or any other supported sensor)

This sub-device can be used directly for sensor calibration. Please note that some V4L2 IOCTL codes may not be available for all types of camera sensors.

Table 4. Sensor video sub-devices V4L2 IOCTL support

V4L2 IOCTL	ID	Description
VIDIOC_G_CTRL	V4L2_CID_VBLANK	Vertical blanking
	V4L2_CID_TEST_PATTERN	Test pattern
	V4L2_CID_ANALOGUE_GAIN	Analogue gain
	V4L2_CID_EXPOSURE	Exposure
	V4L2_CID_PIXEL_RATE	
	V4L2_CID_LINK_FREQ	
VIDIOC_S_CTRL	V4L2_CID_HBLANK	Horizontal blanking
	V4L2_CID_VBLANK	
	V4L2_CID_TEST_PATTERN	
	V4L2_CID_ANALOGUE_GAIN	
	V4L2_CID_EXPOSURE	
	V4L2_CID_HFLIP	Flip image horizontally
	V4L2_CID_VFLIP	Flip image vertically
	V4L2_CID_WIDE_DYNAMIC_RANGE	Enables/disables HDR

cdns_csi2rx

Represents the MIPI CSI2 hardware. It is not represented as a device node in /dev. The user should not interact with it directly.

hailo-pixel-mux

Represents the internal Routing and Filtering Unit (RFU) hardware interconnecting the MIPI CSI2 and the ISP.

hailo-rxwrapper

Represents internal bypass and crediting mechanism. Additionally, it interconnects MIPI CSI to RAM.

isp

Represents the ISP. Used for calibration and maintenance, it is being used by ISP middleware.

1.5. ISP Media Server

The ISP Media Server is a software package responsible for image quality and runtime management of the ISP and camera sensors. Initial image quality calibration parameters are loaded during startup from an XML file and then tuned during runtime. The 3aconfig parameters are tuned during runtime according to configuration stored in the JSON file, `3aconfig.json`. The camera sensor and calibration setup are set in the dedicated text file `Sensor<N>_Entry.cfg` where `<N>` represents the sensor number (e.g., `Sensor0_Entry.cfg`).

The ISP Media Server is comprised of the following modules:

- Image quality statistics collection – user space application which collects and calculates various statistics and events.
- Camera control – dynamically linked camera sensor specific module which initializes and configures the camera sensor.
- 3A library – a dynamically linked image quality library.

The 3A configuration and calibration XML files are produced as part of the tuning and calibration process.

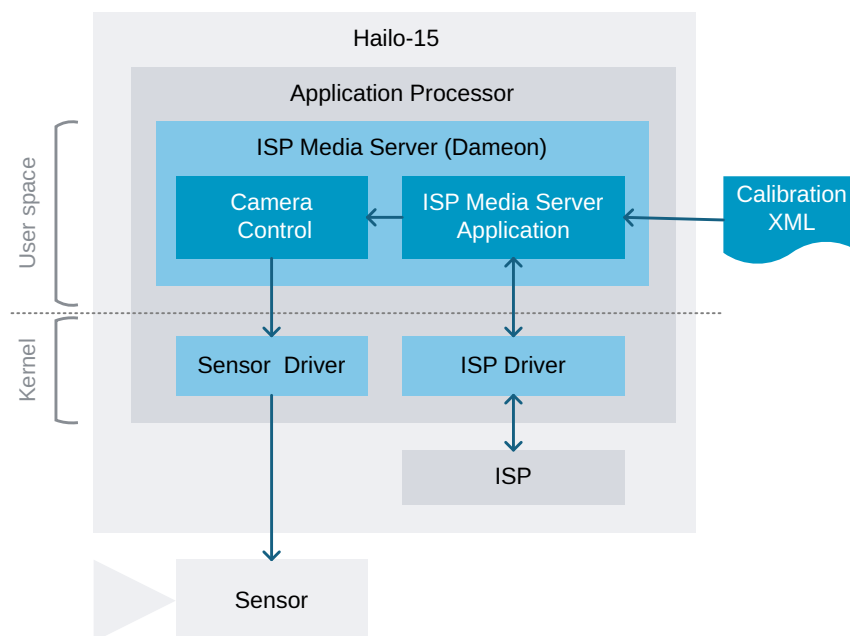


Figure 4. ISP Media Server software architecture

The modules are provided as pre-compiled binaries and are pre-integrated to Linux for Hailo supported boards and sensors. It must run to provide ISP functionality via V4L2. A user space API is provided for 3A and image quality functions.

1.5.1. Daemon Linux Integration

The ISP Media Server is a user space daemon that loads and runs during Linux startup. It monitors the V4L2 video devices for media events and configures the ISP. The V4L2 control APIs are described in the Hailo Imaging ISP Software Stack API manual. They can be used to customize the handling of 3A behavior. The API allows users to execute custom control of image quality in runtime, including custom 3A solutions.

Note: On startup the Media Server daemon will execute default image quality maintenance. To enable user custom solutions, the default maintenance should be disabled in the `3aconfig.json` file. Each module of the image

processor corresponds to a different component in 3aconfig.json, and can be enabled/disabled individually (such as Auto Exposure, Color processing, Edge Enhancement).

1.6. Video Encoder

The Hailo-15™ supports hardware accelerated video encoding in H.265 and H.264. Hailo provides a Linux integrated driver as well as a low-level library, a GStreamer element and Hailo Media Library API.

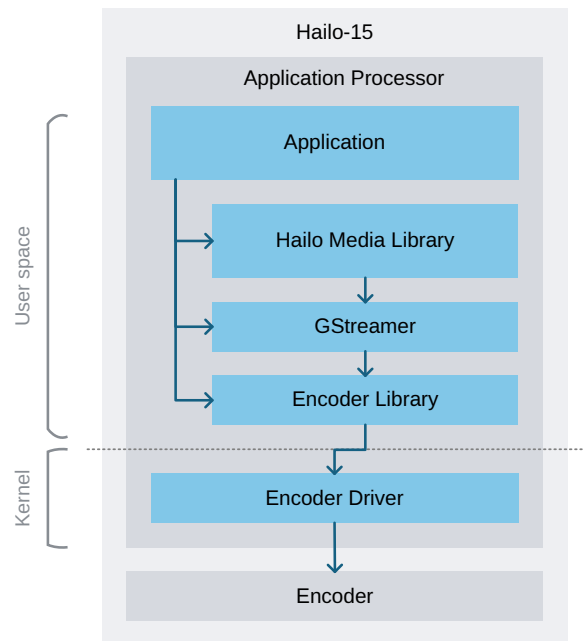


Figure 5. Video Encoder software architecture

The encoder software architecture is comprised of:

- Encoder Library – user space dynamically linked library which manages and provides an API to the encoder driver.
- Encoder driver – kernel space driver which interacts with the user space library and the underlying hardware.
- Media Library module and GStreamer element - a convenient way to integrate the encoder in media library and Gstreamer pipelines

The encoder's documentation is available in Media Library.

2. Changelog

Hailo Imaging v1.11.0 (March 2026)

- ISP
 - Added support for Sony IMX662 camera sensor for both SDR and HDR modes
 - Added HDR compression control for ISP stitcher mode, with configurable compression in media server configuration
 - Added configuration file-based logging system with extended logging capabilities
 - Improved HDR exposure calculation to better utilize long exposure integration time before applying gain
 - Improved sensor driver error handling
 - Fixed "multiple Auto Exposure (AE) algorithm issues, including ROI calculation errors and edge cases with black image input"
 - Fixed "antiflicker artifacts in HDR mode when auto-exposure is disabled"
 - Fixed "Auto White Balance (AWB) parameter range validation"
 - Fixed "incorrect RHS1 configuration in ISI sensor driver"
 - Fixed "inconsistencies in Auto Exposure (AE) configuration parameter"
 - Removed support for Sony IMX334 camera sensor

Hailo Imaging v1.10.0 (January 2026)

- ISP
 - Hailo-15L only: Added support for HDR and AI-ISP Gen2 for Sony IMX678 sensor based designs
 - Added support for CCM desaturation based on gain level
 - Added support for SDR <-> HDR fast mode switching during streaming.
 - Added support for AI-ISP Gen2 with Sony IMX715 sensor based designs
 - Added support for setting HCG for short exposure in HDR mode
 - Added support for dual sensor simultaneous streaming
 - Updated limits for the following 3aconfig parameters: A2dnr.sigma, ADmsc.sharpen_r1, ADmsc.sharpen_r2, ADmsc.sharpen_r3
 - Fixed "Long integration time in HDR" issue while working with Sony IMX715 sensor.
 - Fixed "frame timeout" message which may potentially be seen during long HDR activations
 - Hailo-15L only: Fixed statistical infinite loop which may occur during operation of AE priority. ("OOM-killer" message in dmesg)
 - Fixed "isp dataloss" message occurrence in HDR mode.
 - Fixed division by 0 occurrence when using AE ROI

Hailo Imaging v1.9.1 (December 2025)

- ISP
 - Fixed "HDR luma reporting"
 - Fixed "When time is set using 'date' command, timeout error may occur"
 - Fixed "SP2 error may occur when 3dnr is disabled at stream close"
 - Fixed "Adaptive Auto Exposure setpoint is not updated correctly when operating in SDR mode"
 - Fixed "Auto Exposure shutter increases when the setpoint is decreased"

Hailo Imaging v1.9.0 (October 2025)

- ISP
 - Added Handlers (warn print) for Hailo15L error IRQs
 - Added isp_general_3a_unix_epoch: Timestamp mechanism for sw monitoring
 - Modify encoder memory from DT reserved to DMA
 - Support setting RHS1 from control (short exposure limit)
 - Move FE buffer to use CMA instead of DMA
 - Added Validation for 3aconfig parameters
 - Added 3aconfig locking & retry mechanism
 - Added priorityHDR parameter - dedicated AE priority for HDR mode
 - Bugfix: Segfault on video close
 - Bugfix: FE timeout during stream toggle

Hailo Imaging v1.8.1 (August 2025)

- ISP
 - Bugfix: FPS drop in HDR (20FPS / 10FPS)
 - Bugfix: AWB result in imx664 HDR when WB gains are close to maximal
 - Bugfix: AE stability in low luminance values (<5)
 - Added "minAbsLockDiff" optional parameter in 3aconfig AdaptiveAE / HDRAdaptiveAE Section: defines a minimal luma value for locking AE algorithm.

Hailo Imaging v1.8.0 (July 2025)

- ISP
 - Dynamic range 2D interpolation in 3aconfig.json for AWdr, AGamma
 - HDR manager example: use epoll instead of select when getting raw frames
 - Logger cache management thread, avoid high ram usage
 - Bugfix: FD leak in isp_media_server
 - Bugfix: Error handling in isp_media_server file access
 - Sensor DCG support for imx675
 - Change default Is HDR ratio

- Add v42-control isp_ae_sem_setpoint to query the Adaptive setpoint
- Add v42-control isp_ae_gain_limits. isp_ae_integration_time_limits
- Add v4l2 control isp_ae_hist64_channel, isp_ae_hist64_mode
- Add v4l2 control isp_ae_dynamic_range
- Added IMX664 support (ISI + kernel driver + injection)
- AcprocPost: Histogram equalization, bilateral filters
- Support DRAM -> ISP 12bit read (injection)
- support parsing calibration file as json instead of xml
- Bugfix: imx715 - wrong FPS in HDR
- Bugfix: double BLS compensation in AWB
- H15L HDR support with imx675

Hailo Imaging v1.7.1 (May 2025)

- ISP
 - Bugfix: Frameloss on HDR long stream (sync RX wrapper frames)

Hailo Imaging v1.7.0 (April 2025)

- ISP
 - HDRAdaptiveAE - new AE section in 3aconfig.json for optimized HDR AE
 - Parametric path for Sensor0_Entry.cfg
 - Parametric path for 3aconfig.json
 - AGamma, AWdr exposure and dynamic range interpolation in 3aconfig.json
 - BLS: change AbLs to work based on system gain and not sensor gain
 - ACprocPost: new sharpness parameters
 - DCG support for imx675
 - Bugfix: imx678 wrong vmax calculation
 - Bugfix: Sensor driver i2c race
 - Bugfix: imx675 hailo_ctrl exposure/gain API
 - Bugfix: Raw image capture with exposure > 1s
 - Bugfix: AcprocPost FD leak

Hailo Imaging v1.6.1 (February 2025)

- ISP
 - New tuning tool (beta ver.) supporting tuning for most of the important IQ parameters, not including low light support. See Hailo Imaging Calibration and Tuning User Guide
 - imx715 base driver support for multiple exposures acquisition
 - imx715 add antiflicker support to driver
 - imx715 add to stream example
 - Bugfix: ca_enable / dci_enable api coherency

- Add support for frame timestamps based on sensor acquisition time (for EIS)
- Injection tool: added stream_eth option - stream raws from host continuously.
- Injection tool: Added dummy sensor driver and setup script.
- HDR native example added to code samples
- Increased 2dol Max short exposure time to 2ms on imx678
- Bugfix: ACprocPost default value for lowpass alpha
- Bugfix: CAC get apis
- HDR move to dma buff for improved performance with AI

Hailo Imaging v1.6.0 (January 2025)

- ISP
 - imx678 setup script & 3aconfig alignment to default
 - ca_mode support
 - DCG sensor support in imx678 driver & v4l control
 - DCG sensor support for raw capture on imx678
 - ACprocPost New fields (for post isp filters)
 - NV12 clipping range - change to full range

Hailo Imaging v1.5.2 (December 2024)

- ISP
 - Added 3aconfig parameters interpolation based on bitrate
 - Added "AcprocPost " in 3aconfig to support post ISP DSP filters

Hailo Imaging v1.5.1 (November 2024)

- New user guide document
- ISP
 - Bugfix: isp_dg_enable get control
 - Bugfix: isp_cproc_brightness - fix brightness cap (depending on contrast)
 - Added isp_ae_sensor_gain - v4l2 control for modifying the sensor gain during manual exposure control
 - Added Exposure-time based Antiflickering. controlled by 3aconfig field antiFlickerAfps
 - Added v4l control isp_ae_hdr_gains - get gains for all exposures in HDR
 - Added v4l control isp_ae_integration_times - get exposure times for all exposures in HDR

Hailo Imaging v1.5.0 (October 2024)

- ISP
 - Support HDR 3DOL @ 20FPS streaming
 - ✱ Note – The HDR calibration and tuning files (sony_imx678.xml and 3config.json respectively) are for specific lenses. Different lens setups may require different calibration and tuning for optimal image quality even when the same sensor is used.
 - DDR BW reduction when 3DNR is disabled
 - Added sensor / ISP digital gain split support to control the gain between sensor and ISP (tuning dependent)
 - Added Sensor / ISP digital gain split support
 - Increased priority of kernel thread af_monitor – faster response to AF measurement
 - IMX715 driver / basic streaming support
 - Bug fixes
 - Known issues
 - ✱ Video analytics running on HDR stream might cause system instability. For this version it is recommended to run analytics without HDR until hotfix is provided.
- Encoder
 - HDR mode bug fixes - The HDR mode prevents the buffer from overflowing, controlled by hrd-cpb-size parameter.
 - RC algorithm changes to support CVBR mode
 - New user config for encoding with less arguments
 - Encoder uses presets for all additional arguments
 - ✱ Presets are located as CSV file on the filesystem
 - Added optional padding to prevent underflows (HDR and CVBR modes)
 - Automatic selection of encoding profile and level based on resolution and bitrate
 - Automatic selection of encoding gop-length and monitor-frames
 - Encoded bitstream provides PTS information
 - Added support for video frontend and encoder information via CPP media library
 - Bug fixes
 - Known issues
 - ✱ Encoder Performance with Multiple Streams – The encoder has a theoretical capacity of 290 MP. However, there is an open issue with the performance of pipelines that reach very high utilization of the encoder, When the CPU is busy, it fails to trigger the next frame promptly after the current one finishes.

3. Imaging Algorithms Overview

3.1. Introduction

This section provides a high level overview of the image processing algorithms running on the Hailo-15 vision processor. These algorithms are performed by various elements of the Hailo-15, including the ISP (Image Signal Processor), DSP (Digital Signal Processor) and the AP (Application Processor).

The ISP algorithms are controlled directly by the ISP SW Stack, and are mainly configured through the 3aconfig.json. The other algorithms (which usually involve other components of the Hailo-15) are mainly controlled by the Hailo Media Library.

More information about the APIs, tuning and usage please refer to the corresponding documentation.

3.2. ISP Algorithms

3.2.1. 3A Algorithms

The 3A Algorithms are adaptive algorithms that are handled by the Media Server daemon as part of the ISP SW Stack. The algorithms automatically adjust sensor, ISP and lens parameters (where supported) depending on the scene in order to produce a better image. This is done by exposing statistics from the ISP which allows the software to control the parameters accordingly.

- **Auto Exposure (AE)** - This algorithm is responsible for automatically adjusting controlling the different parameters affecting the total system exposure of the captured image to ensure that it is properly exposed, based on the lighting conditions:

- **Gain:** Also known as “ISO”, refers to amplification of the signal. Gain increases image brightness, but amplifies the noise as well.

There are two methods of applying gain (analog gain and digital gain).

Analog gain can be applied by the CMOS sensor before conversion to digital format. As a rule of thumb, applying analog gain is always preferred over digital gain.

Digital gain can be applied both by the CMOS sensor (if supported) and by the ISP. Digital gain is applied on the digital signal (after ADC, and can lead to loss of detail and increased noise).

- **Exposure Time:** Also known as “Integration Time” or “Shutter Speed”, refers to the amount of time (usually in milliseconds) that the image sensor is exposed to light.

A longer exposure time allows more light to hit the sensor, which is useful in low-light conditions, but can cause motion blur if the subject or camera moves.

- **Aperture:** Also known as “Iris”, refers to the opening of the camera lens through which light passes into the sensor. It is measured in F-Stops (e.g F/1.6, F/2.8. etc...).

A larger aperture (smaller F-Stop number) lets in more light, but creates a shallower depth-of-field, while a smaller aperture (larger F-Stop number) lets in less light, but creates a wider depth-of-field.

- **Auto Focus (AF)** - This algorithms determines the optimal focus distance by analyzing the sharpness of the image based on changes in intensity or contrast.

The auto focus algorithm requires control of a motorized lens in order to adjust the different focusing elements.

- **Auto White-Balance (AWB)** - This algorithm adjusts the color balance of the image to ensure that white objects appear white under various lighting conditions and color temperatures.

The algorithm applies different gains to the red, green or blue channels to adjust the color balance of the image.

3.2.2. ISP Denoising Algorithms

The filters listed below are handled by the ISP hardware, and are intended to reduce the noise in the image. Each algorithm has tunable parameters for adjusting its' strength, usually depending on the system gain level.

- **2DNR** - This is a spatial (2-dimensional) noise reduction filter, which implements a variation of the non-local means (NLM) algorithm.

The 2DNR filter is intended to preserve edges, by using considering non-local information and not just local neighboring pixels. 2DNR is less effective when the image has high noise level, and may cause blurring.

- **3DNR** - This is a temporal (3-dimensional) noise reduction filter, which implements a pixel-wise first-order temporal IIR (Infinite Impulse Response) filter.

The 3DNR filter uses a motion detection mechanism in order to reduce motion ghosting artifacts. 3DNR is less effective when fast motion is involved, and may cause ghosting.

- **DPF** - This is a sharpness preserving denoising block that implements a bilateral filter (combining a range filter and a spatial filter).

3.2.3. Additional ISP Algorithms

The blocks and algorithms below are handled as part of different modules of the ISP hardware stack, and provide additional image processing capabilities as part of the vision pipeline.

- **BLC (Black Level Correction)** - Also known as "*BLS (Black Level Subtraction)*", this module is responsible for clearing *dark current noise* in order to ensure that the darkest areas in the image are actually black.
- **LSC (Lens Shading Correction)** - This module corrects uneven illumination caused by the optical properties of the lens (called *lens shading* or *vignetting*). This phenomenon results in the image corners to appear darker than the center.
- **WDR (Wide Dynamic Range)** - This module performs local tone mapping to enhance visibility of details in high dynamic range areas. This is done by adjusting the brightness and contrast locally, allowing to compress the dynamic range while preserving important details.
- **DMSC (Demosaic)** - This module is responsible for reconstructing a full-color RGB image from the partial bayer filter image, and includes the following sub-modules:
 - **CAC (Color Aberration Correction)** - This module fixes transverse (lateral) chromatic aberrations. Transverse aberration occurs when different wavelengths are focused at different positions in the focal plane, because the magnification and/or distortion of the lens also varies with wavelength.
 - **De-Purple** - This module fixes the axial chromatic aberration which causes the Purple-fringing artifact.
 - **Demoire** - This module removes unwanted moire patterns, which often occur when capturing images of screens or fine-textures (such as fabrics).
 - **Sharpening** - Achieves an apparent sharpening by emphasizing the image's high-frequency bands.
- **EE (Edge Enhancement)** - Used in order to enhance the Y channel of edges and desaturate the UV channels of edges. Desaturation is done to decrease color saturation, color aberration and color fringing effects of the edge.
- **Gamma Correction** - Performs global tone mapping by adjusting the luminance of the image using a pre-supplied non-linear curve.
- **CPROC (Color Processing)** - This module is used to adjust different color parameters of the image, such as contrast, brightness, hue and saturation.

3.3. Hailo Media Library Vision Algorithms

The Hailo Media Library handles additional computer vision algorithms that involve multiple components of the system. Among these algorithms are the following:

- **AI Denoising** - Employed to further denoise the ISP image, and is much more effective in higher gains (in challenging low-light conditions).
- **LDC (Lens Distortion Correction)** - This module uses a *Dewarp* function on the Hailo-15 DSP function to correct distortions caused by the optical characteristics of the camera lens. These distortions make straight lines seem curved, or objects to seem stretched/compressed (depending on the lens characteristics and location of the objects).
- **EIS (Electrical Image Stabilization)** - This is a digital technique used to minimize effects of camera shaking, resulting in a smoother video footage. This is done by sampling an external gyroscope to detect motion and adjust the frames accordingly.
- **HDR (High Dynamic Range)** - This feature is intended to deal with high dynamic range scenes (in which the difference between the bright and dark parts of the image is very noticeable). In such scenes, a single frame cannot be properly exposed (some areas of the frame are overexposed or underexposed, while other areas are properly exposed). To deal with such scenes, HDR typically captures 2 or 3 frames with different exposure settings, and stitches them back to a single image.

For further information on the above algorithms, refer to the Hailo Media Library User Guide.

4. Tutorials

While Hailo's media library is recommended for automatic ISP configuration with GStreamer and the C++ API, users may also manually configure the ISP using the V4L controls for specific use cases. These code samples show several cases of the IQ (image quality) settings being modified directly by the user and serve as a reference for customers who need to configure the ISP manually for particular scenarios.

4.1. v4l_ctrl_example

4.1.1. Motivation

In some scenarios, manual ISP configuration may be necessary to achieve specific image processing outcomes. For example, assume in a daylight scene where we want the auto exposure (AE) to adjust according to the light level in the bottom left corner of the screen (the region of interest, or ROI). This ensures the image will not be overexposed in this ROI, and the rest of the picture is not a concern. In such cases, the ISP can manually be configured to calculate the exposure by giving more weight to this ROI and smaller weights to the other ROIs. This example will demonstrate this along with other parameters that will be set.

This code example shows how to use the V4L2 framework to get and set different controls in the ISP Controls API. It is intended as a reference for implementing software to control various ISP configurations using the V4L2 Controls API.

4.1.2. Requirements

- The main video device path exists (`/dev/video0`).
- A stream is already running.
- (Optional) If the user has compiled an image of their own, the ISP needs to be calibrated according to their sensor. Please run the corresponding script (`setup_imx675.sh`, `setup_imx664.sh`, or `setup_imx678_sdr.sh`) to reset the ISP parameters before starting the video stream.

4.1.3. Examples

- Set/get for the `isp_wb_r_gain` control.
- Set/get for the `isp_wb_cc_matrix` control.
- Set/get for the `isp_ae_roi` and `roi_weight_list`.
- Set/get for the `isp_ae_iris_limits` control (only works for sensors that support iris control; not supported by IMX678, IMX664, and IMX675).

Note: Auto algorithms might change the values that are set in the example, meaning the get print might show a different value. To turn off the auto-update, modify the `3aconfig.json` file located at `/usr/bin/3aconfig.json` and disable the specific algorithm.

4.1.4. Usage

Run the `v4l_ctrl_example` command to execute the example.

4.1.5. Code Structure

Print all of the ISP Controls

Intentionally all the controls are being pulled and printed so that they can be called by their name and not by their ID. They should all be held in `QExtCtrlsList_t` so that each time you get or set a control, the function `find-QExtCtrl` is used with `QExtCtrlsList_t` to call them by name instead of their ID.

Get and Set Controls

`isp_wb_r_gain` will set the white balance gain for the red channel only.

Obtain a handle to the `isp_wb_r_gain` control by name:

```
wb_r_gain_ctrl = findQExtCtrl(qCtrls, "isp_wb_r_gain")
```

Use `wb_r_gain_ctrl` to determine its ID and the expected value:

```
isp_wb_r_gain, id: 0x984000, num elems: 1, elem size: 4
```

Use the values to create a struct `ExtCtrl` with the same ID and size as `wb_r_gain_ctrl` but with a new value:

```
memset(&ExtCtrl, 0, sizeof(ExtCtrl))
int val = 0
ExtCtrl.id = wb_r_gain_ctrl->id
ExtCtrl.size = wb_r_gain_ctrl->elem_size * wb_r_gain_ctrl->elems
ExtCtrl.ptr = &val
```

Use `ExtCtrl` and the `set_ext_ctrls` function to define the new value:

```
set_ext_ctrls(fd_video, &ExtCtrl)
```

For the `set_ext_ctrls` function, simply use `ioctl` wrapped inside a custom function `xioctl` to set the new value:

```
static int set_ext_ctrls(int fd, struct v4l2_ext_control *ExtCtrl) {
    int rc = 0
    struct v4l2_ext_controls ExtCtrls
    ExtCtrls.count = 1
    ExtCtrls.controls = ExtCtrl
    ExtCtrls.which = V4L2_CTRL_ID2WHICH(ExtCtrl->id)
    rc = xioctl(fd, VIDIOC_S_EXT_CTRLS, &ExtCtrls)
    if (rc) {
        printf("%s ioctl VIDIOC_S_EXT_CTRLS failed with %d, errno: %d\n", __func__,
            rc, errno)
    }
    return rc
}
```

This process is repeated for all the controls defined in the code. Each control is accessed by its name, and the respective value is set or retrieved as needed.

Auto Exposure with ROI Example

In the `set_roi_ctrls` function, `roi_matrix[ ]` and `roi_weight_list[ ]` are defined:

- `roi_matrix[ ]`: Divides the image into windows.
- `roi_weight_list[ ]`: Defines the weight assigned to each window.

In this example, 4 windows are defined, and weights are assigned to each of them. The auto exposure will now be calculated only by the top left corner and its weights.

4.1.6. Additional Examples

- **White Balance Control ("isp_wb_cc_matrix")**: Retrieve, set, and get the white balance color correction matrix.
- **Auto Focus Control ("isp_af_enable" and "isp_af_window")**: Enable auto focus and set/get auto focus window settings.
- **Iris Control ("isp_ae_iris_limits")**: Set and get iris limits for supported sensors.
- **AF Measurement ("isp_af_measurement")**: Retrieve auto focus measurement data.

4.2. v4l_event_handling_example

4.2.1. Motivation

Handling events outside the ISP can be crucial for implementing custom auto algorithms that the ISP does not support or for using alternative implementations. Events may occur every frame or after every few frames. This example demonstrates how to subscribe and capture relevant events but does not implement new algorithms.

4.2.2. Code Example for Handling ISP Stat Events Externally

This code example demonstrates how to subscribe to ISP statistics events and handle them externally. This approach is particularly useful for implementing custom configurations for the ISP based on these statistics. Events can be subscribed to and captured without making any changes to the `3aconfig.json` file. However, if the intention is to replace the default ISP algorithms with custom solutions, the corresponding auto algorithms must be disabled in the `3aconfig.json` file. Note that the ISP events do not correspond directly to specific modules in `3aconfig.json` on a one-to-one basis. Therefore, each user needs to identify and disable the relevant modules and use the events to implement their own custom handling.

4.2.3. Requirements

- The main video device path exists (`/dev/video0`).
- A stream is already running.

4.2.4. Example Behavior

- **Subscribing to ISP Stats Events:** The example utilizes a hardcoded list `events_to_handle` to subscribe to certain ISP stats events:

```
static bool events_to_handle[] = {
    [HAILO15_UEVENT_ISP_EXP_STAT] = true,
    [HAILO15_UEVENT_ISP_EXPV2_STAT] = true,
    [HAILO15_UEVENT_ISP_HIST_STAT] = true,
    [HAILO15_UEVENT_ISP_AWB_STAT] = false,
    [HAILO15_UEVENT_ISP_AFM_STAT] = true,
    [HAILO15_UEVENT_SENSOR_DATALOSS_STAT] = true,
    [HAILO15_UEVENT_VSM_DONE_STAT] = true,
    [HAILO15_UEVENT_SENSOR_STREAMING_STAT] = true
};
```

- **Polling for Events:** The code polls for `VIDIOC_DQEVENT` events of type `HAILO15_UEVENT_ISP_STAT`.
- **Handling Events:** Based on the event's `hailo15_event_stat_id`, the event is handled accordingly:
 - Some statistic events, like `HAILO15_UEVENT_VSM_DONE_STAT` and `HAILO15_UEVENT_SENSOR_DATALOSS_STAT`, do not require an additional request.
 - For other statistics, an additional `VIDIOC_G_EXT_CTRL`s call is made for the appropriate control. Custom handling of the statistic event is invoked (in this example, it prints the checksum).

4.2.5. Code Structure

Initial Setup

- **Define Constants and Structures:** Various constants, such as `CAMDEV_AEV2_STATIC`, and data structures for storing statistics, like `AeLumaStatus`, `AeExpStatistics`, `AeHistogram`, and `AfMeasurementStatus`.
- **Event Handling Functions:**
 - `checksum`: Calculates a checksum for the data.
 - `handle_isp_stats_event`: Handles ISP stats events by calculating the checksum and printing the event details.
 - `handle_extra_stats_event`: Handles additional events like sensor data loss and VSM completion.

Main Functionality

- **File Descriptor and Event Subscription:**
 - Opens the video device.
 - Subscribes to events based on the `events_to_handle[]` array.
- **Event Polling and Handling:**
 - Polls for events and handles them based on the event ID.
 - For each event, the appropriate control ID and size are set, and an `ioctl` call is made to get the control data.
 - Calls `handle_isp_stats_event` or `handle_extra_stats_event` based on the event type.
- **Cleanup and Signal Handling:**

- `cleanup` to unsubscribe from events and close the video device.
- `sigint_handler` to handle SIGINT (Ctrl+C) to perform cleanup before exiting.

4.2.6. Usage

To run the example, use the following command:

```
v4l_event_handling_example
```

4.3. v4l_stream_example

4.3.1. Motivation

The following code sample illustrates the process of writing a simple native application to configure a V4L device for streaming and to start acquiring frames. This example covers the essential steps to set up the device, configure its parameters, and handle the data streams.

There are five methods to obtain frames from the ISP:

1. **GStreamer API using Hailo's Media Library on top of V4L:** The simplest approach, allowing the media library GStreamer element to grab frames from the ISP. (Refer to the media library documentation for examples.)
2. **GStreamer API using "v4l2src" and then piping to Hailo's Media Library:** Utilizes `v4l2src` as the source of the GStreamer pipeline, which can then be piped to the media library. This provides more control over V4L through the GStreamer element. (Refer to the media library documentation for examples.)
3. **C++ API of the Hailo's Media Library on top of V4L:** Allows the media library to automatically use V4L to grab frames, making it very easy to use. (Refer to the media library documentation for examples.)
4. **C API of V4L post-ISP (without HML, no GStreamer):** Provides fine control over the ISP, making it suitable for testing and niche use cases where precise control is required.
5. **C API of V4L raw (before ISP, without HML, no GStreamer):** Bypasses the ISP, delivering raw format streams directly from the sensor. This method is ideal for testing and specialized applications requiring raw data.

In this example, we will demonstrate how to implement methods 4 and 5. However, the recommended approaches are to use the Hailo Media Library with either C++ or GStreamer (methods 1, 2, and 3).

VSM is metadata provided by the ISP, used to implement image stabilization without requiring an external gyro. If an external gyro is used, VSM may not be necessary.

4.3.2. Description

The `v4l_stream_example` is a tool written in C for capturing video frames from a V4L2 compatible device. It provides options to capture frames in various formats and perform tests like capturing frames and extracting VSM data.

4.3.3. Device Information

- **/dev/video0 and /dev/video1:** These devices stream data from the ISP.
- **/dev/video2:** This device bypasses the ISP and captures raw data directly from the sensor. It outputs SRGGB12, SRGGB12P, or SGBRG12 raw Bayer formats with 12 bits per color.

4.3.4. Requirements

- V4L2 compatible video device
- Enough space to save the captured files, if capture mode is selected

4.3.5. Features

- **Test Modes:** Run different test modes including frame capture and VSM data extraction.
- **Format Support:** Supports various frame formats including raw, NV12, RGB, and YUY2.
- **Raw Frames Capture:** Capture frames from a raw image video device (/dev/video2).
- **Customization:** Configure the number of frames to capture, number of buffers to use, requested FPS, width, height, and more.
- **Save Option:** Option to save raw captures.

4.3.6. Parameters

- **width (int):** Width of the output frame. Default: 3840.
- **height (int):** Height of the output frame. Default: 2160.
- **format (str):** Format of the output frame. Currently supported formats: nv12, rgb, yuy2, raw_rggb12p, raw_rggb12, raw_gbrg12. Default: nv12.
- **device (str):** Path of the video device to test. Default: /dev/video0.
- **out-path (str):** File path for the captured frames. Default: capture.out.
- **vsm-out-path (str):** File path for VSM data output. Ignored unless the test type is vsm-test. Default: vsm.out.
- **type (str):** Type of test to run. Currently supported types: capture, vsm-test. Default: capture.
- **num-frames (int):** Number of frames to capture. Ignored unless the test type is capture. Default: 30.
- **num-buffers (int):** Number of buffers to use when capturing. Ignored unless the test type is capture. Default: 5.
- **save (int):** Option to save raw captures. Default: 1.
- **fps (int):** Requested frames per second (fps). Default: 30.

4.3.7. Usage

For detailed usage instructions, run the following command:

```
v4l_stream_example --help
```

To capture frames from a video device, use the following command:

```
v4l_stream_example --type=capture --width=3840 --height=2160 --format=nv12 --
↪device=/dev/video0 --out-path=capture.out --num-frames=30 --num-buffers=5 --
↪save=1 --fps=30
```

This command captures 30 frames in NV12 format with a resolution of 3840x2160 pixels from `/dev/video0`. It saves the captured frames to `capture.out` and uses 5 buffers for capturing with a requested frame rate of 30 frames per second.

VSM Test

To run a VSM test and extract VSM data along with capturing frames, use the following command:

```
v4l_stream_example --type=vsm-test --width=3840 --height=2160 --format=nv12 --
↪device=/dev/video0 --out-path=capture.out --vsm-out-path=vsm.out --num-
↪frames=30 --num-buffers=5 --fps=30
```

This command captures 30 frames in NV12 format with a resolution of 3840x2160 pixels from `/dev/video0`. It saves the captured frames to `capture.out` and extracts VSM data to `vsm.out`.

Raw Capture Example

Before running the tool for raw capture, ensure that the sensor is preconfigured with the desired parameters. For example:

```
hailo_ctrl -d $(find_subdevice_path.sh imx) -c exposure=150000
```

Then, run the raw capture tool with the following command:

```
v4l_stream_example --device=/dev/video2 --type=capture --format=raw_rggb12p --num-
↪frames=400
```

This command captures 400 raw frames of format `rggb12p` from the raw image video device `/dev/video2`. The outputs will be stored as separate files (name format is hardcoded, starting with `out_0.raw`).

4.3.8. Detailed Code Explanation

Core Functions

Opening the Device:

- `open_device`: Opens the specified video device for reading and writing.

Setting Format and FPS:

- `set_format`: Configures the format of the video frames based on user input.
- `set_fps`: Sets the frames per second (fps) for capturing video.

Buffer Management:

- `init_buffers`: Initializes memory-mapped buffers for capturing video frames.
- `queue_buffers`: Queues all initialized buffers for video capture.

- `free_buffers`: Releases allocated buffer memory.

Streaming Control:

- `start_stream`: Starts the video stream.
- `stop_stream`: Stops the video stream.

Frame Capture and Processing:

- `read_frame`: Reads a frame from the video device.
- `process_frame`: Processes and saves the captured frame data.
- `write_frame`: Writes raw frame data to a specified output file.

Test Execution**Test Loops:**

- `capture_frames_test_loop`: Captures a specified number of frames and processes each frame. If the format is raw and saving is enabled, the frames will be saved as separate files.
- `frames_vsm_test_loop`: Captures frames and extracts VSM data, saving the results to the specified file.

Running Tests:

- `run_test`: Runs the specified test (capture or vsm-test) by setting up the video device, initializing buffers, and starting the appropriate test loop.

4.4. HDR Manager Example Code

4.4.1. Motivation

The recommended approach for utilizing HDR is through the Hailo Media Library, which efficiently manages all buffers and seamlessly integrates the stitching API. However, for customers who prefer to independently manage the buffers and ISP, this example offers valuable insights and guidance on manual HDR processing. The example demonstrates how to correctly manage buffers to orchestrate the workflow from capturing frames directly from the sensor, processing them through the neural network core (NNcore), and then writing the processed frames to the ISP input.

4.4.2. Overview

The HDR Manager example demonstrates the management and processing of high dynamic range (HDR) video streams using the Video4Linux2 (V4L2) API in conjunction with Hailo's stitching functionality. The workflow involves capturing groups of frames with different exposures directly from the sensor, stitching these frames into a single frame using the neural network core (NNcore), and then writing the stitched frame to the ISP for regular processing. While this example is demonstrated in 4K resolution, it can also be adapted for Full HD. The focus is on managing the buffers correctly—reading and writing them to the ISP, sensor, and neural network core (NNcore)—to ensure proper HDR processing.

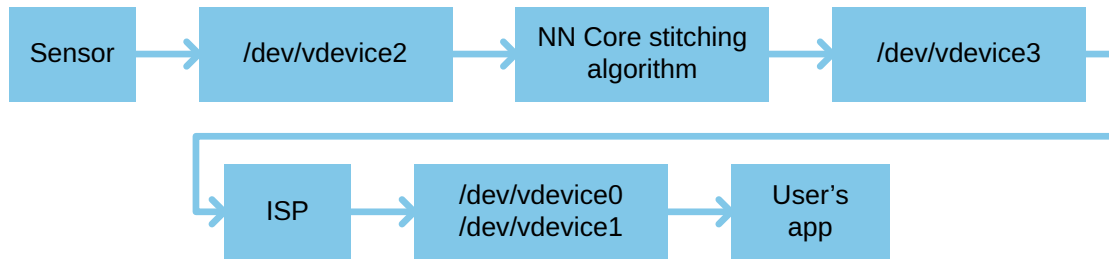


Figure 6. Block diagram of the HDR example

4.4.3. Usage

Note:

1. It is recommended to use different threads within the same process to handle this task. This ensures that HailoRT can be used efficiently without activating the Hailort Service, as it cannot be accessed from multiple processes simultaneously without Hailort Service.
2. While the application is running and reading buffers from the ISP, this example application runs in the background, reading frames from the sensor, stitching them using the neural network (NN) core, and then writing the stitched frames to the ISP. It is essential to start the user application that reads from the ISP before running this code. Additionally, when shutting down, the user application must be stopped first and then to exit the application. This sequence ensures proper buffer management and prevents any potential data corruption or processing errors.

Run the Example

```
hdr_manager_example
```

4.4.4. Code Structure

This example is divided into two main files: `main.cpp` (main application) and `hailo_stitch.hpp` (stitching functionality).

Stitching

The stitching functionality is encapsulated in the `HailortAsyncStitching` class. This class demonstrates a classic example of how to utilize the NNcore using HailoRT (Hailo's driver to communicate with the NNcore). Below is a high-level overview of its main components:

Initialization: In this part, the neural network is configured, loaded onto the NNcore, and activated so that buffers can be sent to the network. This configuration is done using a `.hef` file (a network compiled by Hailo DFC (Data Flow Compiler)).

NNcore Instance:

In the `init` function, an instance of the NNcore is obtained using `Vdevice`.

Loading the HEF File:

After configuring the NNcore instance, the HEF file is loaded onto the NNcore using `m_hef_path`.

Configuring Model Inputs and Outputs:

For each input and output of the model, the type (int8, int16, or float) is defined. In this case, the HAILO_FORMAT_TYPE_UINT16 is used together with the dimension HAILO_FORMAT_ORDER_NHWC (Number of samples, Height, Width, Channels). Note that this can be changed from input/output to input/output.

Model Configuration:

The model is configured using `m_infer_model->configure()`.

Creating Bindings:

Bindings are created so that we can read and write to the model using `m_configured_infer_model.create_bindings()`.

Processing:

The `process` function is used to read and write from the NNcore asynchronously.

Main Application

Device Opening:

Opens the raw capture video device (`/dev/video2`) to get frames from the sensor without passing through the ISP:

```
isp_in_fd = open_device("/dev/video10");
```

Opens the video device `/dev/video10` to write frames to the ISP:

```
isp_in_fd = open_device("/dev/video10");
```

Format and FPS Setting:

Sets the video color format, resolution, and 3 exposures (DOL_NUM_EXP_RAW) per frame for the raw capture vdevice:

```
ret = set_format(VIDEO_RAW_CAPTURE, INPUT_WIDTH_4K, INPUT_HEIGHT_4K, V4L2_PIX_FMT_
↳SRGB12, DOL_NUM_EXP_RAW);
```

Sets the frame rate to 20 FPS. Frames come in groups of 3 exposures 20 times per second. The default frame rate is 8 FPS, but to achieve 20 FPS post-HDR, capturing 3 frames and stitching them into one frame requires this setting:

```
ret = set_raw_capture_fps();
```

Sets the video color format, resolution, and 1 stitched frame for the ISP input vdevice:

```
ret = set_format(VIDEO_ISP_IN, INPUT_WIDTH_4K, INPUT_HEIGHT_4K, V4L2_PIX_FMT_
↳SRGB12, DOL_NUM_EXP_STITCHED);
```

Buffer Initialization:

Initializes buffers for the raw capture device and the ISP input device, and for the white balance.

```
ret = init_buffers(VIDEO_RAW_CAPTURE, DOL_NUM_EXP_RAW);
```

```
ret = init_buffers(VIDEO_ISP_IN, DOL_NUM_EXP_STITCHED);
```

Notes how the struct `buffer` wraps `v4l2_buffers` is being used:

```
struct buffer {
    int num_planes;
    int sizes[MAX_NUM_OF_PLANES];
    void *planes[MAX_NUM_OF_PLANES];
    struct v4l2_buffer v4l2_buf;
    int first_use;
};
```

The WB buffer is initialized as a regular array:

```
wb_buffer = (unsigned char*)mmap(NULL, wb_buffer_size, PROT_WRITE | PROT_READ, MAP_
↪ANONYMOUS | MAP_PRIVATE, -1, 0);
```

Start Streaming:

Queues buffers and starts the video stream for both devices using `queue_buffers` and `start_stream` functions.

Main Processing Loop:

1. The `mcm_loop` function handles reading frames from the video devices, processing them using the stitching API, and writing the buffers to the ISP.
2. Waits for an application to start reading buffers from the ISP (i.e., the user application):

```
wait_for_yuv_stream_start();
```

3. Configures HDR exposure ratios, where `LS_RATIO` represents the long-to-short exposure ratio and `VS_RATIO` represents the very-short-to-short exposure ratio, both in fixed-point format via the `isp_hdr_ratio` control:

```
setRatio(LS_RATIO, VS_RATIO);
```

4. Drops outdated buffers in the queue to start with the newest buffers received. Since there was a waiting time for the user application to start, there are out-of-date buffers in the queue. This function will drop all the buffers currently in the queue:

```
dequeue_all_buffers(VIDEO_RAW_CAPTURE);
```

5. Starts queuing the up-to-date buffers:

```
queue_buffers(VIDEO_RAW_CAPTURE);
```

6. Receive a full buffer from the sensor and an empty buffer for writing to the ISP:

```
index2 = read_frame(VIDEO_RAW_CAPTURE, DOL_NUM_EXP_RAW);
index3 = read_frame(VIDEO_ISP_IN, DOL_NUM_EXP_STITCHED);
```

7. Retrieves white balance gains from the ISP and updates the `wbBuffer` with adjusted and quantized values:

```
updateWBGains(wb_buffer);
```

8. Stitches the buffers with different exposures, and the buffer with the WB gains into one buffer using the `NNcore`, and fills `index3`:

```
stitcher->process(buffers[VIDEO_RAW_CAPTURE][index2].planes, wb_buffer,
↪buffers[VIDEO_ISP_IN][index3].planes[0]);
```

9. Waits for the asynchronous stitching process to complete:

```
while(!async_finished){
    continue;
}
```

10. Writes `index3` to the ISP and returns the `index2` buffer to the queue as an empty buffer for reuse:

```
queue_buffer(VIDEO_ISP_IN, index3);
queue_buffer(VIDEO_RAW_CAPTURE, index2);
```


5. Sensor Porting Guide

5.1. Introduction

The Hailo-15 SoC is designed to support a range of sensors ‘out-of-the-box’, including models like Sony’s IMX675, IMX678, and others. However, there may be scenarios where there would be a need to integrate additional sensors that are not included in the prebuilt configuration. This guide is intended to help by describing how to port new sensors onto the Hailo-15, covering the necessary steps to ensure that the new sensor operates correctly and is properly integrated within the Yocto-based OS.

5.2. Prerequisites

Before diving into the sensor porting process, it’s essential to ensure that certain prerequisites are in place:

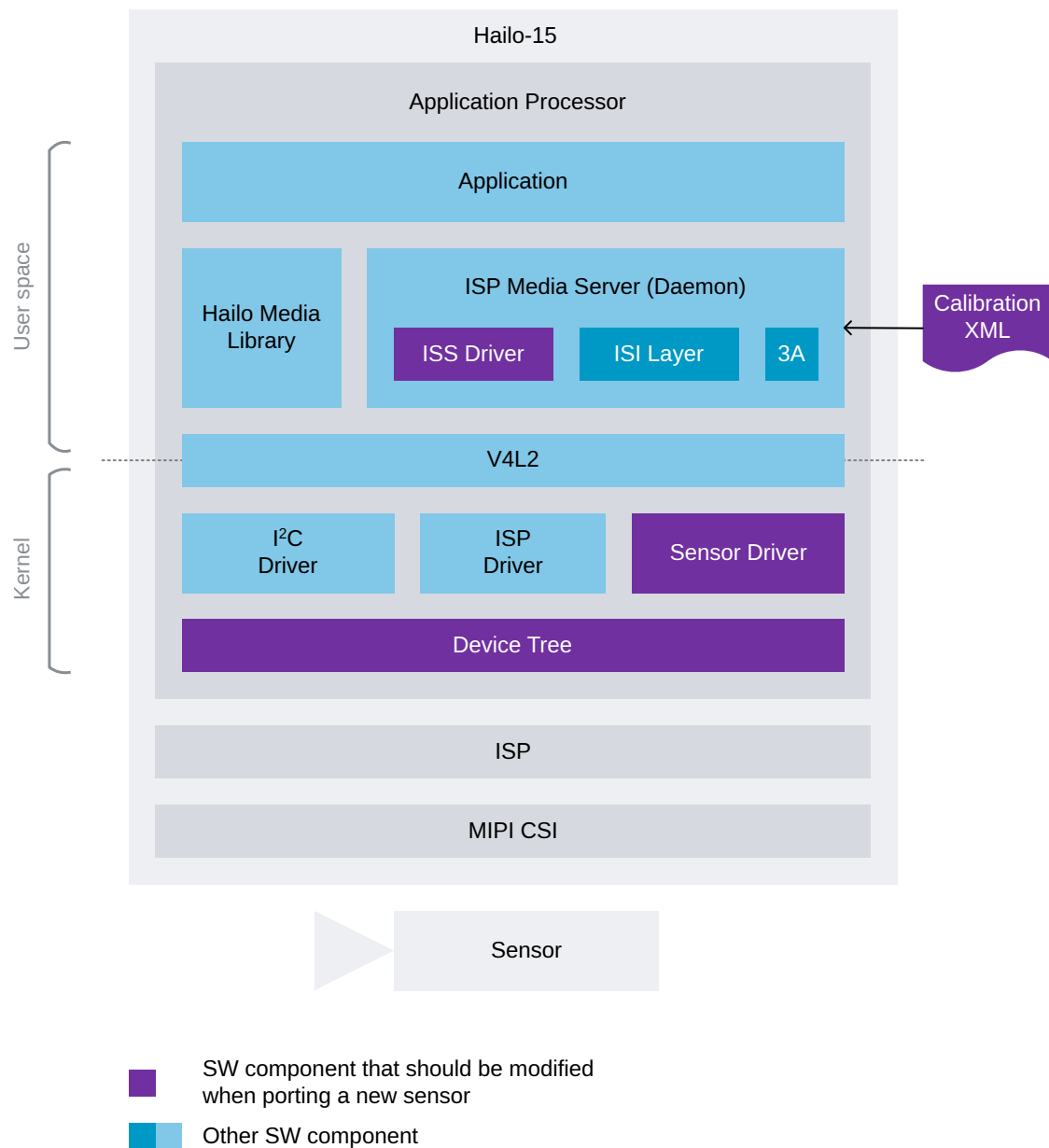
- **Required Knowledge:** This guide assumes basic familiarity with the Yocto Project build environment and embedded software development. The user should be comfortable working with kernel and user-space drivers, device tree modifications, and debugging low-level hardware-software interactions.
- **Physical Access to the Sensor:** Ensure there is a suitable sensor physically available for testing and integration.
- **Sensor Documentation:** Access to the sensor’s datasheet is required, including operating modes, register maps, and other configuration details. Having the sensor’s driver (preferably pre-written) will streamline the porting process.
- **Yocto Layer Setup:** Ensure that a custom meta-<vendor> Yocto layer has already been created. If this has not been set up yet, refer to the Yocto BSP Customization section of the Hailo OS User Guide for instructions on how to create and configure this layer. This layer will serve as the foundation for integrating the sensor with the Hailo-15 platform.
- **Optional Equipment:** Although not mandatory, having access to an oscilloscope or logic analyzer can be handy for debugging communication issues between the sensor and the Hailo-15, measuring oscillator frequency, and so on.

5.3. System Overview

In this section, we will take a high-level look at the files and software components involved in porting a new sensor to the Hailo-15 vision processor. The following image shows a block diagram that highlights the relevant components that should be modified (purple) and some other components (blue) which are relevant for this process.

5.3.1. Detailed Breakdown of Key Components

- **ISP Media Server:** A software package responsible for image quality and runtime management of the ISP and camera sensors.
- **Calibration XML:** Initial image quality calibration parameters are loaded during startup from an XML and then tuned during runtime. Note that when porting a sensor, this file may not be available as the calibration hasn’t happened yet. This interdependence leads to a “chicken and egg” situation, which is worked around in this guide by using a reference calibration XML and only modifying its resolution to the new sensor’s.
- **ISI Layer:** The ISI layer (Independent Sensor Interface) is able to handle different image sensors. It receives calls from user APIs, CamEngine, and 3A, and passes functions to the ISS (Image Sensor Specific) driver.
- **ISS Driver:** ISS Driver is a sensor-specific implementation that needs to be modified when bringing up a new sensor. The sensor-specific attributes and behavior are derived from the sensor datasheet and calibration data.



- **Sensor (Kernel) Driver:** The kernel driver is responsible for V4L2 functionality and, depending on the framework chosen, may also offer an API to support the ISS driver integration.
- **Device Tree:** Defines the hardware configuration for the new sensor board, and specifies its settings such as I2C address, clock frequency, MIPI-CSI2 data lanes, and so on.

5.4. Sensor Driver Frameworks

When porting a sensor, there are two possible driver frameworks one may choose – having the Sensor Module in the ISI Layer (user space) or in the kernel. This is determined by setting the appropriate *KernelDriverFlag* value and implementing the required functionality:

5.4.1. Sensor Module in ISI Layer

In this framework, the actual sensor control happens in user space through the ISI layer. The I2C read/write access is handled in the ISS Driver via *IOCTL* system calls to the */dev/i2c-x* device file. Here, the *KernelDriverFlag* in the *Sensor_Context_s* structure must be set to *1*, indicating that the sensor driver operates within the ISI layer rather than the kernel.

Note that the kernel driver is still required, as it registers the sensor as a V4L2 sub-device and supports the required V4L2 operations and streaming control.

This flow is shown in [Sensor Module in ISI Layer](#).

5.4.2. Sensor Module in Linux Kernel

In this framework, the ISI Layer interacts with the kernel driver through an API, and the kernel driver, in turn, initiates the relevant I2C read/write access to the sensor's registers. For this framework, *KernelDriverFlag* in the *Sensor_Context_s* structure must be set to *0*.

This flow is illustrated in [Sensor Module in Kernel](#), where the sensor driver in the kernel interacts directly with the sensor via the I2C bus. In this case, *KernelDriverFlag* in the *Sensor_Context_s* structure must be set to *0* to indicate that the kernel sensor driver is used.

Note: Hailo reference code and this guide focus on the Sensor module in the ISI Layer framework.

5.5. Setting Up Yocto for Kernel Driver Development

This guide assumes that the creation of a *meta-<vendor>* Yocto layer has already been completed, as outlined in the Hailo OS User Guide. The following steps will set up the Yocto environment, bringing it to a point where customization, compilation, and testing of the kernel driver can be performed easily. For this specific guide, porting of a theoretical Sony-based sensor called *imx000* will be demonstrated.

1. **Activate the Yocto environment** by running the following command from *meta-hailo-soc*:

```
. poky/oe-init-build-env
```

2. **Set up the new kernel module configuration in the meta:**

- Navigate to the *meta-<vendor>* directory and create the following directories:

```
cd ../meta-<vendor>
mkdir -p recipes-core/packagegroups/
mkdir -p recipes-kernel/linux-yocto-hailo/cfg/
```

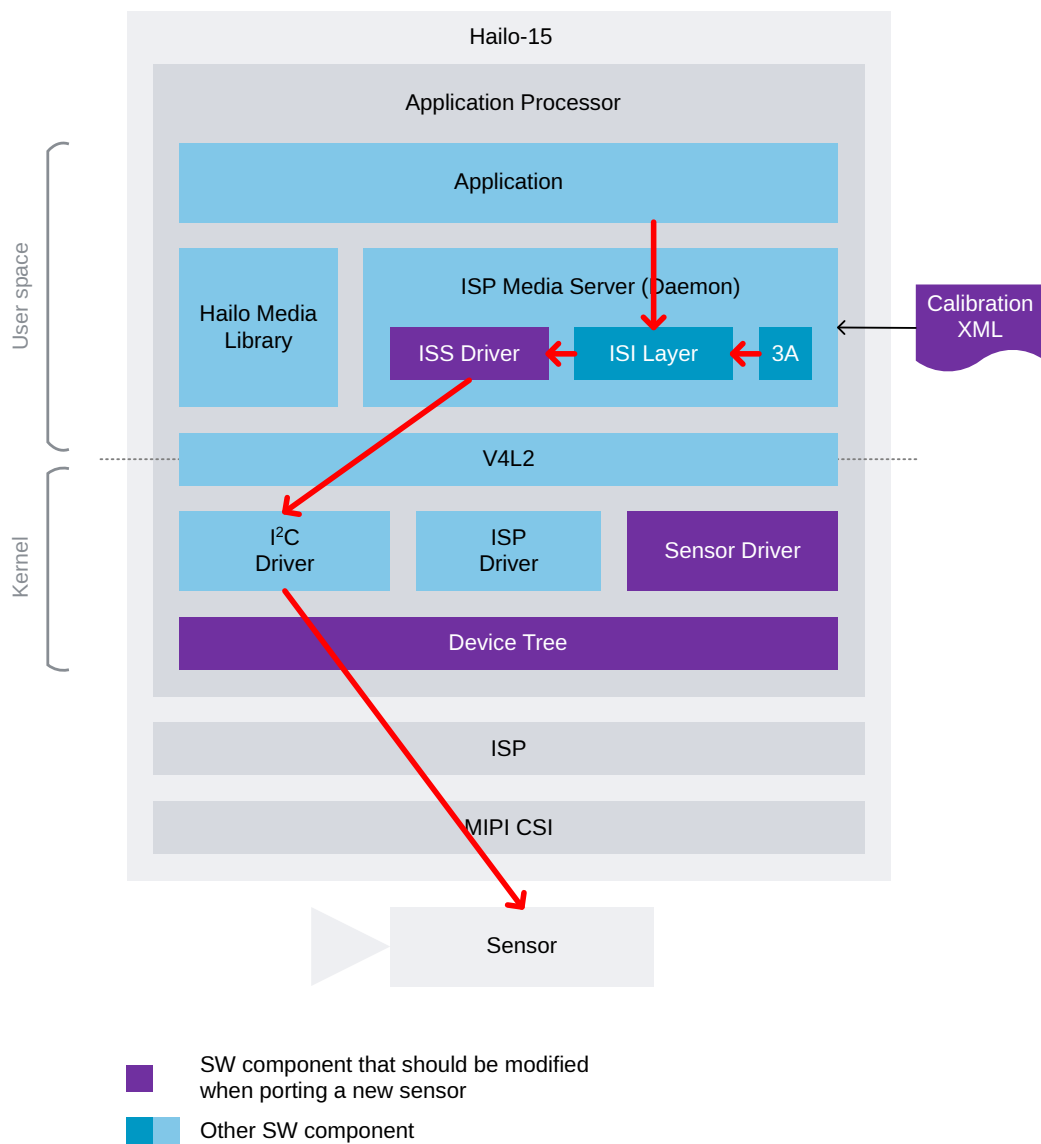


Figure 7. Sensor Module in ISI Layer

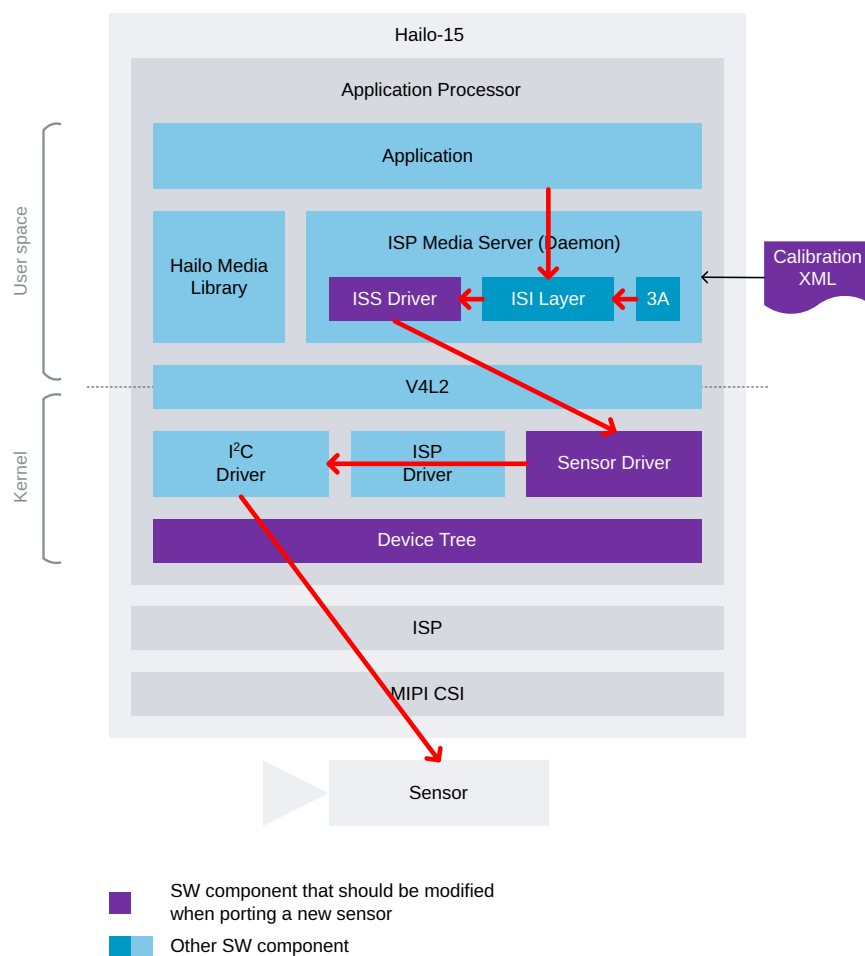


Figure 8. Sensor Module in Kernel

- In the *recipes-core/packagegroups/* directory, create the file *packagegroup-hailo-imaging.bbappend* and add the following line:

```
IMAGING_EXTRA_RDEPENDS:append = " iss-drivers"
```

- In the *recipes-kernel/linux-yocto-hailo/cfg/* directory, create the file *custom-sensor-configuration.cfg* and add the following line:

```
CONFIG_VIDEO_IMX000=m
```

- In the *recipes-kernel/linux-yocto-hailo/* directory, create the file *linux-yocto-hailo.bbappend* and add the following lines:

```
FILESEXTRAPATHS:prepend := "${THISDIR}:"  
SRC_URI:append = " file://cfg/custom-sensor-configuration.cfg"
```

3. Use 'devtool' to locally customize the repository and include it in the Makefile:

```
devtool modify linux-yocto-hailo
```

The local repository should now be present under *meta-hailo-soc/build/workspace/sources/linux-yocto-hailo*.

4. Make changes in 'drivers/media/i2c/':

- In *Kconfig*, add a *config VIDEO_IMX000* entry. You may use other config entries as reference.
- In the *Makefile*, add the following line to include the *imx000* driver:

```
obj-$(CONFIG_VIDEO_IMX000) += imx000.o
```

5. Troubleshoot: If the kernel object does not compile, try removing the cache from the local (devtool) *linux-yocto-hailo*:

```
rm -rf oe* .kernel-meta .metadir .config*
```

Done! Now our Yocto environment is ready for the next step – Kernel Driver Development.

5.6. Kernel Driver Development

The goal of this section is to guide you through the steps needed to develop the sensor's kernel driver and, ultimately, capture a raw image from the new sensor. At this point, you should have the local repository *meta-hailo-soc/build/workspace/sources/linux-yocto-hailo/*, created using the *devtool* command, and modified to accommodate the new driver.

To ensure that the local repository will be used during compilation instead of the external one, run *devtool status* and verify that it shows:

```
linux-yocto-hailo: path/to/meta-hailo-soc/build/workspace/sources/linux-yocto-hailo
```

Now let's add the driver:

1. **Create the driver source file for the sensor** (*imx000.c* in our case) in the local repository, under *drivers/media/i2c/*.
2. In our chosen framework, the driver's main purpose relates to the *v4l2* functionality. The *v4l2* sub-device is initialized in the probe function. You are encouraged to refer to existing drivers published by Hailo, such as *imx675.c*, *imx678.c*, and others as you proceed with writing the following functions that are required for a successful probe of the sensor, listed below. Register maps, supported resolutions, formats, and other details should be retrieved from the sensor's datasheet.
3. For supporting the *v4l* video standard, a minimal set of *v4l2_subdev_ops* should be implemented. These can be separated into two groups

3.1 v4l2_subdev_video_ops

```
/* V4L2 subdevice ops */
static const struct v4l2_subdev_video_ops imx000_video_ops = {
    .s_stream = imx000_set_stream,
};
```

3.1.1 s_stream is the most basic v4l2 video operation. It is in charge of turning the video stream on/off

```
/**
 * imx000_set_stream() - Enable sensor streaming
 * @sd: pointer to imx000 subdevice
 * @enable: set to enable sensor streaming
 *
 * Return: 0 if successful, error code otherwise.
 */
static int imx000_set_stream(struct v4l2_subdev *sd, int enable);
```

3.2 v4l2_subdev_pad_ops

```
static const struct v4l2_subdev_pad_ops imx000_pad_ops = {
    .init_cfg = imx000_init_pad_cfg,
    .enum_mbus_code = imx000_enum_mbus_code,
    .enum_frame_size = imx000_enum_frame_size,
    .get_fmt = imx000_get_pad_format,
    .set_fmt = imx000_set_pad_format,
};
```

3.2.1 init_cfg - Initialized the subdevice pad configuration (for the linux media device framework)

```
/**
 * imx000_init_pad_cfg() - Initialize sub-device pad configuration
 * @sd: pointer to imx000 V4L2 sub-device structure
 * @sd_state: V4L2 sub-device state
 *
 * Return: 0 if successful, error code otherwise.
 */
static int imx000_init_pad_cfg(struct v4l2_subdev *sd,
    struct v4l2_subdev_state *sd_state)
```

3.2.2 enum_mbus_code - Enumerates v4l2 sub-device mbus codes (for linux media framework)

```
/**
 * imx000_enum_mbus_code() - Enumerate V4L2 sub-device mbus codes
 * @sd: pointer to imx000 V4L2 sub-device structure
 * @sd_state: V4L2 sub-device state
 * @code: V4L2 sub-device code enumeration need to be filled
 *
 * Return: 0 if successful, error code otherwise.
 */
static int imx000_enum_mbus_code(struct v4l2_subdev *sd,
    struct v4l2_subdev_state *sd_state,
    struct v4l2_subdev_mbus_code_enum *code)
```

3.2.3 enum_frame_size - Enumerates v4l2 sub-device frame sizes (for linux media framework)

```
/**
 * imx000_enum_frame_size() - Enumerate V4L2 sub-device frame sizes
 * @sd: pointer to imx000 V4L2 sub-device structure
 * @sd_state: V4L2 sub-device state
 * @fsize: V4L2 sub-device size enumeration needs to be filled
 *
```

(continues on next page)

(continued from previous page)

```

* Return: 0 if successful, error code otherwise.
*/
static int imx000_enum_frame_size(struct v4l2_subdev *sd,
    struct v4l2_subdev_state *sd_state,
    struct v4l2_subdev_frame_size_enum *fsize);

```

3.2.4 get_fmt - Get the subdevice pad format (resolution, data format)

```

/**
 * imx000_get_pad_format() - Get subdevice pad format
 * @sd: pointer to imx000 V4L2 sub-device structure
 * @sd_state: V4L2 sub-device state
 * @fmt: V4L2 sub-device format need to be set
 *
 * Return: 0 if successful, error code otherwise.
 */
static int imx000_get_pad_format(struct v4l2_subdev *sd,
    struct v4l2_subdev_state *sd_state,
    struct v4l2_subdev_format *fmt);

```

3.2.5 set_fmt Set the subdevice pad format (resolution, data format)

```

/**
 * imx000_set_pad_format() - Set subdevice pad format
 * @sd: pointer to imx000 V4L2 sub-device structure
 * @sd_state: V4L2 sub-device state
 * @fmt: V4L2 sub-device format need to be set
 *
 * Return: 0 if successful, error code otherwise.
 */
static int imx000_set_pad_format(struct v4l2_subdev *sd,
    struct v4l2_subdev_state *sd_state,
    struct v4l2_subdev_format *fmt);

```

These should be populated as callbacks of v4l2_subdev_ops

```

static const struct v4l2_subdev_ops imx000_subdev_ops = {
    .video = &imx000_video_ops,
    .pad = &imx000_pad_ops,
};

```

The subdev ops structure is used to initialise the subdevice during the driver probing.

```

/* Initialize subdev */
v4l2_i2c_subdev_init(&imx000->sd, client, &imx000_subdev_ops);

```

3.3 Additionally it is recommended to implement and populate the power management operations

```

static const struct dev_pm_ops imx000_pm_ops = { SET_RUNTIME_PM_OPS(
    imx000_power_off, imx000_power_on, NULL) };

```

3.3.1 power_on - power on function for power management framework

```

/**
 * imx000_power_on() - Sensor power on sequence
 * @dev: pointer to i2c device
 *
 * Return: 0 if successful, error code otherwise.
 */
static int imx000_power_on(struct device *dev);

```

3.3.2 power_off - power off function for power management framework


```
/**
 * imx000_power_off() - Sensor power off sequence
 * @dev: pointer to i2c device
 *
 * Return: 0 if successful, error code otherwise.
 */
static int imx000_power_off(struct device *dev);
```

These are used as part of the driver binding

- For a successful driver binding, set the compatible field to the correct value, in our case sony,imx000.

```
static const struct of_device_id imx000_of_match[] = {
    { .compatible = "sony,imx000" },
    {}
};
```

And implement probe & remove functions

```
/**
 * imx000_probe() - I2C client device binding
 * @client: pointer to i2c client device
 *
 * Return: 0 if successful, error code otherwise.
 */
static int imx000_probe(struct i2c_client *client);

/**
 * imx000_remove() - I2C client device unbinding
 * @client: pointer to I2C client device
 *
 * Return: 0 if successful, error code otherwise.
 */
static int imx000_remove(struct i2c_client *client);
```

And finally to bind the driver:

```
static struct i2c_driver imx000_driver = {
    .probe_new = imx000_probe,
    .remove = imx000_remove,
    .driver = {
        .name = "imx000",
        .pm = &imx000_pm_ops,
        .of_match_table = imx000_of_match,
    },
};
```

- Lastly, modify the device tree to describe the new sensor. Assuming the porting is done on a hailo15 SBC board, edit arch/arm64/boot/dts/hailo/hailo15-sbc.dts. Change the compatible field to accommodate the new driver. The camera-sensor devices that are supported can be removed, however, if the decision is to keep them, one should modify the detect function in the driver accordingly.

```
&i2c_0 {
    status = "ok";

    sensor_0: camera-sensor@1a {
        status = "ok";
        compatible = "sony,imx000";
```

(continues on next page)

(continued from previous page)

```

reg = <0x1a>;
clocks = <&sensor_clk>;
clock-names = "inclk";
clock-frequency = <24000000>;
csi-id = <0>;
reset-gpios = <&gpio0 4 GPIO_ACTIVE_HIGH>;
xmaster-gpios = <&gpio0 3 GPIO_ACTIVE_HIGH>;
port {
    sensor_out_csi2rx: endpoint {
        data-lanes = <1 2 3 4>;
        remote-endpoint = <&csi2rx_in_sensor>;
        link-frequencies = /bits/ 64 <891000000 1440000000 1782000000>;
    };
};
};
};
};

```

Once the device tree accurately describes the new sensor and is aligned with the driver, one may proceed with the compilation.

1. Compile a new image with the new driver and device-tree:

```
bitbake core-image-hailo-dev
```

The image .wic file should be generated under: *meta-hailo-soc/build/tmp/deploy/images/hailo15-sbc/*

Alternatively, for faster debug iterations, deploy directly to the target (Hailo15-SBC), if the Yocto development machine has Ethernet access to it:

```
devtool deploy-target core-image-hailo-dev 10.0.0.1
```

2. Boot up the system.

3. Verify the module has been loaded properly:

- **3.a. The new kernel object will be present at:** */lib/modules/5.15.32-yocto-standard/kernel/drivers/media/i2c/imx000.ko'*
- **3.b.** Review the *dmesg* log to check for any boot issues with the module.
- **3.c.** Run *lsmod* to verify that the module has been loaded:

```
lsmod
```

```

root@hailo15:~$ lsmod
Module                  Size  Used
hailo15_isp             90112  1
hailo15_video_cap       36864  3
hailo15_video_out       24576  0
hailo15_rxwrapper       20480  1
hailo15_pixel_mux       16384  1
crct10dif_ce            20480  1
cdns_csi2rx             24576  1
imx334                  24576  0
hailo_integrated_nnc     73728  0
hx280enc                40960  0
imx000                  28672  1
fuse                    131072  1

```

- **3.d.** Verify the sensor, imx000 in our case, is registered as one of the v4l sub-devices:

4. Capture a raw image from the v4l sub-device, bypassing the ISP.

```
root@hailo15:/sys/class/video4linux$ cat v4l-subdev*/name
hailo15_pixel_mux.7c01c000.hail
cdns_csi2rx.7c011000.csi-bridge
imx000 0-001a
hailo15_rxwrapper.0
hailo-isp
```

A successful result should be a coherent image, in the expected resolution, which is not cut or deformed.

Use the `raw_image_capture` tool, which should already be present in your Linux environment.

Run `raw_image_capture` to display the usage instructions. In our `imx000` example, the format supported is a 12-bit 4K, as defined by the driver's supported modes, so we can use the capture tool as follows:

```
raw_image_capture frame_4k.raw -1 -1 imx000 12 3840 2160
```

```
root@hailo15:~$ raw_image_capture frame_4k.raw -1 -1 imx000 12 3840 2160
Found subdev cdns_csi2rx device /dev/v4l-subdev1
entered LEF gain: -1
entered LEF exposure: -1
entered sensor name: imx000
entered bpp: 12
Starting raw image capture, mode: sdr, width: 3840, height: 2160
Found subdev imx000 device /dev/v4l-subdev2
sending VIDIOC_REQBUFS
Starting capture
sending VIDIOC_G_CTRL exposure
sending VIDIOC_G_CTRL gain
LEF Exposure is now: 1608, gain is now: 0
wrote 16588800 bytes successfully
Stopping capture
finished
```

Another option for achieving the same target is by using the `v4l_stream_example`. It provides more flow control and can be used for debugging. However, it will require compilation because the pre-compiled binary already on your filesystem supports specific sensors. This will be changed in future releases.

The source for this example is in the imaging-sub-system repository:

`imaging-sub-system/units/hailo/external_examples/v4l_stream_example/`

After generating the raw image and verifying the image is coherent, at the correct resolution, etc. we can proceed to writing the ISS driver.

5.7. ISS Driver Development

This section will guide you through the steps needed to develop the sensor's ISS driver and, ultimately, get a stream from `/dev/video0`, meaning with ISP integration this time. Additionally, we aim to ensure the 3A configuration works during the stream process.

5.7.1. Setting Up Yocto for ISS Driver Development

Similar to the preparation work done in the "Setting Up Yocto for Kernel Driver Development" section, this section focuses on preparing the Yocto environment for compiling and installing the ISS driver on the image. As before, it is assumed that a customization layer `meta-<vendor>` exists, and that the development and setup will continue for the same sensor, "imx000".

If not already active, activate the Yocto environment by running the following command from `meta-hailo-soc`:

```
. poky/oe-init-build-env
```

Create the required directory in the meta. Assuming you're in the build directory:

```
mkdir -p ../meta-<vendor>/recipes-imaging/imaging-sub-system/
```

Create the file *iss-drivers.bbappend* in the new *imaging-sub-system* directory and override *SENSORS_LIBS* with the new driver lib we're about to add:

```
SENSORS_LIBS = "HAILO_IMX000"
```

Use *devtool* to locally customize the *iss-drivers* repository:

```
devtool modify iss-drivers
```

Inside the local *iss-drivers* repository, create a folder for the new sensor, following the structure of existing Hailo sensor folders. This folder should contain the source, headers, *CMakeLists.txt*, and calibration XML file. If you do not yet have a calibration XML, copy one from an existing sensor.

For this stage, we'll focus only on renaming the functions within the source files, headers, and *CMakeLists.txt* to match the new sensor (*imx000*), without making modifications to the function implementations. Specific adjustments to each function will come in the next steps of the development process.

Example Directory Structure: Output of "tree" command under *build/workspace/sources/iss-drivers/imaging-sub-system/scripts/units/isi/drv/HAILO_IMX000*:

```
tree
.
├── calib
│   └── IMX000
│       └── HAILO_IMX000.xml
├── CMakeLists.txt
├── include_priv
│   └── IMX000_priv.h
├── source
│   └── IMX000.c
```

Modify the calibration XML file to reflect the sensor's specific resolution. Search and replace all resolution values to match those of the new sensor. Failure to do this can lead to failures in the *STREAMON* ioctl call.

Build and deploy the image using *bitbake* or *devtool* as specified in the [Kernel Driver Development](#) section.

5.7.2. Sensor Configuration After ISS Driver Deployment

A configuration file for the sensor should be found in the H15 filesystem at */usr/bin/Sensor0_entry.cfg*. This file helps the ISP media server configure and load the sensor correctly. This configuration file is simply a reference file that needs to be edited, so after a successful Yocto build and deployment of the sensor ISS driver, modify it to point to the new driver. In future releases, multiple sensors will be supported, and additional configuration files (*Sensor1_entry.cfg*, *Sensor2_entry.cfg*, and so on) will be used.

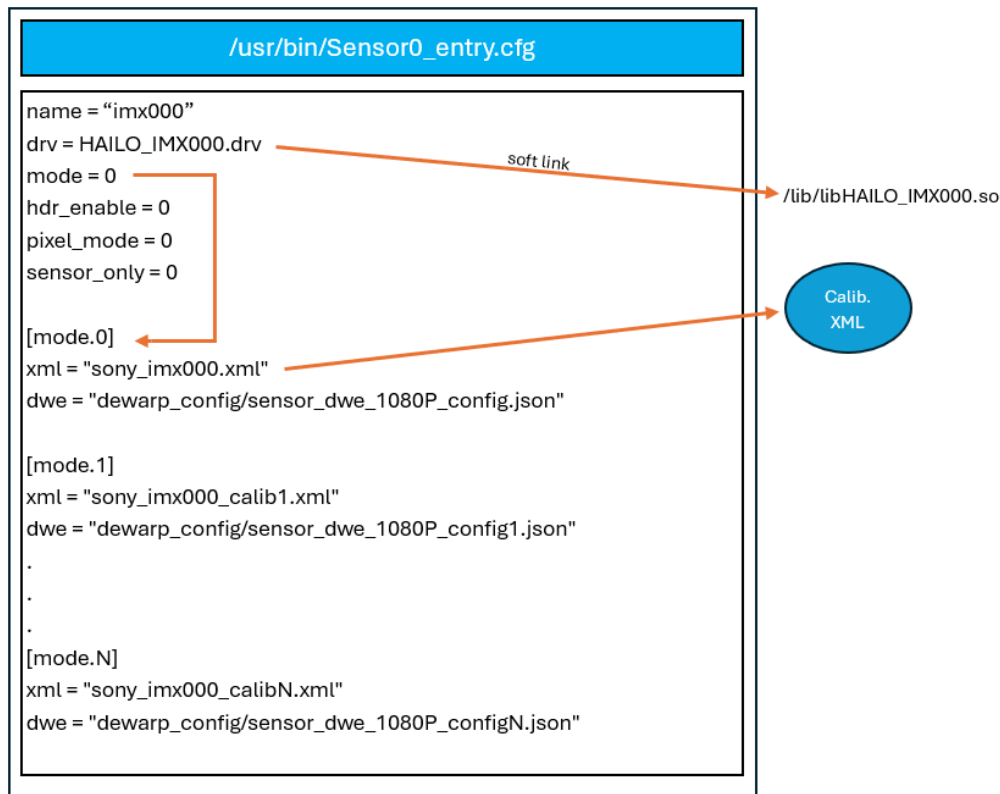
As shown in the image below, this is the expected structure of the configuration file:

- The *name* field must match the kernel driver name specified in the *struct i2c_driver* structure's *.driver.name* field. For example, if the kernel driver defines *.driver.name* = "*imx000*", then the configuration file should have *name*="*imx000*".
- The configuration file points to the ISS driver's link, *HAILO_IMX000.drv*, which should be linked to the shared object file (*HAILO_IMX000.so* / *HAILO_IMX000.so.<version_number>* - both of which point to the same file), produced during the build.

```
tree
├── lib
│   ├── libHAILO_IMX000.so -> libHAILO_IMX000.so.1.0.0
│   └── libHAILO_IMX000.so.1.0.0
├── usr
│   └── bin
│       ├── HAILO_IMX000.drv -> ../../lib/libHAILO_IMX000.so.1.0.0
│       ├── sony_imx000_fhd.xml
│       ├── sony_imx000_hdr.xml
│       └── sony_imx000.xml
```

- The selected sensor *mode* in the current configuration. Each mode is linked to a specific sensor calibration (Applies to all Hailo-15 flavours) and dewarp configuration (applies only to Hailo-15L). The sensor might support multiple modes for different scenarios.
- The *sensor_i2c_bus* and *sensor_i2c_address* fields should be set to the correct values for the new sensor. These values are used by the ISS driver to communicate with the sensor over the I2C bus.
- The *af_i2c_bus* and *af_i2c_address* fields are for future use and are not currently supported.
- The *custom_readout_timing_short* is a field that can be used to set the custom readout timing for short exposure (e.g. RHS1 in many Sony sensors)

Verify that this structure is intact and that the necessary files, such as the driver, calibration XMLs, and configuration files, are properly located. Incorrect file paths or missing files will prevent the sensor from working properly.



5.7.3. Testing ISP-Enabled Streaming

If everything was set up correctly, it should now be possible to obtain a stream from `/dev/video0`. Assuming the H15 platform has an Ethernet connection to the development machine, the following GStreamer pipelines can be run from the terminal:

On the H15 board:

```
GST_DEBUG=3 gst-launch-1.0 -v v4l2src device=/dev/video0 ! video/x-raw, format=NV12,
↳ framerate=30/1, width=720, height=480 ! gdpdpay ! tcpserver sink host=10.0.0.1
↳ port=3000
```

On the development machine:

```
gst-launch-1.0 -v tcpclientsrc host=10.0.0.1 port=3000 ! gdpdpay ! videoconvert !
↳ ximagesink sync=false
```

If `gdp` is missing from the development machine, install it with:

```
sudo apt-get install gstreamer1.0-plugins-bad
```

A successful test would result in a stream being displayed on the development machine. The stream quality may appear suboptimal, potentially even entirely dark or overly bright. This is expected, as further tuning is necessary to adjust the driver functions to the specific sensor characteristics, including 3A configurations, on top of the “real” calibration, which may be missing at this point.

The following section will provide an in-depth look at the ISS driver functions and include a full API reference to support these adjustments. Refer to the sensor’s datasheet to make appropriate parameter modifications for a clear and balanced stream.

5.7.4. ISS Driver Flow

This sections contains the structures and API references needed for the ISS Driver development

Sensor Callbacks

The various sensor operations are implemented and populated as the callbacks of *IsiSensor_s* data structure. For a successful sensor driver binding, it is required to populate *IsiSensor_s* structure, camera driver ID and *IsiGetSensorIss_t* callbacks into the *IsiCamDrvConfig_s* data structure. The sensor callback binding should be done in the *IsiGetSensorIss_t* callback. The ISP SW will operate the sensor by accessing the sensor API’s via this structure. for example: Sensor gain control & exposure time control for AEC operations.

Create ISS API in ISI Layer

1. Type def the sensor Context_t, *IMX000_Context_t* in this example, for the sensor handle. This should be done in the *include_priv* header file, *IMX000_priv.h* in this case.

```
typedef struct IMX000_Context_s
{
    IsiSensorContext_t IsiCtx;          /**< common context of ISI and ISI driver
    ↳layer; @note: MUST BE FIRST IN DRIVER CONTEXT */

    struct vvsensor_mode_s SensorMode;
    uint32_t      KernelDriverFlag;
    char          SensorRegCfgFile[64];

    uint32_t      HdrMode;
    uint32_t      Resolution;
    uint32_t      MaxFps;
    uint32_t      MinFps;
    uint32_t      CurrFps;

    IsiCaps_t      CapsConfig;
    bool_t         Configured;
    bool_t         Streaming;
    bool_t         TestPattern;

    bool_t         isAfpsRun;

    float          one_line_exp_time;
    uint32_t       MaxIntegrationLine;
    uint16_t       MinIntegrationLine;
    uint32_t       gain_accuracy;

    uint32_t       original_vmax;
    uint32_t       cur_rhs1;
    uint32_t       cur_rhs2;
```

(continues on next page)

(continued from previous page)

```

int            unlimit_fps;

uint16_t       FrameLengthLines;
uint16_t       CurFrameLengthLines;

float          AecMinGain;
float          AecMaxGain;

float          AecMinIris;
float          AecMaxIris;
float          AecIris;

float          AecMinIntegrationTime;
float          AecMaxIntegrationTime;

float          AecIntegrationTimeIncrement;
float          AecGainIncrement;

float          AecCurGainLEF;
float          AecCurIntegrationTimeLEF;
float          AecCurGainSEF1;
float          AecCurIntegrationTimeSEF1;
float          AecCurGainSEF2;
float          AecCurIntegrationTimeSEF2;

bool           GroupHold;
uint32_t       OldGain;
float          OldIntegrationTime;
uint32_t       OldGainSEF1;
uint32_t       OldIntegrationTimeSEF1;

int            subdev;
bool           enableHdr;
uint8_t        pattern;
int            i2c_fd;
} IMX000_Context_t;

```

2. Implement *IsiCamDrvConfig* for the sensor library callback function. Define the *IsiCamDrvConfig_s* data structure. Data members defined in this data structure include the sensor ID (CameraDriverID) and the function pointer to the *IsiSensor_s* data structure. Using the address of the *IsiCamDrvConfig_s* structure, the driver can then access the sensor API attached to the function pointer.

```

IsiCamDrvConfig_t IsiCamDrvConfig = {
    .CameraDriverID = 0x0000,
    .pIsiHalEnumModeIss = IMX000_IsiHalEnumModeIss,
    .pIsiGetSensorIss = IMX000_IsiGetSensorIss,
};

```

Notes:

- IsiCamDrvConfig is defined in source/IMX000.c.
- IMX000_IsiHalQuerySensorIss() uses the IOCTL command VVSENSORIOC_QUERY to get all the modes supported by the sensor.
- IMX000_IsiGetSensorIss() can initialize the *IsiSensor_s* data structure. It is called by an upper-level application described in the *Sensor Callbacks* section. Then the application can get the address of all the callback functions.

Structures

The structures specified below are in use by the ISS Driver.

IsiSensorInstanceConfig_s

This structure defines the configuration structure used to create a new sensor instance.

Table 5. IsiSensorInstanceConfig_s Structure

Structure Members	Type	Description
HalHandle	HalHandle_t	Handle of HAL session to use
HalDevID	uint32_t	HAL device ID of this sensor
I2cBusNum	uint8_t	The I2C bus the sensor is connected to
SlaveAddr	uint16_t	The I2C slave address the sensor is configured to
I2cAfBusNum	uint8_t	The I2C bus the AF module is connected to
SlaveAfAddr	uint16_t	The I2C slave address of the configured AF module
SensorModelIndex	uint32_t	The current sensor mode index
*pSensor	IsiSensor_t	The pointer to the sensor driver interface
hSensor	IsiSensorHandle_t	Sensor handle returned by IsiCreatelss()
szSensorNodeName[32]	char	Unused.

IsiCamDrvConfig_s

This structure defines camera sensor driver specific data.

Table 6. IsiCamDrvConfig_s Structure

Structure Mem- bers	Type	Description
CameraDriverID	uint32_t	Camera sensor driver ID
*pIsiHalEnum- Modelss	IsiHalEnum- Modelss_t	Query sensor mode with HAL handle.
*pIsiGetSen- sorlss	IsiGetSen- sorlss_t	The function pointer to initialize the member IsiSensor in current structure.
IsiSensor	IsiSensor_s	The structure includes the sensor name and the function pointers to control the sensor in the ISI layer.

IsiSensor_s

This structure defines attributes and function pointers for the sensor.

Table 7. IsiSensor_s Structure

Structure Members	Type	Description
*pszName	const char	Name of the camera-sensor.
*pRegisterTable	IsiRegisterFlags_s	Pointer to register table.
*pIsiCaps	Isicaps_t	Pointer to sensor capabilities.

Continued on next page

Table 7 – continued from previous page

Structure Members	Type	Description
plsiCreatelss	IsiCreatelss_t	Create a sensor handle.
plsiReleaselss	IsiReleaselss_t	Release a sensor handle.
plsiReadReglss	IsiReadReglss_t	Read sensor register.
plsiWriteReglss	IsiWriteReglss_t	Write sensor register.
plsiGetModelss	IsiGetModelss_t	Get sensor mode.
plsiSetModelss	IsiSetModelss_t	Set sensor mode.
plsiEnumModelss	IsiEnumModelss_t	Enum sensor mode.
plsiGetCapslss	IsiGetCapslss_t	Get sensor capabilities.
plsiSetuplss	IsiSetuplss_t	Setup sensor capabilities.
plsiCheckConnectionlss	IsiCheckConnectionlss_t	Check sensor connection with I2C.
plsiGetRevisionlss	IsiGetRevisionlss_t	Read sensor revision register (if available).
plsiSetStreaminglss	IsiSetStreaminglss_t	Enable/disable streaming of data once sensor is configured.
plsiSetPowerlss	IsiSetPowerlss_t	Turn sensor power on/off.
plsiExposureControllss	IsiExposureControllss_t	[AEC function]
plsiExposureControlExpandedlss	IsiExposureControlExpandedlss_t	[AEC function]
plsiUnlimitFpslss	IsiUnlimitFpslss_t	[AEC function] Allow dynamic change of FPS
plsiLimitFpslss	IsiLimitFpslss_t	[AEC function] Block dynamic change of FPS
plsiSetIrislss	IsiSetIrislss_t	[AEC function] Set the iris
plsiGetIrisLimitslss	IsiGetIrisLimitslss_t	[AEC function] Get the iris limits
plsiSetIrisLimitslss	IsiSetIrisLimitslss_t	[AEC function] Set the iris limits
plsiGetGainLimitslss	IsiGetGainLimitslss_t	[AEC function]
plsiGetIntegrationTimeLimitslss	IsiGetIntegrationTimeLimitslss_t	[AEC function]
plsiGetCurrentExposurelss	IsiGetCurrentExposurelss_t	[AEC function] Get the currently adjusted AE values (gain and integration time).
plsiGetGainlss	IsiGetGainlss_t	[AEC function]
plsiGetVSGainlss	IsiGetVSGainlss_t	[AEC function]
plsiGetLongGainlss	IsiGetLongGainlss_t	[AEC function]
plsiGetGainIncrementlss	IsiGetGainIncrementlss_t	[AEC function]
plsiSetGainlss	IsiSetGainlss_t	[AEC function]
plsiGetIntegrationTimelss	IsiGetIntegrationTimelss_t	[AEC function]
plsiGetVSIIntegrationTimelss	IsiGetVSIIntegrationTimelss_t	[AEC function]
plsiGetLongIntegrationTimelss	IsiGetLongIntegrationTimelss_t	[AEC function]
plsiGetIntegrationTimeIncremen- tlss	IsiGetIntegrationTimeIncremen- tlss_t	[AEC function]
plsiSetIntegrationTimelss	IsiSetIntegrationTimelss_t	[AEC function]
plsiGetFpslss	IsiGetFpslss_t	[FPS function]
plsiSetFpslss	IsiSetFpslss_t	[FPS function]

Continued on next page

Table 7 – continued from previous page

Structure Members	Type	Description
plsiGetAutoFpsInfolss	IsiGetAutoFpsInfolss_t	[FPS function]
plsiGetStartEvIss	IsiGetStartEvIss_t	[AEC function]
plsiGetAgainDgainIss	IsiGetAgainDgainIss_t	Get AGain and DGain set by user
plsiSetAgainDgainIss	IsiSetAgainDgainIss_t	Set AGain and DGain by user
plsiGetIspStatusIss	IsiGetIspStatusIss_t	Get ISP status (AE, BLC, WB).
plsiSetBclIss	IsiSetBclIss_t	Set sensor BLC.
plsiSetWbIss	IsiSetWbIss_t	Set sensor WB.
plsiSetTpgIss	IsiSetTpgIss_t	Set sensor test pattern.
plsiGetTpgIss	IsiGetTpgIss_t	Get sensor test pattern
plsiGetExpandCurveIss	IsiGetExpandCurveIss_t	Get sensor expand curve.
plsiGetCompressCurveIss	IsiGetCompressCurveIss_t	Get sensor compress curve.
plsiExtendFuncIss	IsiExtendFuncIss_t	Reserved, Extend function.
plsiGetOtpDataIss	IsiGetOtpDataIss_t	Get sensor OTP data.
plsiGetHdrBlankingLines	IsiGetHdrBlankingLinesIss_t	Get sensor HDR blanking lines configuration (amount of LEF lines before other exposures first line from sensor)
plsiFocusCreateIss	IsiFocusCreateIss_t	[AF function] Reserved
plsiFocusReleaseIss	IsiFocusReleaseIss_t	[AF function] Reserved
plsiFocusGetCalibrateIss	IsiFocusGetCalibrateIss_t	[AF function] Reserved
plsiFocusSetIss	IsiFocusSetIss_t	[AF function] Reserved
plsiFocusGetIss	IsiFocusGetIss_t	[AF function] Reserved

ISS Sensor API reference

This section describes the API defined in the ISS driver, *IMX000.c* in this case. Users may implement a subset of the API's defined in this table. API's that are not populated, will not be available for control by the ISP sw. The upper application layer can use the structure of *IsiCamDrvConfig_s* to call the following functions.

Table 8. Sensor API Functions and Description

Sensor API	Description
SENSOR STRUCTURES	
IsiCamDrvConfig_s IsiCamDrvConfig{}	Provide a structure for upper layer to access function pointer
SENSOR FUNCTIONS	
IMX000_IsiSetPowerIss(...)	Power-up/power-down the sensor
IMX000_IsiReadRegIss(...)	Read sensor register value
IMX000_IsiWriteRegIss(...)	Write sensor register value
IMX000_IsiCreateIss(...)	Create a new sensor instance and assign resources to sensor
IMX000_IsiReleaseIss(...)	Release a sensor instance
IMX000_IsiGetModelIss(...)	Get sensor mode information
IMX000_IsiSetModelIss(...)	Set sensor mode index
IMX000_IsiEnumModelIss(...)	Query support sensor mode

Continued on next page

Table 8 – continued from previous page

Sensor API	Description
IMX000_IsiGetCapsIss(...)	Get sensor caps ability
IMX000_IsiSetupIss(...)	Set sensor format and initialize sensor
IMX000_IsiCheckConnectionIss(...)	Check sensor connect status
IMX000_IsiGetRevisionIss(...)	Get sensor ID
IMX000_IsiSetStreamingIss(...)	Set sensor streaming status
IMX000_IsiGetIntegrationTimeIss(...)	Get exposure time s
IMX000_IsiSetIntegrationTimeIss(...)	Set exposure time s
IMX000_IsiGetGainIss(...)	Get sensor gain
IMX000_IsiSetGainIss(...)	Set sensor gain
IMX000_IsiGetFpsIss(...)	Get sensor frame rate (fixed point q10)
IMX000_IsiSetFpsIss(...)	Set sensor frame rate (fixed point q10)
IMX000_IsiGetAutoFpsInfoIss(...)	Get sensor auto frame rate information (fixed point q10)
IMX000_IsiGetIspStatusIss(...)	Get sensor module status (BLC, WB)
IMX000_IsiGetStartEvIss(...)	Get AE start exposure
IMX000_IsiSetBliIss(...)	Set sensor BLC
IMX000_IsiSetWBIss(...)	Set sensor WB
IMX000_IsiGetExpandCurveIss(...)	Get sensor expand curve
IMX000_IsiGetCompressCurveIss(...)	Stitch HDR and native HDR 16-/20-bit to 12-bit compressed curve
IMX000_IsiSetTpgIss(...)	Set sensor test pattern mode
IMX000_plsiGetOtpDataIss	Get sensor OTP data
IMX000_plsiFocusCreateIss	[AF function] Reserved
IMX000_plsiFocusReleaseIss	[AF function] Reserved
IMX000_plsiFocusGetCalibrateIss	[AF function] Reserved
IMX000_plsiFocusSetIss	[AF function] Reserved
IMX000_plsiFocusGetIss	[AF function] Reserved

Part II

API Reference

6. V4L Control

6.1. AE

The Auto Exposure module is designed as a general statistics module for ISP automatic exposure control with the AE control loop implemented in the software. Note: Both in set/get the digital gain module control/gain/integration time control must disable ae

Command	Description	Parameters	Control ID
hailo_ctrl -d /dev/video0 -c isp_ae_enable=1	Disable or enable ae.	int 0/1	ISP_CID_AE_ENABLE
hailo_ctrl -d /dev/video0 -C isp_ae_enable	Disable or enable ae.	int 0/1	ISP_CID_AE_ENABLE
hailo_ctrl -d /dev/video0 -c isp_ae_reset=1	Reset ae., both 0 and 1 will reset ae.	int 0/1	ISP_CID_AE_RESET
hailo_ctrl -d /dev/video0 -c isp_ae_sem_mode=2	Scene evaluation mode. 0: disable 1: fix 2: adaptive	int 0~2	ISP_CID_AE_SEM_MODE
hailo_ctrl -d /dev/video0 -C isp_ae_sem_mode	Scene evaluation mode. 0: disable 1: fix 2: adaptive	int 0~2	ISP_CID_AE_SEM_MODE
hailo_ctrl -d /dev/video0 -c isp_ae_flicker_period=1	ae flicker period. 0: Off 1:100Hz 2: 120Hz	int 0~2	ISP_CID_AE_FLICKER_PERIOD
hailo_ctrl -d /dev/video0 -C isp_ae_flicker_period	ae flicker period. 0: Off 1:100Hz 2: 120Hz	int 0~2	ISP_CID_AE_FLICKER_PERIOD
hailo_ctrl -d /dev/video0 -c isp_ae_afps=0	ae afps mode. 0: Disable 1:Enable	int 0/1	ISP_CID_AE_AFPS
hailo_ctrl -d /dev/video0 -C isp_ae_afps	ae afps mode. 0: Disable 1:Enable	int 0/1	ISP_CID_AE_AFPS
hailo_ctrl -d /dev/video0 -c isp_ae_setpoint=50	ae setpoint.	int 0~255	ISP_CID_AE_SETPOINT
hailo_ctrl -d /dev/video0 -C isp_ae_setpoint	ae setpoint.	int 0~255	ISP_CID_AE_SETPOINT
hailo_ctrl -d /dev/video0 -c isp_ae_tolerance=36	Tolerance of stable range.	int 0~100	ISP_CID_AE_TOLORANCE
hailo_ctrl -d /dev/video0 -C isp_ae_tolerance	Tolerance of stable range.	int 0~100	ISP_CID_AE_TOLORANCE
hailo_ctrl -d /dev/video0 -c isp_ae_dampover=80	Damp over coefficient.	int 0~100	ISP_CID_AE_DAMPOVER
hailo_ctrl -d /dev/video0 -C isp_ae_dampover	Damp over coefficient.	int 0~100	ISP_CID_AE_DAMPOVER
hailo_ctrl -d /dev/video0 -c isp_ae_dampunder=20	Damp under coefficient.	int 0~100	ISP_CID_AE_DAMPUNDER
hailo_ctrl -d /dev/video0 -C isp_ae_dampunder	Damp under coefficient.	int 0~100	ISP_CID_AE_DAMPUNDER
hailo_ctrl -d /dev/video0 -C isp_ae_hist	Isp histogram	uint32_t 1x256 array	ISP_CID_AE_HIST
hailo_ctrl -d /dev/video0 -C isp_ae_hist64	Isp histogram64	uint32_t 1x32 array	ISP_CID_AE_HIST64
hailo_ctrl -d /dev/video0 -c isp_ae_hist64_mode=1	Isp histogram64 mode	int 0~5	ISP_CID_AE_HIST64_MODE
hailo_ctrl -d /dev/video0 -C isp_ae_hist64_mode	Isp histogram64 mode	int 0~5	ISP_CID_AE_HIST64_MODE
hailo_ctrl -d /dev/video0 -c isp_ae_hist64_channel=1	Isp histogram64 channel 0: de-gamma, 1: LSC, 2: AWB, 3: WDR, 4: Demosaic 5: cross talk, 6: reserved, 7: gama out	int 0~7	ISP_CID_AE_HIST64_CHANNEL
hailo_ctrl -d /dev/video0 -C isp_ae_hist64_channel	Isp histogram64 channel	int 0~7	ISP_CID_AE_HIST64_CHANNEL

Continued on next page

Table 9 – continued from previous page

Command	Description	Parameters	Control ID
hailo_ctrl /dev/video0 -C isp_ae_luma	Isp luminance avg luminance for 5x5 blocks	uint32_t 5x5 array	ISP_CID_AE_LUMA
hailo_ctrl -d /dev/video0 -t float -set-vector isp_ae_roi_weight=[0.2,0.2,0.4,0.5,0.6,0.1,0.1,0.5,0.3,0.5,0.6,0.1,0.4,0.2,0.4,0.1]	Weight of region of interest. according to roi number.	float 16x array 0.0~1.0	ISP_CID_AE_ROI_WEIGHT
hailo_ctrl -d /dev/video0 -t float -C isp_ae_roi_weight	Weight of region of interest. according to roi number.	float 16x array 0.0~1.0	ISP_CID_AE_ROI_WEIGHT
hailo_ctrl -d /dev/video0 -set-vector isp_ae_roi=<value>	Region of interest. [hOffset,vOffset,width,height]. according to roi number.	uint32_t 16x4 array	ISP_CID_AE_ROI
hailo_ctrl -d /dev/video0 -C isp_ae_roi	Region of interest. [hOffset,vOffset,width,height]. according to roi number.	uint32_t 16x4 array	ISP_CID_AE_ROI
hailo_ctrl -d /dev/video0 -c isp_ae_gain=1024	Analog gain and digital gain. Scalar multiplier x1024	int 1024~102389	ISP_CID_AE_GAIN
hailo_ctrl -d /dev/video0 -C isp_ae_gain	Analog gain and digital gain. Scalar multiplier x1024	int 1024~102389	ISP_CID_AE_GAIN
hailo_ctrl -d /dev/video0 -c isp_ae_integration_time=500	Integration time of sensor. Usec.	int 1~999999	ISP_CID_AE_INTEGRATION_TIME
hailo_ctrl -d /dev/video0 -C isp_ae_integration_time	Integration time of sensor. Usec.	int 1~999999	ISP_CID_AE_INTEGRATION_TIME
hailo_ctrl -d /dev/video0 -C isp_ae_exp_status	Exposure status: 1=converged, 0= converging.	uint32_t 4096x4 array	ISP_CID_AE_EXP_STATUS
hailo_ctrl -d /dev/video0 -c isp_ae_iris=100	sensor iris value. Fnum.	Int 100~99999	ISP_CID_AE_IRIS
hailo_ctrl -d /dev/video0 -C isp_ae_iris	sensor iris value. Fnum.	Int 100~99999	ISP_CID_AE_IRIS
hailo_ctrl -d /dev/video0 -C isp_ae_iris_limits = [0.0,15.0]	Sensor iris limits. Fnum.	uint32_t 1x12 array	ISP_CID_AE_IRIS_LIMITS
hailo_ctrl -d /dev/video0 -C isp_ae_iris_limits	Sensor iris limits. Fnum.	uint32_t 1x12 array	ISP_CID_AE_IRIS_LIMITS
hailo_ctrl -d /dev/video0 -c isp_dg_enable=0	Disable or enable digital gain.	int 0/1	ISP_CID_DG_ENABLE
hailo_ctrl -d /dev/video0 -c isp_dg_gain=100	isp digital gain. Scalar multiplier x100	int 100~25599	ISP_CID_DG_GAIN
hailo_ctrl -d /dev/video0 -C isp_dg_gain	isp digital gain. Scalar multiplier x100	int 100~25599	ISP_CID_DG_GAIN
hailo_ctrl -d /dev/video0 -c isp_ae_sp_ratio=1600	set point ratio. This is the ratio between lef and sef. Fnum.	int 100~25500	ISP_CID_AE_SP_RATIO
hailo_ctrl -d /dev/video0 -C isp_ae_sp_ratio	set point ratio. This is the ratio between lef and sef. Fnum.	int 100~25500	ISP_CID_AE_SP_RATIO
hailo_ctrl -d /dev/video0 -c isp_ae_lef_threshold=20	threshold that below it, a patch is assumed from lef	int 0~255	ISP_CID_AE_LEF_THRESHOLD
hailo_ctrl -d /dev/video0 -C isp_ae_lef_threshold	threshold that below it, a patch is assumed from lef	int 0~255	ISP_CID_AE_LEF_THRESHOLD
hailo_ctrl -d /dev/video0 -C isp_ae_sem_setpoint	modified setpoint after all AE loops	int 0~255	ISP_CID_AE_SEM_SETPOINT
hailo_ctrl -d /dev/video0 -C isp_ae_dynamic_range	Current dynamic range of the scene (0=bright, 1=hdr, 2=dark)	int 0~2	ISP_CID_AE_DYNAMIC_RANGE

6.2. AWB

Note: wb control must disable awb.

Command	Description	Parameters	Control ID
hailo_ctrl -d /dev/video0 -c isp_awb_enable=0	Disable or enable awb.	int 0/1	ISP_CID_AWB_ENABLE
hailo_ctrl -d /dev/video0 -c isp_awb_reset=0	Reset awb, both 0 and 1 will reset awb.	int 0/1	ISP_CID_AWB_RESET
hailo_ctrl -d /dev/video0 -C isp_awb_mode	awb working mode. 0: manual 1: auto	int 0/1	ISP_CID_AWB_MODE
hailo_ctrl -d /dev/video0 -C isp_awb_damping	Enable or disable damping.	int 0/1	ISP_CID_AWB_DAMPING
hailo_ctrl -d /dev/video0 -C isp_awb_illum_index	Illuminance profile index.	int 0~10	ISP_CID_AWB_ILLUM_INDEX
hailo_ctrl -d /dev/video0 -C isp_awb_illum_profiles	All illuminance profiles name.	string	ISP_CID_AWB_ILLUM_PROFILES
hailo_ctrl -d /dev/video0 -C isp_awb_illum_num	awb illuminance profiles count.	int 0~10	ISP_CID_AWB_ILLUM_NUM
hailo_ctrl -d /dev/video0 -C isp_awb_rg_proj_max	awb project max.	int 0~500	ISP_CID_AWB_RG_PROJ_MAX
hailo_ctrl -d /dev/video0 -C isp_awb_sky_rb_proj_max	awb sky project max.	int 0~500	ISP_CID_AWB_SKY_RB_PROJ_MAX
hailo_ctrl -d /dev/video0 -C isp_awb_indoor_rb_proj_min	awb indoor project max.	int 0~500	ISP_CID_AWB_INDOOR_RB_PROJ_MIN
hailo_ctrl -d /dev/video0 -C isp_awb_outdoor_rb_proj_min	awb outdoor project max.	int 0~500	ISP_CID_AWB_OUTDOOR_RB_PROJ_MIN
hailo_ctrl -d /dev/video0 -set-vector isp_awb_measure_window=[0,0,100,200]	awb measuring window with [hOffset, vOffset, width, height].	uint16_t 4x array 0~0xFFFF	ISP_CID_AWB_MEASURE_WINDOWDOWN
hailo_ctrl -d /dev/video0 -c isp_wb_r_gain=293	wb red channel gain.	int 100~1023	ISP_CID_WB_R_GAIN
hailo_ctrl -d /dev/video0 -C isp_wb_r_gain	wb red channel gain.	int 100~1023	ISP_CID_WB_R_GAIN
hailo_ctrl -d /dev/video0 -c isp_wb_gr_gain=101	wb gain of green channel in red row.	int 100~1023	ISP_CID_WB_GR_GAIN
hailo_ctrl -d /dev/video0 -C isp_wb_gr_gain	wb gain of green channel in red row.	int 100~1023	ISP_CID_WB_GR_GAIN
hailo_ctrl -d /dev/video0 -c isp_wb_gb_gain=101	wb gain of green channel in blue row.	int 100~399	ISP_CID_WB_GB_GAIN
hailo_ctrl -d /dev/video0 -C isp_wb_gb_gain	wb gain of green channel in blue row.	int 100~399	ISP_CID_WB_GB_GAIN
hailo_ctrl -d /dev/video0 -c isp_wb_b_gain=136	wb blue channel gain.	int 100~399	ISP_CID_WB_B_GAIN
hailo_ctrl -d /dev/video0 -C isp_wb_b_gain	wb blue channel gain.	int 100~399	ISP_CID_WB_B_GAIN
hailo_ctrl -d /dev/video0 -t float -set-vector isp_wb_cc_matrix=[1.586,-0.070,-0.461,-0.609,1.984,-0.234,0.008,-1.219,2.508]	Coefficient for cross talk matrix. Every channel sum of coefficients should be 1.	float 9x array	ISP_CID_WB_CC_MATRIX
hailo_ctrl -d /dev/video0 -t float -C isp_wb_cc_matrix	Coefficient for cross talk matrix. Every channel sum of coefficients should be 1.	float 9x array	ISP_CID_WB_CC_MATRIX
hailo_ctrl -d /dev/video0 -c isp_wb_cc_r_offset=-30	Red offset for the cross talk matrix.	int 2048~2047	ISP_CID_WB_CC_R_OFFSET
hailo_ctrl -d /dev/video0 -C isp_wb_cc_r_offset	Red offset for the cross talk matrix.	int 2048~2047	ISP_CID_WB_CC_R_OFFSET
hailo_ctrl -d /dev/video0 -c isp_wb_cc_g_offset=-14	Green offset for the cross talk matrix.	int 2048~2047	ISP_CID_WB_CC_G_OFFSET
hailo_ctrl -d /dev/video0 -C isp_wb_cc_g_offset	Green offset for the cross talk matrix.	int 2048~2047	ISP_CID_WB_CC_G_OFFSET
hailo_ctrl -d /dev/video0 -c isp_wb_cc_b_offset=-62	Blue offset for the cross talk matrix.	int 2048~2047	ISP_CID_WB_CC_B_OFFSET
hailo_ctrl -d /dev/video0 -C isp_wb_cc_b_offset	Blue offset for the cross talk matrix.	int 2048~2047	ISP_CID_WB_CC_B_OFFSET

6.3. AF

Command	Description	Parameters	Control ID
hailo_ctrl -d /dev/video0 -c isp_af_enable=0	Disable or enable auto focus.	int 0/1	ISP_CID_AF_ENABLE
hailo_ctrl -d /dev/video0 -set-vector isp_af_window=[[5,2,128,128], [256,256,128,128], [512,512,128,128]]	Auto focus windows selection.3 windows defined by 4 points each. Format is x,y,w,h. Windows size (w*h) should not exceed 128^3 at the risk of measurement overflow.	uint32_t 3x4 array. x>=5,y>=2. w+x, y+h cannot exceed image width/height	ISP_CID_AF_WINDOW
hailo_ctrl -d /dev/video0 -C isp_af_measurement	AF statistic measurement: sum1, sum2, sum3, luma1, luma2, luma3	uint32_t 1x6 array	ISP_CID_AF_MEASUREMENT

6.4. 2DNR

Note: set/get pre-gamma/strength/sigma control must run in manual mode.

Command	Description	Parameters	Control ID
hailo_ctrl -d /dev/video0 -c isp_2dnr_enable=0	Disable or enable 2dnr.	int 0/1	ISP_CID_2DNR_ENABLE
hailo_ctrl -d /dev/video0 -c isp_2dnr_mode=0	2dnr working mode. 0:maunal 1: auto must disable 2dnr and set mode, finally enable 2dnr.	int 0/1	ISP_CID_2DNR_MODE
hailo_ctrl -d /dev/video0 -C isp_2dnr_mode	2dnr working mode. 0:maunal 1: auto must disable 2dnr and set mode, finally enable 2dnr.	int 0/1	ISP_CID_2DNR_MODE
hailo_ctrl -d /dev/video0 -c isp_2dnr_pre_gamma_strength=127	Denoise pre-gama strength.	int 0~127	ISP_CID_2DNR_PRE_GAMMA_STRENGTH
hailo_ctrl -d /dev/video0 -C isp_2dnr_pre_gamma_strength	Denoise pre-gama strength.	int 0~127	ISP_CID_2DNR_PRE_GAMMA_STRENGTH
hailo_ctrl -d /dev/video0 -c isp_2dnr_strength=127	Denoise strength.	int 0~127	ISP_CID_2DNR_STRENGTH
hailo_ctrl -d /dev/video0 -C isp_2dnr_strength	Denoise strength.	int 0~127	ISP_CID_2DNR_STRENGTH
hailo_ctrl -d /dev/video0 -c isp_2dnr_sigma=100	Sigma for filter.	int 10~10000	ISP_CID_2DNR_SIGMA
hailo_ctrl -d /dev/video0 -C isp_2dnr_sigma	Sigma for filter.	int 10~10000	ISP_CID_2DNR_SIGMA

6.5. GC

Command	Description	Parameters	Control ID
hailo_ctrl -d /dev/video0 -c isp_gc_enable=0	Disable or enable gc.	int 0/1	ISP_CID_GC_ENABLE
hailo_ctrl -d /dev/video0 -c isp_gc_red_x_distance=<value>	Distance of x in gamma curve of red channel curve.	uint32 64x array 0~0x1F	ISP_CID_GC_R_X_DISTANCE
hailo_ctrl -d /dev/video0 -C isp_gc_red_x_distance	Distance of x in gamma curve of red channel curve.	uint32 64x array 0~0x1F	ISP_CID_GC_R_X_DISTANCE
hailo_ctrl -d /dev/video0 -c isp_gc_red_x_data=<value>	x data in gamma curve of red channel curve.	uint32 64x array 0~0xFFFF	VVCAM_ISP_CID_GC_R_X_DATA
hailo_ctrl -d /dev/video0 -C isp_gc_red_x_data	x data in gamma curve of red channel curve.	uint32 64x array 0~0xFFFF	VVCAM_ISP_CID_GC_R_X_DATA
hailo_ctrl -d /dev/video0 -c isp_gc_red_y_data=<value>	y data in gamma curve of red channel curve	uint32 64x array 0~0x3FF	ISP_CID_GC_R_Y_DATA
hailo_ctrl -d /dev/video0 -C isp_gc_red_y_data	y data in gamma curve of red channel curve	uint32 64x array 0~0x3FF	ISP_CID_GC_R_Y_DATA
hailo_ctrl -d /dev/video0 -c isp_gc_green_x_distance=<value>	Distance of x in gamma curve of green channel curve.	uint32 64x array 0~0x1F	ISP_CID_GC_G_X_DISTANCE
hailo_ctrl -d /dev/video0 -C isp_gc_green_x_distance	Distance of x in gamma curve of green channel curve.	uint32 64x array 0~0x1F	ISP_CID_GC_G_X_DISTANCE
hailo_ctrl -d /dev/video0 -c isp_gc_green_x_data=<value>	x data in gamma curve of green channel curve.	uint32 64x array 0~0xFFFF	ISP_CID_GC_G_X_DATA
hailo_ctrl -d /dev/video0 -C isp_gc_green_x_data	x data in gamma curve of green channel curve.	uint32 64x array 0~0xFFFF	ISP_CID_GC_G_X_DATA
hailo_ctrl -d /dev/video0 -c isp_gc_green_y_data=<value>	y data in gamma curve of green channel curve.	uint32 64x array 0~0x3FF	ISP_CID_GC_G_Y_DATA
hailo_ctrl -d /dev/video0 -C isp_gc_green_y_data	y data in gamma curve of green channel curve.	uint32 64x array 0~0x3FF	ISP_CID_GC_G_Y_DATA
hailo_ctrl -d /dev/video0 -c isp_gc_blue_x_distance=<value>	Distance of x in gamma curve of blue channel curve.	uint32 64x array 0~0x1F	ISP_CID_GC_B_X_DISTANCE
hailo_ctrl -d /dev/video0 -C isp_gc_blue_x_distance	Distance of x in gamma curve of blue channel curve.	uint32 64x array 0~0x1F	ISP_CID_GC_B_X_DISTANCE
hailo_ctrl -d /dev/video0 -c isp_gc_blue_x_data=<value>	x data in gamma curve of blue channel curve.	uint32 64x array 0~0xFFFF	ISP_CID_GC_B_X_DATA
hailo_ctrl -d /dev/video0 -C isp_gc_blue_x_data	x data in gamma curve of blue channel curve.	uint32 64x array 0~0xFFFF	ISP_CID_GC_B_X_DATA
hailo_ctrl -d /dev/video0 -c isp_gc_blue_y_data=<value>	y data in gamma curve of blue channel curve.	uint32 64x array 0~0x3FF	ISP_CID_GC_B_Y_DATA
hailo_ctrl -d /dev/video0 -C isp_gc_blue_y_data	y data in gamma curve of blue channel curve.	uint32 64x array 0~0x3FF	ISP_CID_GC_B_Y_DATA

6.6. WDR

Command	Description	Parameters	Control ID
hailo_ctrl -d /dev/video0 -c isp_wdr_enable=1	Disable or enable wdr.	int 0/1	ISP_CID_WDR_ENABLE
hailo_ctrl -d /dev/video0 -c isp_wdr_strength=26	wdr local strength.	int 0~128	ISP_CID_WDR_STRENGTH
hailo_ctrl -d /dev/video0 -C isp_wdr_strength	wdr local strength.	int 0~128	ISP_CID_WDR_STRENGTH
hailo_ctrl -d /dev/video0 -c isp_wdr_high_strength=26	wdr high strength.	int 0~128	ISP_CID_WDR_HIGH_STRENGTH
hailo_ctrl -d /dev/video0 -C isp_wdr_high_strength	wdr high strength.	int 0~128	ISP_CID_WDR_HIGH_STRENGTH
hailo_ctrl -d /dev/video0 -c isp_wdr_low_strength=100	wdr low strength.	int 0~255	ISP_CID_WDR_LOW_STRENGTH
hailo_ctrl -d /dev/video0 -C isp_wdr_low_strength	wdr low strength.	int 0~255	ISP_CID_WDR_LOW_STRENGTH
hailo_ctrl -d /dev/video0 -c isp_wdr_global_strength=52	wdr global strength.	int 0~128	ISP_CID_WDR_GLOBAL_STRENGTH
hailo_ctrl -d /dev/video0 -C isp_wdr_global_strength	wdr global strength.	int 0~128	ISP_CID_WDR_GLOBAL_STRENGTH
hailo_ctrl -d /dev/video0 -c isp_wdr_flat_strength=10	wdr flat area strength.	int 0~19	ISP_CID_WDR_FLAT_STRENGTH
hailo_ctrl -d /dev/video0 -C isp_wdr_flat_strength	wdr flat area strength.	int 0~19	ISP_CID_WDR_FLAT_STRENGTH
hailo_ctrl -d /dev/video0 -c isp_wdr_flat_threshold=10	wdr flat area threshold.	int 0~20	ISP_CID_WDR_FLAT_THRESHOLD
hailo_ctrl -d /dev/video0 -C isp_wdr_flat_threshold	wdr flat area threshold.	int 0~20	ISP_CID_WDR_FLAT_THRESHOLD
hailo_ctrl -d /dev/video0 -c isp_wdr_contrast=10	wdr contrast	Int - 1023~1023	ISP_CID_WDR_CONTRAST
hailo_ctrl -d /dev/video0 -C isp_wdr_contrast	wdr contrast	Int - 1023~1023	ISP_CID_WDR_CONTRAST

6.7. EE

Note: All controls must run in manual mode except enable/mode/reset control.

Command	Description	Parameters	Control ID
hailo_ctrl -d /dev/video0 -c isp_ee_enable=1	Disable or enable ee.	int 0/1	ISP_CID_EE_ENABLE
hailo_ctrl -d /dev/video0 -c isp_ee_reset=1	Reset ee, both 0 or 1 will reset it.	int 0/1	ISP_CID_EE_RESET
hailo_ctrl -d /dev/video0 -c isp_ee_mode=1	ee working mode: 0: manual 1: auto must disable ee, and then set mode, finally enable ee.	int 0/1	ISP_CID_EE_MODE
hailo_ctrl -d /dev/video0 -C isp_ee_mode	ee working mode: 0: manual 1: auto must disable ee, and then set mode, finally enable ee.	int 0/1	ISP_CID_EE_MODE
hailo_ctrl -d /dev/video0 -c isp_ee_edge_gain=536	Sensitivity of edge detection.	uint16 0~65535	ISP_CID_EE_EDGE_GAIN
hailo_ctrl -d /dev/video0 -C isp_ee_edge_gain	Sensitivity of edge detection.	uint16 0~65535	ISP_CID_EE_EDGE_GAIN
hailo_ctrl -d /dev/video0 -c isp_ee_strength=12	Strength of smooth filter.	uint8 0~128	ISP_CID_EE_STRENGTH
hailo_ctrl -d /dev/video0 -C isp_ee_strength	Strength of smooth filter.	uint8 0~128	ISP_CID_EE_STRENGTH
hailo_ctrl -d /dev/video0 -c isp_ee_uv_gain=512	Gain of chroma on edge.	uint16 0~65535	ISP_CID_EE_UV_GAIN
hailo_ctrl -d /dev/video0 -C isp_ee_uv_gain	Gain of chroma on edge.	uint16 0~65535	ISP_CID_EE_UV_GAIN
hailo_ctrl -d /dev/video0 -c isp_ee_y_gain_down=1200	Gain of luma on edge in dark part of the edge.	uint16 0~65535	ISP_CID_EE_Y_GAIN_DOWN
hailo_ctrl -d /dev/video0 -C isp_ee_y_gain_down	Gain of luma on edge in dark part of the edge.	uint16 0~65535	ISP_CID_EE_Y_GAIN_DOWN
hailo_ctrl -d /dev/video0 -c isp_ee_y_gain_up=10000	Gain of luma on edge in bright part of the edge.	uint16 0~65535	ISP_CID_EE_Y_GAIN_UP
hailo_ctrl -d /dev/video0 -C isp_ee_y_gain_up	Gain of luma on edge in bright part of the edge.	uint16 0~65535	ISP_CID_EE_Y_GAIN_UP

6.8. CPROC

Command	Description	Parameters	Control ID
hailo_ctrl -d /dev/video0 -c isp_cproc_enable=0	Disable or enable cproc.	int 0/1	ISP_CID_CPROC_ENABLE
hailo_ctrl -d /dev/video0 -C isp_cproc_enable	Disable or enable cproc.	int 0/1	ISP_CID_CPROC_ENABLE
hailo_ctrl -d /dev/video0 -c isp_cproc_chroma_out=1	cproc chroma out range type. difference mode has difference color conversion coefficient of matrix. 1: BT601 2: Full	int 1/2	ISP_CID_CPROC_CHROMA_OUT
hailo_ctrl -d /dev/video0 -C isp_cproc_chroma_out	cproc chroma out range type. difference mode has difference color conversion coefficient of matrix. 1: BT601 2: Full	int 1/2	ISP_CID_CPROC_CHROMA_OUT
hailo_ctrl -d /dev/video0 -c isp_cproc_luma_out=1	cproc luma out range type. 1: BT601 2: Full	int 1/2	ISP_CID_CPROC_LUMA_OUT
hailo_ctrl -d /dev/video0 -C isp_cproc_luma_out	cproc luma out range type. 1: BT601 2: Full	int 1/2	ISP_CID_CPROC_LUMA_OUT
hailo_ctrl -d /dev/video0 -c isp_cproc_luma_in=1	cproc luma in range type. 1: BT601 2: Full	int 1/2	ISP_CID_CPROC_LUMA_IN
hailo_ctrl -d /dev/video0 -C isp_cproc_luma_in	cproc luma in range type. 1: BT601 2: Full	int 1/2	ISP_CID_CPROC_LUMA_IN
hailo_ctrl -d /dev/video0 -c isp_cproc_brightness=120	cproc brightness.	Int - 128~127	ISP_CID_CPROC_BRIGHTNESS
hailo_ctrl -d /dev/video0 -C isp_cproc_brightness	cproc brightness.	Int - 128~127	ISP_CID_CPROC_BRIGHTNESS
hailo_ctrl -d /dev/video0 -c isp_cproc_saturation=1	cproc saturation.	int 0~199	ISP_CID_CPROC_SATURATION
hailo_ctrl -d /dev/video0 -C isp_cproc_saturation	cproc saturation.	int 0~199	ISP_CID_CPROC_SATURATION
hailo_ctrl -d /dev/video0 -c isp_cproc_hue=256	cproc hue.	int- 90~89	ISP_CID_CPROC_HUE
hailo_ctrl -d /dev/video0 -C isp_cproc_hue	cproc hue.	int- 90~89	ISP_CID_CPROC_HUE
hailo_ctrl -d /dev/video0 -c isp_cproc_contrast=30	cproc contrast	int 30~199	ISP_CID_CPROC_CONTRAST
hailo_ctrl -d /dev/video0 -C isp_cproc_contrast	cproc contrast	int 30~199	ISP_CID_CPROC_CONTRAST

6.9. CA

Note: all controls should run in manual mode except enable/mode control.

Command	Description	Parameters	Control ID
hailo_ctrl -d /dev/video0 -c isp_ca_enable=1	Disable or enable ca. This can only be enabled if ee is enabled. Additionally, dci and ca should be enabled together.	Int 0/1	ISP_CID_CA_ENABLE
hailo_ctrl -d /dev/video0 -c isp_ca_mode=0	ca working mode. 0: manual 1: auto must disable ca, and then set mode, finally enable ca.	int 0/1	ISP_CID_CA_MODE
hailo_ctrl -d /dev/video0 -C isp_ca_mode	ca working mode. 0: manual 1: auto must disable ca, and then set mode, finally enable ca.	int 0/1	ISP_CID_CA_MODE
hailo_ctrl -d /dev/video0 -c isp_ca_color_adjust_mode=1	ca curve adjust mode. 0: Brightness 1: Saturation 2: Brightness and Saturation	int 0/1/2	ISP_CID_CA_COLOR_ADJUST_MODE
hailo_ctrl -d /dev/video0 -C isp_ca_color_adjust_mode	ca curve adjust mode. 0: Brightness 1: Saturation 2: Brightness and Saturation	int 0/1/2	ISP_CID_CA_COLOR_ADJUST_MODE
hailo_ctrl -d /dev/video0 -c isp_ca_curve_type=1	ca curve type. 0: S 1: Parabolic	int 0/1	ISP_CID_CA_CURVE_TYPE
hailo_ctrl -d /dev/video0 -C isp_ca_curve_type	ca curve type. 0: S 1: Parabolic	int 0/1	ISP_CID_CA_CURVE_TYPE
hailo_ctrl -d /dev/video0 -c isp_ca_s_curve_inflection=15	S curve inflection point.	int 0~64	ISP_CID_CA_S_CURVE_INFLECTION
hailo_ctrl -d /dev/video0 -C isp_ca_s_curve_inflection	S curve inflection point.	int 0~64	ISP_CID_CA_S_CURVE_INFLECTION
hailo_ctrl -d /dev/video0 -c isp_ca_s_curve_exponent=20	S curve exponent.	int 0~30	ISP_CID_CA_S_CURVE_EXPONENT
hailo_ctrl -d /dev/video0 -C isp_ca_s_curve_exponent	S curve exponent.	int 0~30	ISP_CID_CA_S_CURVE_EXPONENT
hailo_ctrl -d /dev/video0 -c isp_ca_parabolic_curve_factor=100	Parabolic curve factor.	int 0~200	ISP_CID_CA_PARABOLIC_CURVE_FACTOR

6.10. CAC

Command	Description	Parameters	Control ID
hailo_ctrl -d /dev/video0 -c isp_cac_enable=1	Disable or enable cac.	int 0/1	ISP_CID_CAC_ENABLE
hailo_ctrl -d /dev/video0 -C isp_cac_enable	Disable or enable cac.	int 0/1	ISP_CID_CAC_ENABLE
hailo_ctrl -d /dev/video0 -C isp_cac_blue_linear	cac blue linear coefficient for radical blue shift calculation. Formula: (A_Red * r + B_Red * r^2 + C_Red * r^3).	int 256~256	ISP_CID_CAC_BLUE_LINEAR
hailo_ctrl -d /dev/video0 -C isp_cac_blue_square	cac blue square coefficient for radical blue shift calculation.	int 256~256	ISP_CID_CAC_BLUE_SQUARE
hailo_ctrl -d /dev/video0 -C isp_cac_blue_cubic	cac blue cubical coefficient for radical blue shift calculation.	int 256~256	ISP_CID_CAC_BLUE_CUBIC
hailo_ctrl -d /dev/video0 -C isp_cac_red_linear	cac red linear coefficient for radical red shift calculation.	int 256~256	ISP_CID_CAC_RED_LINEAR
hailo_ctrl -d /dev/video0 -C isp_cac_red_square	cac red square coefficient for radical red shift calculation.	int 256~256	ISP_CID_CAC_RED_SQUARE
hailo_ctrl -d /dev/video0 -C isp_cac_red_cubic	cac red cubical +coefficient for radical red shift calculation	int 256~256	ISP_CID_CAC_RED_CUBIC
hailo_ctrl -d /dev/video0 -C isp_cac_h_center_offset	cac horizontal center offset of image.	int 2047~2047	ISP_CID_CAC_H_CENTER_OFFSET
hailo_ctrl -d /dev/video0 -C isp_cac_v_center_offset	cac vertical center offset of image.	int - 2047~2047	ISP_CID_CAC_V_CENTER_OFFSET

6.11. 3DNR

Note: All controls must run in manual mode except for enable/mode/auto level control.

Command	Description	Parameters	Control ID
hailo_ctrl -d /dev/video0 -c isp_3dnr_enable=1	Disable or enable 3dnr.	int 0/1	ISP_CID_3DNR_ENABLE
hailo_ctrl -d /dev/video0 -C isp_3dnr_enable	Disable or enable 3dnr.	int 0/1	ISP_CID_3DNR_ENABLE
hailo_ctrl -d /dev/video0 -c isp_3dnr_mode=1	3dnr working mode. 0: manual 1: auto must disable 3dnr and then set mode, finally enable 3dnr.	int 0/1	ISP_CID_3DNR_MODE
hailo_ctrl -d /dev/video0 -C isp_3dnr_mode	3dnr working mode. 0: manual 1: auto must disable 3dnr and then set mode, finally enable 3dnr.	int 0/1	ISP_CID_3DNR_MODE
hailo_ctrl -d /dev/video0 -c isp_3dnr_auto_level=1	3dnr auto level. only work in auto mode.	int 0~100	ISP_CID_3DNR_MODE
hailo_ctrl -d /dev/video0 -C isp_3dnr_auto_level	3dnr auto level. only work in auto mode.	int 0~100	ISP_CID_3DNR_MODE
hailo_ctrl -d /dev/video0 -c isp_3dnr_delta_factor=536	Temporal delta.	uint16 0~65535	ISP_CID_3DNR_DELTA_FACTOR
hailo_ctrl -d /dev/video0 -C isp_3dnr_delta_factor	Temporal delta.	uint16 0~65535	ISP_CID_3DNR_DELTA_FACTOR
hailo_ctrl -d /dev/video0 -c isp_3dnr_motion_factor=12	Motion detection for denoise3d.	uint8 0~128	ISP_CID_3DNR_MOTION_FACTOR
hailo_ctrl -d /dev/video0 -C isp_3dnr_motion_factor	Motion detection for denoise3d.	uint8 0~128	ISP_CID_3DNR_MOTION_FACTOR
hailo_ctrl -d /dev/video0 -c isp_3dnr_strength=100	General strength of denoise3d.	uint16 0~128	ISP_CID_3DNR_STRENGTH
hailo_ctrl -d /dev/video0 -C isp_3dnr_strength	General strength of denoise3d.	uint16 0~128	ISP_CID_3DNR_STRENGTH

6.12. BLS

Command	Description	Parameters	Control ID
hailo_ctrl -d /dev/video0 -c isp_bls_red=64	Fixed black level for red pixels-signed.	uint16 0~4095	ISP_CID_BLS_RED
hailo_ctrl -d /dev/video0 -C isp_bls_red	Fixed black level for red pixels-signed.	uint16 0~4095	ISP_CID_BLS_RED
hailo_ctrl -d /dev/video0 -c isp_bls_green_red=64	Fixed black level for gr pixels-signed.	uint16 0~4095	ISP_CID_BLS_GREEN_RED
hailo_ctrl -d /dev/video0 -C isp_bls_green_red	Fixed black level for gr pixels-signed.	uint16 0~4095	ISP_CID_BLS_GREEN_RED
hailo_ctrl -d /dev/video0 -c isp_bls_green_blue=64	Fixed black level for gb pixels-signed.	uint16 0~4095	ISP_CID_BLS_GREEN_BLUE
hailo_ctrl -d /dev/video0 -C isp_bls_green_blue	Fixed black level for gb pixels-signed.	uint16 0~4095	ISP_CID_BLS_GREEN_BLUE
hailo_ctrl -d /dev/video0 -c isp_bls_blue=64	Fixed black level for blue pixels-signed.	uint16 0~4095	ISP_CID_BLS_BLUE
hailo_ctrl -d /dev/video0 -C isp_bls_blue	Fixed black level for blue pixels-signed.	uint16 0~4095	ISP_CID_BLS_BLUE

6.13. DMSC

Command	Description	Parameters	Control ID
hailo_ctrl -d /dev/video0 -c isp_dmsc_enable=1	Disable or enable dmsc.	int 0/1	ISP_CID_DMSC_ENABLE
hailo_ctrl -d /dev/video0 -c isp_dmsc_enable_demoire=1	Enable or disable demoire.	int 0/1	ISP_CID_DMSC_ENABLE_DEMOIRE
hailo_ctrl -d /dev/video0 -C isp_dmsc_enable_demoire	Enable or disable demoire.	int 0/1	ISP_CID_DMSC_ENABLE_DEMOIRE
hailo_ctrl -d /dev/video0 -c isp_dmsc_enable_depurples=1	Enable or disable demosaic depurples.	int 0/1	ISP_CID_DMSC_ENABLE_DEPURPLE
hailo_ctrl -d /dev/video0 -C isp_dmsc_enable_depurples	Enable or disable demosaic depurples.	int 0/1	ISP_CID_DMSC_ENABLE_DEPURPLE
hailo_ctrl -d /dev/video0 -c isp_dmsc_enable_sharpen=1	Enable or disable demosaic sharpen.	int 0/1	ISP_CID_DMSC_ENABLE_SHARPEN
hailo_ctrl -d /dev/video0 -C isp_dmsc_enable_sharpen	Enable or disable demosaic sharpen.	int 0/1	ISP_CID_DMSC_ENABLE_SHARPEN
hailo_ctrl -d /dev/video0 -c isp_dmsc_enable_sharpen_line=1	Enable or disable demosaic line sharpen.	int 0/1	ISP_CID_DMSC_ENABLE_SHARPEN_LINE
hailo_ctrl -d /dev/video0 -C isp_dmsc_enable_sharpen_line	Enable or disable demosaic line sharpen.	int 0/1	ISP_CID_DMSC_ENABLE_SHARPEN_LINE
hailo_ctrl -d /dev/video0 -c isp_dmsc_enable_skin=1	Enable or disable demosaic skin area filter.	int 0/1	ISP_CID_DMSC_ENABLE_SKIN
hailo_ctrl -d /dev/video0 -C isp_dmsc_enable_skin	Enable or disable demosaic skin area filter.	int 0/1	ISP_CID_DMSC_ENABLE_SKIN
hailo_ctrl -d /dev/video0 -c isp_dmsc_threshold=120	Demosaic threshold for bayer demosaicing texture.	int 0~255	ISP_CID_DMSC_THRESHOLD
hailo_ctrl -d /dev/video0 -C isp_dmsc_threshold	Demosaic threshold for bayer demosaicing texture.	int 0~255	ISP_CID_DMSC_THRESHOLD
hailo_ctrl -d /dev/video0 -c isp_dmsc_denoise_strength=16	Demosaic denoise strength for green channel smoothing filter.	int 0~32	ISP_CID_DMSC_DENOISE_STRENGTH
hailo_ctrl -d /dev/video0 -C isp_dmsc_denoise_strength	Demosaic denoise strength for green channel smoothing filter.	int 0~32	ISP_CID_DMSC_DENOISE_STRENGTH
hailo_ctrl -d /dev/video0 -c isp_dmsc_sharpen_size=8	Demosaic green channel sharpen size.	int 0~16	ISP_CID_DMSC_SHARPEN_SIZE
hailo_ctrl -d /dev/video0 -C isp_dmsc_sharpen_size	Demosaic green channel sharpen size.	int 0~16	ISP_CID_DMSC_SHARPEN_SIZE
hailo_ctrl -d /dev/video0 -c isp_dmsc_interpolat_dir_thr_max=2048	Demosaic direction select threshold value in bright area, the interpolation operation between maximum Threshold and minimal threshold.	int 0~4095	ISP_CID_DMSC_INTERPOLAT_DIR_THR_MAX
hailo_ctrl -d /dev/video0 -C isp_dmsc_interpolat_dir_thr_max	Demosaic direction select threshold value in bright area, the interpolation operation between maximum Threshold and minimal threshold.	int 0~4095	ISP_CID_DMSC_INTERPOLAT_DIR_THR_MAX
hailo_ctrl -d /dev/video0 -c isp_dmsc_interpolat_dir_thr_min=128	Demosaic direction select threshold value in bright area, the interpolation operation between maximum Threshold and minimal threshold.	int 0~4095	ISP_CID_DMSC_INTERPOLAT_DIR_THR_MIN
hailo_ctrl -d /dev/video0 -C isp_dmsc_interpolat_dir_thr_min	Demosaic direction select threshold value in bright area, the interpolation operation between maximum Threshold and minimal threshold.	int 0~4095	ISP_CID_DMSC_INTERPOLAT_DIR_THR_MIN
hailo_ctrl -d /dev/video0 -c isp_dmsc_demoire_thr_t1=0	Demosaic weighting curve of demoire interpolation results in output result, the curve X axis, first point.	int 0~255	ISP_CID_DMSC_DEMOIRE_THR_T1

Continued on next page

Table 10 – continued from previous page

Command	Description	Parameters	Control ID
hailo_ctrl -d /dev/video0 -C isp_dmsc_demoire_thr_t1	Demosaic weighting curve of demoire interpolation results in output result, the curve X axis, first point.	int 0~255	ISP_CID_DMSC_DEMOIRE_THR_T1
hailo_ctrl -d /dev/video0 -c isp_dmsc_demoire_thr_t2_shift=7	Demosaic weighting curve of demoire interpolation results in output result, the curve X axis, second point.	int 0~8	ISP_CID_DMSC_DEMOIRE_THR_T2_SHIFT
hailo_ctrl -d /dev/video0 -C isp_dmsc_demoire_thr_t2_shift	Demosaic weighting curve of demoire interpolation results in output result, the curve X axis, second point.	int 0~8	ISP_CID_DMSC_DEMOIRE_THR_T2_SHIFT
hailo_ctrl -d /dev/video0 -c isp_dmsc_demoire_thr_r1=0	Demosaic weighting curve of demoire interpolation results in output result, the curve Y axis, first point.	int 0~256	ISP_CID_DMSC_DEMOIRE_THR_R1
hailo_ctrl -d /dev/video0 -C isp_dmsc_demoire_thr_r1	Demosaic weighting curve of demoire interpolation results in output result, the curve Y axis, first point.	int 0~256	ISP_CID_DMSC_DEMOIRE_THR_R1
hailo_ctrl -d /dev/video0 -c isp_dmsc_demoire_thr_r2=128	Demosaic weighting curve of demoire interpolation results in output result, the curve Y axis, second point	int 0~256	ISP_CID_DMSC_DEMOIRE_THR_R2
hailo_ctrl -d /dev/video0 -C isp_dmsc_demoire_thr_r2	Demosaic weighting curve of demoire interpolation results in output result, the curve Y axis, second point	int 0~256	ISP_CID_DMSC_DEMOIRE_THR_R2
hailo_ctrl -d /dev/video0 -c isp_dmsc_demoire_edge_r1=0	Demosaic weight curve of prefer to find the direction interpolation in the process of demoire interpolation. The curve y axis, first point.	int 0~256	ISP_CID_DMSC_DEMOIRE_EDGE_R1
hailo_ctrl -d /dev/video0 -C isp_dmsc_demoire_edge_r1	Demosaic weight curve of prefer to find the direction interpolation in the process of demoire interpolation. The curve y axis, first point.	int 0~256	ISP_CID_DMSC_DEMOIRE_EDGE_R1
hailo_ctrl -d /dev/video0 -c isp_dmsc_demoire_edge_r2=128	Demosaic weight curve of prefer to find the direction interpolation in the process of demoire interpolation. The curve y axis, second point	int 0~256	ISP_CID_DMSC_DEMOIRE_EDGE_R2
hailo_ctrl -d /dev/video0 -C isp_dmsc_demoire_edge_r2	Demosaic weight curve of prefer to find the direction interpolation in the process of demoire interpolation. The curve y axis, second point	int 0~256	ISP_CID_DMSC_DEMOIRE_EDGE_R2
hailo_ctrl -d /dev/video0 -c isp_dmsc_demoire_edge_t1=0	Demosaic Weight curve of prefer to find the direction interpolation in the process of demoire interpolation. The curve Y axis, first point.	int 0~511	ISP_CID_DMSC_DEMOIRE_EDGE_T1
hailo_ctrl -d /dev/video0 -C isp_dmsc_demoire_edge_t1	Demosaic Weight curve of prefer to find the direction interpolation in the process of demoire interpolation. The curve Y axis, first point.	int 0~511	ISP_CID_DMSC_DEMOIRE_EDGE_T1
hailo_ctrl -d /dev/video0 -c isp_dmsc_demoire_edge_t2_shift=8	Demosaic weight curve of prefer to find the direction interpolation in the process of demoire interpolation. The curve x axis, second point, the Left shift value.	int 0~9	ISP_CID_DMSC_DEMOIRE_EDGE_T2_SHIFT

Continued on next page

Table 10 – continued from previous page

Command	Description	Parameters	Control ID
hailo_ctrl -d /dev/video0 -C isp_dmsc_demoire_edge_t2_shift	Demosaic weight curve of prefer to find the direction interpolation in the process of demoire interpolation. The curve x axis, second point, the Left shift value.	int 0~9	ISP_CID_DMSC_DEMOIRE_EDGE_T2_SHIFT
hailo_ctrl -d /dev/video0 -c isp_dmsc_demoire_area_thr=8	Demosaic demoire area threshold.	int 0~32	ISP_CID_DMSC_DEMOIRE_AREA_THR
hailo_ctrl -d /dev/video0 -C isp_dmsc_demoire_area_thr	Demosaic demoire area threshold.	int 0~32	ISP_CID_DMSC_DEMOIRE_AREA_THR
hailo_ctrl -d /dev/video0 -c isp_dmsc_demoire_sat_shrink=24	Demoire saturation shrink.	int 0~32	ISP_CID_DMSC_DEMOIRE_SAT_SHRINK
hailo_ctrl -d /dev/video0 -C isp_dmsc_demoire_sat_shrink	Demoire saturation shrink.	int 0~32	ISP_CID_DMSC_DEMOIRE_SAT_SHRINK
hailo_ctrl -d /dev/video0 -c isp_dmsc_sharpen_fac_black=20	Sharpen factor of black edge.	int 0~51	ISP_CID_DMSC_SHARPEN_FAC_BLACK
hailo_ctrl -d /dev/video0 -C isp_dmsc_sharpen_fac_black	Sharpen factor of black edge.	int 0~51	ISP_CID_DMSC_SHARPEN_FAC_BLACK
hailo_ctrl -d /dev/video0 -c isp_dmsc_sharpen_fac_white=14	Sharpen factor of white edge.	int 0~511	ISP_CID_DMSC_SHARPEN_FAC_WHITE
hailo_ctrl -d /dev/video0 -C isp_dmsc_sharpen_fac_white	Sharpen factor of white edge.	int 0~511	ISP_CID_DMSC_SHARPEN_FAC_WHITE
hailo_ctrl -d /dev/video0 -c isp_dmsc_sharpen_clip_black=200	Maximum sharpen level of black edge.	int 0~2047	ISP_CID_DMSC_SHARPEN_CLIP_BLACK
hailo_ctrl -d /dev/video0 -C isp_dmsc_sharpen_clip_black	Maximum sharpen level of black edge.	int 0~2047	ISP_CID_DMSC_SHARPEN_CLIP_BLACK
hailo_ctrl -d /dev/video0 -c isp_dmsc_sharpen_clip_white=2000	Maximum sharpen level of white edge.	int 0~2047	ISP_CID_DMSC_SHARPEN_CLIP_WHITE
hailo_ctrl -d /dev/video0 -C isp_dmsc_sharpen_clip_white	Maximum sharpen level of white edge.	int 0~2047	ISP_CID_DMSC_SHARPEN_CLIP_WHITE
hailo_ctrl -d /dev/video0 -c isp_dmsc_sharpen_thr_t1=0	Curve of sharpen level in different texture areas with X axis first point.	int 0~2047	ISP_CID_DMSC_SHARPEN_THR_T1
hailo_ctrl -d /dev/video0 -C isp_dmsc_sharpen_thr_t1	Curve of sharpen level in different texture areas with X axis first point.	int 0~2047	ISP_CID_DMSC_SHARPEN_THR_T1
hailo_ctrl -d /dev/video0 -c isp_dmsc_sharpen_thr_t2_shift=4	Curve of sharpen level in different texture areas with X axis second point, left shift value.	int 0~11	ISP_CID_DMSC_SHARPEN_THR_T2_SHIFT
hailo_ctrl -d /dev/video0 -C isp_dmsc_sharpen_thr_t2_shift	Curve of sharpen level in different texture areas with X axis second point, left shift value.	int 0~11	ISP_CID_DMSC_SHARPEN_THR_T2_SHIFT
hailo_ctrl -d /dev/video0 -c isp_dmsc_sharpen_thr_t3=1024	Curve of sharpen level in different texture areas with X axis third point.	int 0~2047	ISP_CID_DMSC_SHARPEN_THR_T3
hailo_ctrl -d /dev/video0 -C isp_dmsc_sharpen_thr_t3	Curve of sharpen level in different texture areas with X axis third point.	int 0~2047	ISP_CID_DMSC_SHARPEN_THR_T3
hailo_ctrl -d /dev/video0 -c isp_dmsc_sharpen_thr_t4_shift=10	Curve of sharpen level in different texture areas with X axis fourth point, left shift value.	int 0~11	ISP_CID_DMSC_SHARPEN_THR_T4_SHIFT
hailo_ctrl -d /dev/video0 -C isp_dmsc_sharpen_thr_t4_shift	Curve of sharpen level in different texture areas with X axis fourth point, left shift value.	int 0~11	ISP_CID_DMSC_SHARPEN_THR_T4_SHIFT
hailo_ctrl -d /dev/video0 -c isp_dmsc_sharpen_r1=20	Curve of sharpen level in different texture areas with y axis first point.	int 0~256	ISP_CID_DMSC_SHARPEN_R1
hailo_ctrl -d /dev/video0 -C isp_dmsc_sharpen_r1	Curve of sharpen level in different texture areas with y axis first point.	int 0~256	ISP_CID_DMSC_SHARPEN_R1

Continued on next page

Table 10 – continued from previous page

Command	Description	Parameters	Control ID
hailo_ctrl -d /dev/video0 -c isp_dmsc_sharpen_r2=100	Curve of sharpen level in different texture areas with y axis second point.	int 0~256	ISP_CID_DMSC_SHARPEN_R2
hailo_ctrl -d /dev/video0 -C isp_dmsc_sharpen_r2	Curve of sharpen level in different texture areas with y axis second point.	int 0~256	ISP_CID_DMSC_SHARPEN_R2
hailo_ctrl -d /dev/video0 -c isp_dmsc_sharpen_r3=120	Curve of sharpen level in different texture areas with y axis third point.	Int 0~256	ISP_CID_DMSC_SHARPEN_R3
hailo_ctrl -d /dev/video0 -C isp_dmsc_sharpen_r3	Curve of sharpen level in different texture areas with y axis third point.	Int 0~256	ISP_CID_DMSC_SHARPEN_R3
hailo_ctrl -d /dev/video0 -c isp_dmsc_sharpen_line_r1=120	Curve of the overall enhancement of the line stroke. The larger the curve, the more texture will be considered as a line and enhanced, first point y Axis	int 0~256	ISP_CID_DMSC_SHARPEN_LINE_R1
hailo_ctrl -d /dev/video0 -C isp_dmsc_sharpen_line_r1	Curve of the overall enhancement of the line stroke. The larger the curve, the more texture will be considered as a line and enhanced, first point y Axis	int 0~256	ISP_CID_DMSC_SHARPEN_LINE_R1
hailo_ctrl -d /dev/video0 -c isp_dmsc_sharpen_line_r2=51	Curve of the overall enhancement of the line stroke. The larger the curve, the more texture will be considered as a line and enhanced, second point Y axis	Int 0~256	ISP_CID_DMSC_SHARPEN_LINE_R2
hailo_ctrl -d /dev/video0 -C isp_dmsc_sharpen_line_r2	Curve of the overall enhancement of the line stroke. The larger the curve, the more texture will be considered as a line and enhanced, second point Y axis	Int 0~256	ISP_CID_DMSC_SHARPEN_LINE_R2
hailo_ctrl -d /dev/video0 -c isp_dmsc_sharpen_line_strength=426	The strength of line sharpen.	int 0~4095	ISP_CID_DMSC_SHARPEN_LINE_STRENGTH
hailo_ctrl -d /dev/video0 -C isp_dmsc_sharpen_line_strength	The strength of line sharpen.	int 0~4095	ISP_CID_DMSC_SHARPEN_LINE_STRENGTH
hailo_ctrl -d /dev/video0 -c isp_dmsc_sharpen_line_shift1=0	Curve of the overall enhancement of the line stroke. The larger the curve, the more texture will be considered as a line and enhanced, first point x Axis left shift value.	int 0~10	ISP_CID_DMSC_SHARPEN_LINE_SHIFT1
hailo_ctrl -d /dev/video0 -C isp_dmsc_sharpen_line_shift1	Curve of the overall enhancement of the line stroke. The larger the curve, the more texture will be considered as a line and enhanced, first point x Axis left shift value.	int 0~10	ISP_CID_DMSC_SHARPEN_LINE_SHIFT1
hailo_ctrl -d /dev/video0 -c isp_dmsc_sharpen_line_shift2=0	Curve of the overall enhancement of the line stroke. The larger the curve, the more texture will be considered as a line and enhanced, second point X axis left shift value.	int 0~10	ISP_CID_DMSC_SHARPEN_LINE_SHIFT2
hailo_ctrl -d /dev/video0 -C isp_dmsc_sharpen_line_shift2	Curve of the overall enhancement of the line stroke. The larger the curve, the more texture will be considered as a line and enhanced, second point X axis left shift value.	int 0~10	ISP_CID_DMSC_SHARPEN_LINE_SHIFT2

Continued on next page

Table 10 – continued from previous page

Command	Description	Parameters	Control ID
hailo_ctrl -d /dev/video0 -c isp_dmesc_sharpen_line_t1=0	Curve of the overall enhancement of the line stroke. The larger the curve, the more texture will be considered as a line and enhanced, x axis start Point.	Int 0~2047	ISP_CID_DMESC_SHARPEN_LINE_T1
hailo_ctrl -d /dev/video0 -C isp_dmesc_sharpen_line_t1	Curve of the overall enhancement of the line stroke. The larger the curve, the more texture will be considered as a line and enhanced, x axis start Point.	Int 0~2047	ISP_CID_DMESC_SHARPEN_LINE_T1
hailo_ctrl -d /dev/video0 -C isp_dmesc_sharpen_filter_00	3x3 sharpen filter kernel Filter[0][0].	int 0~34	ISP_CID_DMESC_SHARPEN_FILTER_00
hailo_ctrl -d /dev/video0 -C isp_dmesc_sharpen_filter_01	3x3 sharpen filter kernel Filter[0][1].	int 0~34	ISP_CID_DMESC_SHARPEN_FILTER_01
hailo_ctrl -d /dev/video0 -C isp_dmesc_sharpen_filter_02	3x3 sharpen filter kernel Filter[0][2].	int 0~34	ISP_CID_DMESC_SHARPEN_FILTER_02
hailo_ctrl -d /dev/video0 -C isp_dmesc_sharpen_filter_10	3x3 sharpen filter kernel Filter[1][0].	Int 0~34	ISP_CID_DMESC_SHARPEN_FILTER_10
hailo_ctrl -d /dev/video0 -C isp_dmesc_sharpen_filter_11	3x3 sharpen filter kernel Filter[1][1].	int 0~34	ISP_CID_DMESC_SHARPEN_FILTER_11
hailo_ctrl -d /dev/video0 -C isp_dmesc_sharpen_filter_12	3x3 sharpen filter kernel Filter[1][2].	int 0~34	ISP_CID_DMESC_SHARPEN_FILTER_12
hailo_ctrl -d /dev/video0 -C isp_dmesc_sharpen_filter_20	3x3 sharpen filter kernel Filter[2][0].	int 0~34	ISP_CID_DMESC_SHARPEN_FILTER_20
hailo_ctrl -d /dev/video0 -C isp_dmesc_sharpen_filter_21	3x3 sharpen filter kernel Filter[2][1].	int 0~34	ISP_CID_DMESC_SHARPEN_FILTER_21
hailo_ctrl -d /dev/video0 -C isp_dmesc_sharpen_filter_22	3x3 sharpen filter kernel Filter[2][2].	int 0~34	ISP_CID_DMESC_SHARPEN_FILTER_22
hailo_ctrl -d /dev/video0 -c isp_dmesc_depurple_sat_shrink=5	Force saturation compression in the purple fringe area.	int 0~8	ISP_CID_DMESC_DEPURPLE_SAT_SHRINK
hailo_ctrl -d /dev/video0 -C isp_dmesc_depurple_sat_shrink	Force saturation compression in the purple fringe area.	int 0~8	ISP_CID_DMESC_DEPURPLE_SAT_SHRINK
hailo_ctrl -d /dev/video0 -c isp_dmesc_depurple_cbcrc_mode=2	Purple fringe and red fringe remove enable mode. 0: off 1: red 2: blue 3:redblue	int 0~3	ISP_CID_DMESC_DEPURPLE_CBCRC_MODE
hailo_ctrl -d /dev/video0 -C isp_dmesc_depurple_cbcrc_mode	Purple fringe and red fringe remove enable mode. 0: off 1: red 2: blue 3:redblue	int 0~3	ISP_CID_DMESC_DEPURPLE_CBCRC_MODE
hailo_ctrl -d /dev/video0 -c isp_dmesc_depurple_threshold=0	Judgment threshold of purple fringe area.	int 0~255	ISP_CID_DMESC_DEPURPLE_THRESHOLD
hailo_ctrl -d /dev/video0 -C isp_dmesc_depurple_threshold	Judgment threshold of purple fringe area.	int 0~255	ISP_CID_DMESC_DEPURPLE_THRESHOLD
hailo_ctrl -d /dev/video0 -c isp_dmesc_skin_cb_thr_max=0	Maximum threshold of skin color area in Cb color space.	int 0~4095	ISP_CID_DMESC_SKIN_CB_THR_MAX
hailo_ctrl -d /dev/video0 -C isp_dmesc_skin_cb_thr_max	Maximum threshold of skin color area in Cb color space.	int 0~4095	ISP_CID_DMESC_SKIN_CB_THR_MAX
hailo_ctrl -d /dev/video0 -c isp_dmesc_skin_cb_thr_min=0	Minimal threshold of skin color area in Cb color space.	int 0~4095	ISP_CID_DMESC_SKIN_CB_THR_MIN
hailo_ctrl -d /dev/video0 -C isp_dmesc_skin_cb_thr_min	Minimal threshold of skin color area in Cb color space.	int 0~4095	ISP_CID_DMESC_SKIN_CB_THR_MIN
hailo_ctrl -d /dev/video0 -c isp_dmesc_skin_cr_thr_max=0	Maximum threshold of skin color area in Cr color space.	int 0~4095	ISP_CID_DMESC_SKIN_CR_THR_MAX
hailo_ctrl -d /dev/video0 -C isp_dmesc_skin_cr_thr_max	Maximum threshold of skin color area in Cr color space.	int 0~4095	ISP_CID_DMESC_SKIN_CR_THR_MAX
hailo_ctrl -d /dev/video0 -c isp_dmesc_skin_cr_thr_min=0	Minimal threshold of skin color area.	int 0~4095	ISP_CID_DMESC_SKIN_CR_THR_MIN
hailo_ctrl -d /dev/video0 -C isp_dmesc_skin_cr_thr_min	Minimal threshold of skin color area.	int 0~4095	ISP_CID_DMESC_SKIN_CR_THR_MIN
hailo_ctrl -d /dev/video0 -c isp_dmesc_skin_y_thr_max=0	maximum threshold of skin color area in Y color space.	int 0~4095	ISP_CID_DMESC_SKIN_Y_THR_MAX

Continued on next page

Table 10 – continued from previous page

Command	Description	Parameters	Control ID
hailo_ctrl -d /dev/video0 -C isp_dmesc_skin_y_thr_max	maximum threshold of skin color area in Y color space.	int 0~4095	ISP_CID_DMESC_SKIN_Y_THR_MAX
hailo_ctrl -d /dev/video0 -c isp_dmesc_skin_y_thr_min=0	Minimal threshold of skin color area in Y color space.	int 0~4095	ISP_CID_DMESC_SKIN_Y_THR_MIN
hailo_ctrl -d /dev/video0 -C isp_dmesc_skin_y_thr_min	Minimal threshold of skin color area in Y color space.	int 0~4095	ISP_CID_DMESC_SKIN_Y_THR_MIN

6.14. DPCC

Command	Description	Parameters	Control ID
hailo_ctrl -d /dev/video0 -c isp_dpcc_enable=0	Disable or enable dpcc.	int 0/1	ISP_CID_DPCC_ENABLE
hailo_ctrl -d /dev/video0 -c isp_dpcc_set_index=1	dpcc set index for configuration array. must disable dpcc and then set index, finally enable dpcc.	int 1~6	ISP_CID_DPCC_SET_INDEX
hailo_ctrl -d /dev/video0 -C isp_dpcc_set_index	dpcc set index for configuration array. must disable dpcc and then set index, finally enable dpcc.	int 1~6	ISP_CID_DPCC_SET_INDEX

6.15. DPF

Note: All controls must disable, and then set control and finally enable.

Command	Description	Parameters	Control ID
hailo_ctrl -d /dev/video0 -c isp_dpf_enable=1	Enable or disable dpf.	int 0/1	ISP_CID_DPF_ENABLE
hailo_ctrl -d /dev/video0 -c isp_dpf_green_sigma=24	Sigma value for green pixel.	Uint 1~255	ISP_CID_DPF_GREEN_SIGMA
hailo_ctrl -d /dev/video0 -C isp_dpf_green_sigma	Sigma value for green pixel.	Uint 1~255	ISP_CID_DPF_GREEN_SIGMA
hailo_ctrl -d /dev/video0 -c isp_dpf_red_blue_sigma=24	Sigma value for red/blue pixel.	Uint 1~255	ISP_CID_DPF_RED_BLUE_SIGMA
hailo_ctrl -d /dev/video0 -C isp_dpf_red_blue_sigma	Sigma value for red/blue pixel.	Uint 1~255	ISP_CID_DPF_RED_BLUE_SIGMA
hailo_ctrl -d /dev/video0 -c isp_dpf_gradient=540	Gradient for calculating strength.	int 0~12800	ISP_CID_DPF_GRADIENT
hailo_ctrl -d /dev/video0 -C isp_dpf_gradient	Gradient for calculating strength.	int 0~12800	ISP_CID_DPF_GRADIENT
hailo_ctrl -d /dev/video0 -c isp_dpf_offset=540	Additive offset for calculating strength.	int 12800~12800	ISP_CID_DPF_OFFSET
hailo_ctrl -d /dev/video0 -C isp_dpf_offset	Additive offset for calculating strength.	int 12800~12800	ISP_CID_DPF_OFFSET
hailo_ctrl -d /dev/video0 -c isp_dpf_bound_min=540	Min value for strength.	int 0~12800	ISP_CID_DPF_BOUND_MIN
hailo_ctrl -d /dev/video0 -C isp_dpf_bound_min	Min value for strength.	int 0~12800	ISP_CID_DPF_BOUND_MIN
hailo_ctrl -d /dev/video0 -c isp_dpf_division_factor=658	Division factor for calculating strength.	int 0~6400	ISP_CID_DPF_DIVISION_FACTOR
hailo_ctrl -d /dev/video0 -C isp_dpf_division_factor	Division factor for calculating strength.	int 0~6400	ISP_CID_DPF_DIVISION_FACTOR

6.16. LSC

Note: All controls must disable, and then set control and finally enable.

Command	Description	Parameters	Control ID
hailo_ctrl -d /dev/video0 -c isp_lsc_enable=1	Enable or disable lsc.	int 0/1	ISP_CID_LSC_ENABLE
hailo_ctrl -d /dev/video0 --set-vector isp_lsc_x_grad_tbl=[468,474,496,511,461,385,312,297]	Factor for x-gradient calculation.	uint16 8x 0~0xFFFF	ISP_CID_LSC_X_GRAD_TBL
hailo_ctrl -d /dev/video0 -C isp_lsc_x_grad_tbl	Factor for x-gradient calculation.	uint16 8x 0~0xFFFF	ISP_CID_LSC_X_GRAD_TBL
hailo_ctrl -d /dev/video0 --set-vector isp_lsc_y_grad_tbl=[819,819,712,712,655,606,546,511]	Factor for y-gradient calculation.	uint16 8x 0~0xFFFF	ISP_CID_LSC_Y_GRAD_TBL
hailo_ctrl -d /dev/video0 -C isp_lsc_y_grad_tbl	Factor for y-gradient calculation.	uint16 8x 0~0xFFFF	ISP_CID_LSC_Y_GRAD_TBL
hailo_ctrl -d /dev/video0 --set-vector isp_lsc_x_size_tbl=[70,69,66,64,71,85,105,110]	Sector size in x-direction.	uint16 8x 0~0x3FF	ISP_CID_LSC_X_SIZE_TBL
hailo_ctrl -d /dev/video0 -C isp_lsc_x_size_tbl	Sector size in x-direction.	uint16 8x 0~0x3FF	ISP_CID_LSC_X_SIZE_TBL
hailo_ctrl -d /dev/video0 --set-vector isp_lsc_y_size_tbl=[40,40,46,46,50,54,60,64]	Sector size in y-direction.	uint16 8x 0~0x3FF	ISP_CID_LSC_Y_SIZE_TBL
hailo_ctrl -d /dev/video0 -C isp_lsc_y_size_tbl	Sector size in y-direction.	uint16 8x 0~0x3FF	ISP_CID_LSC_Y_SIZE_TBL
hailo_ctrl -d /dev/video0 -c isp_lsc_r_data_tbl=<value>	Correction factor for R color component.	uint16 289x 0~0xFFFF	ISP_CID_LSC_R_DATA_TBL
hailo_ctrl -d /dev/video0 -C isp_lsc_r_data_tbl	Correction factor for R color component.	uint16 289x 0~0xFFFF	ISP_CID_LSC_R_DATA_TBL
hailo_ctrl -d /dev/video0 -c isp_lsc_gr_data_tbl=<value>	Correction factor for Gr color component.	uint16 289x 0~0xFFFF	ISP_CID_LSC_GR_DATA_TBL
hailo_ctrl -d /dev/video0 -C isp_lsc_gr_data_tbl	Correction factor for Gr color component.	uint16 289x 0~0xFFFF	ISP_CID_LSC_GR_DATA_TBL
hailo_ctrl -d /dev/video0 -c isp_lsc_gb_data_tbl=<value>	Correction factor for Gb color component.	uint16 289x 0~0xFFFF	ISP_CID_LSC_GB_DATA_TBL
hailo_ctrl -d /dev/video0 -C isp_lsc_gb_data_tbl	Correction factor for Gb color component.	uint16 289x 0~0xFFFF	ISP_CID_LSC_GB_DATA_TBL
hailo_ctrl -d /dev/video0 -c isp_lsc_b_data_tbl=<value>	Correction factor for B color component.	uint16 289x 0~0xFFFF	ISP_CID_LSC_B_DATA_TBL
hailo_ctrl -d /dev/video0 -C isp_lsc_b_data_tbl	Correction factor for B color component.	uint16 289x 0~0xFFFF	ISP_CID_LSC_B_DATA_TBL

6.17. DCI

Command	Description	Parameters	Control ID
hailo_ctrl -d /dev/video0 -c isp_dci_enable=1	Disable or enable dci. This can only be enabled if ee is enabled. Additionally, dci and ca should be enabled together.	int 0/1	ISP_CID_DCI_ENABLE
hailo_ctrl -d /dev/video0 -C isp_dci_enable	Disable or enable dci.	int 0/1	ISP_CID_DCI_ENABLE
hailo_ctrl -d /dev/video0 -c isp_dci_mode=1	dci working mode. 0: manual 1: auto must disable dci and then set control, finally enable dci.	int 0/1	ISP_CID_DCI_MODE
hailo_ctrl -d /dev/video0 -C isp_dci_mode	dci working mode. 0: manual 1: auto must disable dci and then set control, finally enable dci.	int 0/1	ISP_CID_DCI_MODE
hailo_ctrl -d /dev/video0 -c isp_dci_curve_mode=1	dci curve mode. 0: 2 sections 1: 3 sections	int 0/1	ISP_CID_DCI_CURVE_MODE
hailo_ctrl -d /dev/video0 -C isp_dci_curve_mode	dci curve mode. 0: 2 sections 1: 3 sections	int 0/1	ISP_CID_DCI_CURVE_MODE
hailo_ctrl -d /dev/video0 -c isp_dci_curve_high_exponent=100	dci curve high section exponent.	int 0~200	ISP_CID_DCI_CURVE_HIGH_EXPONENT
hailo_ctrl -d /dev/video0 -C isp_dci_curve_high_exponent	dci curve high section exponent.	int 0~200	ISP_CID_DCI_CURVE_HIGH_EXPONENT
hailo_ctrl -d /dev/video0 -c isp_dci_curve_low_exponent=12	dci curve low section exponent.	int 0~200	ISP_CID_DCI_CURVE_LOW_EXPONENT
hailo_ctrl -d /dev/video0 -C isp_dci_curve_low_exponent	dci curve low section exponent.	int 0~200	ISP_CID_DCI_CURVE_LOW_EXPONENT
hailo_ctrl -d /dev/video0 -c isp_dci_curve_middle_exponent=12	dci curve middle section exponent.	int 0~200	ISP_CID_DCI_CURVE_MIDDLE_EXPONENT
hailo_ctrl -d /dev/video0 -C isp_dci_curve_middle_exponent	dci curve middle section exponent.	int 0~200	ISP_CID_DCI_CURVE_MIDDLE_EXPONENT
hailo_ctrl -d /dev/video0 -c isp_dci_curve_start_x=0	dci curve start of x axis.	uint16 0	ISP_CID_DCI_CURVE_START_X
hailo_ctrl -d /dev/video0 -C isp_dci_curve_start_x	dci curve start of x axis.	uint16 0	ISP_CID_DCI_CURVE_START_X
hailo_ctrl -d /dev/video0 -c isp_dci_curve_start_y=0	dci curve start of y axis.	uint16 0~1024	ISP_CID_DCI_CURVE_START_Y
hailo_ctrl -d /dev/video0 -C isp_dci_curve_start_y	dci curve start of y axis.	uint16 0~1024	ISP_CID_DCI_CURVE_START_Y
hailo_ctrl -d /dev/video0 -c isp_dci_curve_inflection_x=20	dci curve first inflection of x axis.	uint16 0~64	ISP_CID_DCI_CURVE_INFLECTION_X
hailo_ctrl -d /dev/video0 -C isp_dci_curve_inflection_x	dci curve first inflection of x axis.	uint16 0~64	ISP_CID_DCI_CURVE_INFLECTION_X
hailo_ctrl -d /dev/video0 -c isp_dci_curve_inflection_y=100	dci curve first inflection of y axis.	uint16 0~1024	ISP_CID_DCI_CURVE_INFLECTION_Y
hailo_ctrl -d /dev/video0 -C isp_dci_curve_inflection_y	dci curve first inflection of y axis.	uint16 0~1024	ISP_CID_DCI_CURVE_INFLECTION_Y
hailo_ctrl -d /dev/video0 -c isp_dci_curve_inflection2_x=40	dci curve second inflection of x axis.	uint16 0~64	ISP_CID_DCI_CURVE_INFLECTION2_X
hailo_ctrl -d /dev/video0 -C isp_dci_curve_inflection2_x	dci curve second inflection of x axis.	uint16 0~64	ISP_CID_DCI_CURVE_INFLECTION2_X
hailo_ctrl -d /dev/video0 -c isp_dci_curve_inflection2_y=200	dci curve second inflection of y axis.	uint16 0~1024	ISP_CID_DCI_CURVE_INFLECTION2_Y
hailo_ctrl -d /dev/video0 -C isp_dci_curve_inflection2_y	dci curve second inflection of y axis.	uint16 0~1024	ISP_CID_DCI_CURVE_INFLECTION2_Y
hailo_ctrl -d /dev/video0 -c isp_dci_curve_end_x=64	dci curve end of x axis.	uint16 64	ISP_CID_DCI_CURVE_END_X
hailo_ctrl -d /dev/video0 -C isp_dci_curve_end_x	dci curve end of x axis.	uint16 64	ISP_CID_DCI_CURVE_END_X
hailo_ctrl -d /dev/video0 -c isp_dci_curve_end_y=500	dci curve end of y axis.	uint16 0~1024	ISP_CID_DCI_CURVE_END_Y
hailo_ctrl -d /dev/video0 -c isp_dci_curve_end_y	dci curve end of y axis.	uint16 0~1024	ISP_CID_DCI_CURVE_END_Y

7. Automatic Control Module Configuration

7.1. Introduction

3aconfig.json has the following functions:

- Controls whether each module loads and turns on tuning parameters.
- Dynamically configures parameters for different ISP modules.
- Configures parameters for different hardware versions of the same module.
- Adjusts parameters dynamically in real time. The modified parameters are loaded into the ISP code in real time. Customers can quickly tune image effects through this file.

7.2. 3aconfig.json Structure

3aconfig creates different classes under the root node corresponding with the hardware versions of each ISP module through the class name as shown in the example below.

```
"root" : [  
  {  
    "classname" : "Aev2",  
    "enable" : false,  
    "motionThreshold" : 0.5,  
    "roiWeight" : 0.6,  
    "motionFilter" : 0.5,  
    "maxSensorAgain" : 15.5,  
    "maxSensorDgain" : 1,  
    "maxIspDgain" : 12  
  },  
  {  
    "classname" : "Aehdr",  
    "enable" : false,  
    "motionThreshold" : 0.5,  
    "roiWeight" : 0.6,  
    "motionFilter" : 0.5,  
    "maxSensorAgain" : 15.5,  
    "maxSensorDgain" : 1,  
    "maxIspDgain" : 12  
  },  
]
```

3aconfig.json Parameters

classname

The classname parameter is used to distinguish the different automatic control modules. This parameter can also distinguish different hardware versions for the same module.

```
"classname" : "ACproc",  
"classname" : "Aeev1",  
"classname" : "A3dnrv1",  
"classname" : "A3dnrv3"
```

forcecreate

When the forcecreate parameter's value is true, the class of the automatic control module in the code block is forced to be created during initialization. When its value is false, if the value of enable is also false, the class of the module to which the code block belongs is not created.

```
"classname" : "ADpf",
"forcecreate" : true,
```

enable

The enable parameter is used to dynamically turn on or off the automatic control function of the automatic control module. If the value of forcecreate is false, enable must also be false. When enable=false, the module is still active, but is not being automatically updated.

```
"classname" : "ADpf",
"forcecreate" : true,
"enable" : false,
"disable" : false,
```

disable

The disable parameter is used to disable the module. When disable = true - the module is inactive.

```
"classname" : "ADpf",
"forcecreate" : true,
"enable" : false,
"disable" : false,
```

tables

The tables parameters are different for each module, but it contains two identical parameter lists:

- **Linear mode** - When the value of hdr is false, it means that this group of parameters is in the linear mode.
- **HDR mode** - When the value of hdr is true, it means that it is in the HDR mode.

These parameters are open for users to tune by each module.

```
"tables" : [
  {
    "hdr" : false,
    "gains" : [1, 128, 200, 233, 400],
    "bls" : [240, 240, 240, 240, 240]
  },
  {
    "hdr" : true,
    "gains" : [1, 128, 200, 240, 400],
    "bls" : [240, 240, 240, 240, 240]
  }
]
```

Automatic control parameter configuration

Different automatic control module tables must contain two identical code blocks. Each code block must contain the two variables hdr and gains. hdr is used to distinguish whether ISP works in linear mode or HDR mode. The gains parameter is very important. Since different sensor gains have different effects on the image quality, it is necessary to dynamically adjust the parameters of each module according to the sensor gain. The gains parameter is used to configure parameters in different exposure environments. Gains is a one-dimensional array whose length can be adjusted. The longer the parameter, the finer the control. If the length of gains is confirmed, the length of other parameters must also remain with the length of gains.

```
"tables" : [
  {
    "hdr" : false,
    "gains" : [1, 4, 16, 64, 256, 512],
```

(continues on next page)

(continued from previous page)

```

    "strength" : [2, 2, 2, 2, 4, 8],
    "sigma" : [2, 3.0, 4.0, 6.0, 8.0, 12.0],
    "blend_static" : [10, 10, 10, 10, 20, 30],
    "blend_motion" : [80, 80, 80, 80, 80, 80]
  },
  {
    "hdr" : true,
    "gains" : [1, 4, 16, 64, 256, 512],
    "strength" : [2, 2, 2, 2, 2, 2],
    "sigma" : [3, 5.0, 8.0, 13.0, 20.0, 24.0],
    "blend_static" : [10, 10, 10, 10, 20, 30],
    "blend_motion" : [80, 80, 80, 80, 80, 80]
  }
]

```

Adding a New Automatic Control Module

Adding a new automatic control module requires adding a new code block under the root node.

- The code block must contain classname, forcecreate, enable, disable, and tables.
- The tables label must contain two code blocks with the same parameters.
- Each code block must contain hdr, gains, and other parameters of each code block under tables that are the tunable parameters of the automatic control module. If the automatic control module has parameters that do not need interpolation, these parameters can be directly added to tables.

```

{
  "classname" : "newname",
  "forcecreate" : true,
  "enable" : false,
  "disable" : false,
  "Parameter3" : true,
  "Parameter4" : true,
  ...
  "tables" : [
    {
      "hdr" : false,
      "gains" : [1, 24, 128, 256, 512],
      "Parameter1" : [1, 24, 128, 256, 512],
      "Parameter2" : [1, 24, 128, 256, 512],
      "Parameter3" : [1, 24, 128, 256, 512],
      ...
    },
    {
      "hdr" : true,
      "gains" : [1, 24, 128, 256, 512],
      "Parameter1" : [1, 24, 128, 256, 512],
      "Parameter2" : [1, 24, 128, 256, 512],
      "Parameter3" : [1, 24, 128, 256, 512],
      ...
    }
  ]
},

```

Dynamic Modification of Parameters

Parameters of each module may be dynamically modified. The automatic control module will parse the 3aconfig.json file in a loop, so when the modified parameters are saved, they are written to the ISP in the next frame as shown in the pseudocode below:

```

void loopReload() {
  while (bRunning) {

```

(continues on next page)

(continued from previous page)

```

sleep(1); //loading parameters every 1s
string strInputFileName = "3aconfig.json";
ifstream configStream(strInputFileName);
Json::Reader reader;
Json::Value rootNode;

bool parseOk = reader.parse(configStream, rootNode, false);
if (!parseOk) {
    //... error message
    exit(1);
}
{
    Json::Value root = rootNode["root"];
    if (!root.isArray()) exit(1);
    // ... load the modules
}
}
}

```

Achieving Automatic Control

Gain: In the automatic control module, the current sensor gain (Again x Dgain) will be obtained from the AE control process in real time.

The Gain table is used to dynamically interpolate our parameters according to the current ambient sensor gain. The length of a single parameter table should be consistent with the length of the Gain table.

If the parameters of the module need to be adjusted more carefully, the length of the gain table may be increased. and the step size of the gain may be reduced. It should be noted that the first parameter of the gain table needs to fill in the minimum total gain supported by the sensor, and the last parameter needs to write the maximum gain supported by the sensor.

Automatic Control via exposure instead of gain

Instead of dynamically interpolating our parameters according to the current ambient sensor gain, some modules allow for interpolation according to exposure (gain * integration_time).

The interpolation is done in the exact same way as described above, but the gain table is replaced with an exposure table. The length of the exposure table must be consistent with the length of the gain table.

The user should either define "gains" or "exposures" for each module.

It is valid for one module to use "gains", and another module to use "exposures".

As an example, the following line can be switched to use exposure instead of gain:

```
"gains" : [1, 2, 4],
```

It can be changed to the following:

```
"exposures" : [0.005, 0.01, 0.02],
```

Note that integration time unit is seconds, hence the values of "exposures" are generally lower.

The user should tune the "exposures" parameters.

The modules that support this option explicitly state that in their documentation.

2D Interpolation automatic control

Some modules support 2D interpolation, which allows for more complex parameter adjustments based on two variables: gain/exposure and dynamic range (DR) level.

The specific modules that support this option specifically state that in their documentation.

The dynamic range is integer between 0 and 2: 0=bright, 1=HDR, 2=dark, and is automatically determined by the AE module based on the current scene.

The 2d interpolation is optional, and if not used, the module will use 1D interpolation based on gain/exposure only.

Example for converting from 1D interpolation to 2D interpolation:

```
"gains" : [1, 2, 4, 8],
"strength" : [1, 2, 3, 5]
```

This can be converted to:

```
"gains" : [1, 2, 4, 8],
"strength" : [
  [1, 2, 3, 5], // row of dynamic_range=0
  [2, 3, 4, 6], // row of dynamic_range=1
  [3, 4, 5, 7] // row of dynamic_range=2
]
```

Another example: conversion of 1D vector interpolation:

```
"gains" : [1, 2, 4, 8],
"curve" : [ [1,2,3], [4,5,6], [6,7,8], [9,10,11] ]
```

To convert, the user should add another dimension (so 3 dimensions overall), such that after the 2D interpolation, a vector will still be chosen:

```
"gains" : [1, 2, 4, 8],
"curve" : [
  [ [1,2,3], [4,5,6], [6,7,8], [1,10,15] ], // row of dynamic_range=0
  [ [2,3,4], [5,6,7], [7,8,9], [2,16,12] ], // row of dynamic_range=1
  [ [3,4,5], [6,7,8], [8,9,10], [3,10,13] ] // row of dynamic_range=2
]
```

For clarification, it works with both gain and exposure as one dimension (and dynamic range as the other dimension), that is, if both exposure interpolation and 2d interpolation are supported for that module.

Conflicting Modules

It is possible to define multiple modules which are in conflict with each other. In that case the user must make sure to define the file in such a way that only one of the modules is active at a time. For example, AEv1, AEv2, and AdaptiveAE are all different versions of the auto exposure control module, and all control the exposure parameters, such that only one of them may be active at a time.

8. Automatic Control Parameters

8.1. AdaptiveAe

Adaptive Automatic Exposure control. There can only be one Active AE module at a time. 'Enable' state cannot be modified in runtime

Parameter	Description	Type	Range
enable	Enable adaptive Auto exposure. Cannot be modified in runtime.	Boolean	True/false
metaDataEnable	Enable meta data	Boolean	True/False
motionThreshold	motion threshold	float	[0.0,1.0]
roiWeight	Roi weight	float	[0.0,1.0]
motionFilter	Motion filter	float	[0.0,1.0]
targetFilter	Target filter	float	[0.0,1.0]
wdrContrast.min	minimal WDR contrast level	int	[0,255]
wdrContrast.max	Maximal WDR contrast level	int	[0,255]
lowlight.linear_gain	Gain levels for linear repress table	Vector<int>	[0,65535]
low-light.linear_repress	Linear repress table	Vec-tor<float>	[0.0,1.0]
lowlight.hdr_repress	hdr repress table	Vec-tor<float>	[0.0,1.0]
lowlight.hdr_gain	Gain levels for hdr repress table	Vector<int>	[0,65535]
maxSensorAgain	Maximal sensor analogue gain (scalar). Actual limit depends on sensor	int	[1,65535]
maxSensorDgain	Maximal sensor digital gain (scalar). Actual limit depends on sensor	int	[1,65535]
maxIspDgain	Maximal ISP digital gain (scalar)	int	[1,256]
priority	Priority routing table for auto exposure (relevant in SDR mode)	Priority Entry	
priorityHdr	Priority routing table for auto exposure in HDR mode. The exposure values represent the long exposure	Priority Entry	
delay	Auto-Exposure delay in #frames after changing shutter, gain, iris, respectively	Vector<int>	[0,255]
flickerPeriod	Anti banding period: 0=OFF, 1=50Hz, 2=60Hz	int	[0,2]
hcg	Dual Conversion Gain: set true for HCG (High Conversion Gain) mode, false for LCG (Low Conversion Gain) mode	Boolean	True/False
minAbsLockDiff	Locking threshold for AE. Minimal abs diff between current and previous exposure result.	float	[0.0, 255]

8.2. Priority Entry

Entries will have the structure: `[[[Tmin, Tmax],[Gmin,Gmax], [Imin, Imax], [p0, p1,p2]]]`
 For example:
 `[[0.000038, 0.015],[500,500], [1, 1], ["shutter", "gain", "iris"]]`,
 `[[0.000015, 0.033],[500,3900], [1, 1], ["shutter", "gain", "iris"]]`

Parameter	Description	Type
Tmin	Minimal shutter duration in us (if priorityHdr - for long exposure)	float
Tmax	Maximal shutter duration in us (if priorityHdr - for long exposure)	float
Gmin	Minimal gain (if priorityHdr - for long exposure) - scalar multiplier	int
Gmax	Maximal gain (if priorityHdr - for long exposure) - scalar multiplier	int
Imin	Minimal fnum (iris = $i/fnum^2$)	float
Imax	Maximal fnum (iris = $i/fnum^2$)	float

Note: when in HDR (priorityHdr field), the gain & shutter time entries in AdaptiveAE will impact the long exposure. The gain & shutter time for the short / VS exposures will be derived from it using the stitching ratio.

8.3. HDRAdaptiveAE

Adaptive Automatic Exposure control, designed for HDR. There can only be one Active AE module at a time. 'Enable' state cannot be modified in runtime

Parameter	Description	Type	Range
enable	Enable HDR adaptive Auto exposure. Cannot be modified in runtime.	Boolean	True/false
metaDataEnable	Enable meta data	Boolean	True/False
motionThreshold	motion threshold	float	[0.0,1.0]
roiWeight	Roi weight	float	[0.0,1.0]
motionFilter	Motion filter	float	[0.0,1.0]
targetFilter	Target filter	float	[0.0,1.0]
darkImagePercentage	Lower % threshold below which the image is considered dark for SetPoint adjustment	float	[0,100]
brightImagePercentage	Upper % threshold above which the image is considered bright for SetPoint adjustment	float	[0,100]
lowlight.hdr_repress	hdr repress table	Vector<float>	[0.0,1.0]
lowlight.hdr_gain	Gain levels for hdr repress table	Vector<int>	[0,65535]
maxSensorAgain	Maximal sensor analogue gain (scalar). Actual limit depends on sensor	int	[1,65535]
maxSensorDgain	Maximal sensor digital gain (scalar). Actual limit depends on sensor	int	[1,65535]
maxIspDgain	Maximal ISP digital gain (scalar)	int	[1,256]
priority	Priority routing table for auto exposure (relevant in SDR mode)	Priority Entry	
priorityHdr	Priority routing table for auto exposure in HDR mode. The exposure values represent the long exposure	Priority Entry	
delay	Auto-Exposure delay in #frames after changing shutter, gain, iris, respectively	Vector<int>	[0,255]
flickerPeriod	Anti banding period: 0=OFF, 1=50Hz, 2=60Hz	int	[0,2]
hcg	Dual Conversion Gain: set true for HCG (High Conversion Gain) mode, false for LCG (Low Conversion Gain) mode	Boolean	True/False
minAbsLockDiff	Locking threshold for AE. Minimal abs diff between current and previous exposure result.	float	[0.0, 255]

8.4. Awbv2

Automatic white balancing control. There can only be one Active AWB module at a time.

Parameter	Description	Type	Range
enable	Enable auto white balance v2. Cannot be modified in runtime	Boolean	True/false
illuorder	A string representing illumination profiles names and order	string	String of illuminations s.a. "A, D50, D65, D75, TL84, F12, CWF"
indoor	Lookup table of light types indoor state	Vector<float>	[0,1]
outdoor	Lookup table of light types outdoor state	Vector<float>	[0,1]
transition	Lookup table of light types transition state	Vector<float>	[0,1]
colorTemperature	Color temperature of each illumination profile	Vector<int>	[0,65535]
avg	Gray world weight	float	[0.0,1.0]
gains	Gain levels for CCM desaturation	Vector<int>	[0,65535]
desaturationMode	Desaturation mode: 0 - disable, 1 - apply desaturation to the 3x3 matrix, 2 - apply desaturation to the entire CCM transform including matrix and offsets	int	[0,2]
ccmSaturation	CCM saturation matrix where rows = number of entries in illuorder, columns = number of entries in gains, each value is a float between 0.0 (full desaturation) and 1.0 (full saturation)	Vector<Vector<float>	[0.0,1.0]

8.5. Afv1

Automatic focus control. Feature will be supported in future software release

Parameter	Description	Type	Range
enable	Enable auto Auto focus v1. Cannot be modified in runtime	Boolean	True/false
stableThreshold	Stable focus threshold	float	0.0-1.0
stableTolerance	Stable Threshold Tolerance	float	0.0~1.0
maxFocal	Maximum focal	Uint16	[0-65535]
minFocal	Minimal focal	Uint16	[0-65535]
step	Focus step	Int	[0~65535]
pdMseTolerance	PD MSE tolerance, relevant only with PD	Float	0.0~1.0
pdConfThreshold	Pd threshold, relevant only with PD	Int	[0-65535]

8.6. AGamma64

Auto update of RGB gamma curve. Note that curve can be 2D or 3D, depending on whether we interpolate according to dynamic range in addition to gain/exposure

Parameter	Description	Type	Range
enable	Enable auto update of gamma curve. When false, other parameters are ignored	Boolean	True/false
disable	Disable gamma curve.	Boolean	True/false
gains	(Obsolete - prefer using exposures) Gain levels for for gamma adjustment	Vector<int>	[0,65535]
exposures	Exposure (gain * integration_time) levels for wide dynamic range	Vec-tor<float>	[0,65535]
curve	gamma curve per gain level and dynamic range	3D/2D: vec-tor<vector<vector<int>>>	[0,1023]

8.7. ACproc

Auto update of color processing parameters

Parameter	Description	Type	Range
enable	Enable auto update of color processing. When false, other parameters are ignored	Boolean	True/false
disable	Disable color processing.	Boolean	True/false
gains	Gain levels for for color processing	Vector<int>	[0,65535]
contrast	Contrast level	Vec-tor<float>	[0.3,1.99]
bright	Brightness level	Vector<int>	[-128,127]
saturation	Saturation level	Vec-tor<float>	[0.0,1.99]
hue	hue level	Vec-tor<float>	[-0.9,0.9]
chroma_out	color output range	Vector<int>	1-BT601, 2-full range
luma_in	luma input range	Vector<int>	1-BT601, 2-full range
luma_out	luma input range	Vector<int>	1-BT601, 2-full range

8.8. Aeev1

Auto update of Edge enhancement parameters. The Aeev1 entry also contains the color adjustment as a sub entry with its own enable field. This means that when Aee is disabled, Auto update of color adjustment parameters is also disabled, though color adjustment can be disabled without disabling Aee

Parameter	Description	Type	Range
enable	Enable auto update of edge enhancement. When false, other parameters are ignored	Boolean	True/false
disable	Disable edge enhancement.	Boolean	True/false
ca_mode	Color adjustment mode	int	[0,2]
gains	Gain levels for for edge enhancement	Vector<int>	[0,65535]
ee_strength	Edge enhancement strength	Vector<int>	[0,65535]
ee_src_strength	Edge enhancement source strength	Vector<int>	[0,65535]
ee_y_up_gain	Gain for bright parts of the edge	Vector<int>	[0,65535]
ee_y_down_gain	Gain for dark parts of the edge	Vector<int>	[0,65535]
ee_uv_gain	Gain for chroma component of the edge	Vector<int>	[0,65535]
ee_edge_gain	Gain for the edge	Vector<int>	[0,65535]
ca_enable	Enable ca curve per gain	Vector<int>	[0,65535]
ca_curve	ca_curve per gain	Vec-tor<Vector<int>>	[0,1024]
dci_enable	Enable contrast curve per gain	Vector<int>	[0,65535]
dci_curve	contrast curve per gain	Vec-tor<Vector<int>>	[0,1024]

8.9. AWdrv4

Auto update of wide dynamic range module. The values are being 2d interpolated using exposures and dynamic range of an image. Note that fields marked '1D/2D' means that it can either be a matrix or vector, depending on whether to interpolate dynamic range

Parameter	Description	Type	Range
enable	Enable auto update of of wide dynamic range. When false, other parameters are ignored	Boolean	True/false
disable	Disable wide dynamic range.	Boolean	True/false
gains	(Obsolete - prefer using exposures) Gain levels for wide dynamic range	Vector<int>	[0,65535]
exposures	Exposure (gain * integration_time) levels for wide dynamic range	Vec-tor<float>	[0,65535]
strength	wdr strength	1D/2D: Vec-tor<vector<int>>	[0,128]
high_strength	wdr strength for bright parts	1D/2D: vec-tor<vector<int>>	[0,128]
low_strength	wdr strength for dark parts	1D/2D: vec-tor<vector<int>>	[0,255]
global_strength	wdr global strength	1D/2D: vec-tor<vector<int>>	[0,128]
contrast	contrast level	1D/2D: vec-tor<vector<int>>	[-1023,1023]
flat_strength	strength of flat areas	1D/2D: vec-tor<vector<int>>	[0, 19]
flat_thr	Threshold for areas considered flat (no texture)	1D/2D: vec-tor<vector<int>>	[0, 20]

8.10. A3dnrv1

Auto update of 3dnr - temporal denoising

Parameter	Description	Type	Range
enable	Enable auto update of 3dnr. When false, other parameters are ignored	Boolean	True/false
disable	Disable wide dynamic range. Equivalent to strength=0	Boolean	True/false
gains	Gain levels for wide dynamic range	Vector<int>	[0,65535]
strength	Denoise strength	Vector<int>	[0,128]
motion_inv	Inverted motion factor	Vector<int>	[0,100000]
delta	Denoise delta	Vector<int>	[0,1024]

8.11. A2dnrv5

Auto update of 2dnr - spatial denoising

Parameter	Description	Type	Range
enable	Enable auto update of 2dnr. When false, other parameters are ignored	Boolean	True/false
disable	Disable 2dnr. Equivalent to strength=0	Boolean	True/false
gains	Gain levels for wide dynamic range	Vector<int>	[0,65535]
strength	Denoise strength	Vector<int>	[0,127]
sigma	Denoise sigma (STD)	Vec-tor<float>	[0.01,100.0]
blend_static	Static blending parameter	Vector<int>	[0,127]
blend_motion	Motion blending parameter	Vector<int>	[0,127]

8.12. ADpf

Auto update of denoise prefilter

Parameter	Description	Type	Range
enable	Enable Auto update of dpf. When false, other parameters are ignored	Boolean	True/false
disable	Disable dpf	Boolean	True/false
gains	Gain levels for auto denoise prefilter	Vector<int>	[0,65535]
gradient	Denoise strength	Vec-tor<float>	[0,127]
offset	Filter offset	Vec-tor<float>	[-128.0,128.0]
min	Filter bound minimum	Vec-tor<float>	[0.0,128.0]
div	Division factor	Vec-tor<float>	[0,64.0]
sigma_g	Green channel sigma	Vector<int>	[1,255]
sigma_rb	Red and blue channels sigma	Vector<int>	[1, 255]
noise_curve	Denoising curve (low pass filter)	Vec-tor<Vector<int>>	[1,4096]

8.13. ABIs

Auto update of black level subtraction.

Parameter	Description	Type	Range
enable	Enable Auto update of black level subtraction. When false, other parameters are ignored	Boolean	True/false
disable	Disable dpf	Boolean	True/false
gains	Gain levels for black level subtraction	Vector<int>	[0,65535]
bls	Black level subtraction per channel	Vec-tor<Vector<int>>	[0,4095]

8.14. ADmscv2

Auto update of demosaic v2. Demosaic, Sharpening, Depurple.

Parameter	Description	Type	Range
enable	Enable Auto update of dmisc. When false, other parameters are ignored	Boolean	True/false
disable	Disable dmisc module	Boolean	True/false
gains	Gain levels for ddmsc	Vector<int>	[0,65535]
dmisc_sharpen_enable	Sharpening control per gain level	Vector<int>	[0,1]
dmisc_depurple_enable	Depurple control per gain level	Vector<int>	[0,1]
dmisc_sharpen_factor_black	Sharpening factor for black	Vector<int>	[0,511]
dmisc_sharpen_factor_white	Sharpening factor for white	Vector<int>	[0,511]
dmisc_sharpen_clip_black	Sharpening clipping for black	Vector<int>	[0,2047]
dmisc_sharpen_clip_white	Sharpening clipping for white	Vector<int>	[0,2047]
dmisc_sharpen_t1	Sharpening threshold 1	Vector<int>	[0,2047]
dmisc_sharpen_t2_shift	Sharpening threshold 2 shift	Vector<int>	[0,11]
dmisc_sharpen_t3	Sharpening threshold 3	Vector<int>	[0,2047]
dmisc_sharpen_t4_shift	Sharpening threshold 4 shift	Vector<int>	[0,11]
dmisc_denoise_strength	Denoise strength in demosaic	Vector<int>	[0,32]
dmisc_sharpen_size	Sharpening size	Vector<int>	[0,16]
dmisc_sharpen_r1	Sharpening r1	Vector<int>	[0,256]
dmisc_sharpen_r2	Sharpening r3	Vector<int>	[0,256]
dmisc_sharpen_r3	Sharpening r3	Vector<int>	[0,256]
dmisc_depurple_sat_shrink	Saturation shrink for depurple	Vector<int>	[0,8]
dmisc_depurple_cbc_r_mode	Depurple cbc_r mode	Vector<int>	[0,3]
dmisc_depurple_thr	Depurple threshold	Vector<int>	[0,255]

8.15. ACprocPost

Auto update of post-isp color-processing

Parameter	Description	Type	Range
enable	Enable Auto update of ACprocPost	Boolean	True/false
disable	Disable ACprocPost	Boolean	True/false
auto_luma	Whether to automatically stretch Y channel (true = automatic Y stretch, false = manual)	Boolean	True/false
histogram_equalization	Whether to apply histogram equalization on Y channel	Boolean	True/false
gains	Gain levels	Vector<int>	[0,65535]
denoise_method	The denoise filter family to use	String	gaussian/bilateral
blur	Blur level of image (higher value means more blurred)	Vector<int>	[0,4]
bilateral_sigma	Bilateral filter sigma in the color space. Higher values mean that farther colors within the pixel neighborhood will be mixed together, resulting in larger areas of similar color.	Vector<int>	[0,255]
sharpness_level	The spatial extent of the sharpness filter. Higher values result in finer texture being sharpened	Vector<int>	[0,3]
sharpness_amount	The strength of edge enhancement. Higher values mean stronger sharpening	Vector<float>	[0.001,3.999]
sharpness_threshold	The threshold for edge detection for the sharpness filter. Lower values mean more texture but also more noise	Vector<int>	[0,128]
histogram_equalization_clip_threshold	The threshold for clipping in the histogram equalization algorithm.	Vector<float>	[0.0,20.0]
histogram_equalization_alpha	The coefficient between the previous LUT value (last frame) and the new calculated LUT value (this frame)	Vector<float>	[0.0,0.999]
saturation	Saturation (U&V channels) stretching factor	Vector<float>	[0,2]
manual_contrast	Manual contrast (Y channel) stretching factor (1=no change, 2=double contrast)	Vector<float>	[0.3,1.999]
manual_brightness	Brightness (Y channel) addition (0=no change, positive=brighter image)	Vector<int>	[-128,127]
auto_percentile_low	Auto luma stretches Y to fit values between the low and high percentile of Y value. This is the low percentile.	Vector<float>	[0.001,99.999]
auto_percentile_high	Auto luma stretches Y to fit values between the low and high percentile of Y value. This is the high percentile.	Vector<float>	[0.001,99.999]
auto_target_low	Auto luma stretches Y percentiles to fit between the low and high target bit numbers. This is the low target	Vector<int>	[0,255]
auto_target_high	Auto luma stretches Y percentiles to fit between the low and high target bit numbers. This is the high target	Vector<int>	[0,255]
auto_low_pass_filter_alpha	Auto luma applies low-pass filter on calculated brightness and contrast values from current and previous frames and biases the previous values. This is the bias	Vector<float>	[0.0,1.0]

8.16. ALsc

Auto update of lens shading correction. If Active, Lens shading correction will adjust according to reported zoom level.

Parameter	Description	Type	Range
enable	Enable Auto update of lens shading correction	Boolean	True/false
disable	Disable Lsc	Boolean	True/false